

Rodolfo Javier Agosto

Ingeniero en Sistemas Informáticos
Desarrollador Backend Java

Documentación funcional y técnica del sistema, incluyendo análisis, casos de uso, arquitectura y diagramas UML.

www.rodolfoagosto.dev

- 2020 –

Contenido

1. DESCRIPCIÓN GLOBAL DEL PRODUCTO 6

1.1 PROPÓSITO 6
1.2.1 GESTIÓN DE COMPRAS 7
1.2.2 GESTIÓN DE ENTREGAS 8
1.2.3 VALUACIÓN DE INVENTARIO 9

2. ESPECIFICACIÓN FUNCIONAL 11

2.1 ESPECIFICACIÓN PROCESO DE COMPRAS 11
2.1.1 IDENTIFICACIÓN DE ROLES INTERVINIENTES 11
2.1.2 DESCRIPCIÓN FUNCIONAL DEL PROCESO: ENTRADA / COMPORTAMIENTO / SALIDA 12
2.1.3 DIAGRAMA DE PROCESO 13
2.1.4 MODELO CONCEPTUAL 14
2.2 ESPECIFICACIÓN PROCESO DE ENTREGA 14
2.2.1 IDENTIFICACIÓN DE ROLES INTERVINIENTES 14
2.2.2 DESCRIPCIÓN FUNCIONAL DEL PROCESO: ENTRADA / COMPORTAMIENTO / SALIDA 15
2.2.3 DIAGRAMA DE PROCESO 16
2.2.4 MODELO CONCEPTUAL 17
2.3 ESPECIFICACIÓN PROCESO DE VALUACIÓN DE INVENTARIO 17
2.3.1 IDENTIFICACIÓN DE ROLES INTERVINIENTES 17
2.3.2 DESCRIPCIÓN FUNCIONAL DEL PROCESO: ENTRADA / COMPORTAMIENTO / SALIDA 17
2.3.3 DIAGRAMA DE PROCESO 18
2.3.4 MODELO CONCEPTUAL 18

3. ESPECIFICACIONES DE CASOS DE USO 20

3.1 GENERAR NOTA DE PEDIDO CU01.01 20
3.1.1 ESPECIFICACIÓN 20
3.1.2 GRÁFICO DE CASO DE USO 21
3.1.3 DIAGRAMA DE CLASES 22
3.1.4 DIAGRAMA DE SECUENCIA – GENERAR NOTA DE PEDIDO 23
3.1.5 DIAGRAMA DE SECUENCIA – CALCULAR RESERVADAS 24
3.1.6 DIAGRAMA DE SECUENCIA – CALCULANDO PENDIENTES DE ENTREGA 25
3.1.7 DIAGRAMA DE SECUENCIA – CALCULANDO PENDIENTES DE INGRESO 26
3.1.8 DER 27
3.2 INGRESAR REMITO PROVEEDOR CU01.02 28
3.2.1 ESPECIFICACIÓN 28
3.2.2 GRÁFICO DE CASO DE USO 29
3.2.3 DIAGRAMA DE CLASES 30
3.2.4 DIAGRAMA DE SECUENCIA 31
3.2.5 DER 32
3.3 CONTROLANDO REMITO CU01.03 33
3.3.1 ESPECIFICACIÓN 33
3.3.2 GRÁFICO DE CASO DE USO 34
3.3.3 DIAGRAMA DE CLASES 34
3.3.4 DIAGRAMA DE SECUENCIA 35
3.3.5 DER 36
3.4 CONFIRMANDO REMITO PROVEEDOR CU01.04 37
3.4.1 ESPECIFICACIÓN 37
3.4.2 GRÁFICO DE CASO DE USO 38
3.4.3 DIAGRAMA DE CLASES 38
3.4.4 DIAGRAMA DE SECUENCIA 39
3.4.5 DER 40
3.5 CONFIRMANDO ENVÍO CU02.01 41

3.5.1 Especificación 41
3.5.2 Gráfico de Caso de uso 42
3.5.3 Diagrama de Clases 43
3.5.4 Diagrama de Secuencia Confirmando Envío 44
3.5.5 Diagrama de Secuencia Generando Remitos 45
3.5.6 Diagrama de Secuencia Registrando Movimientos 46
3.5.7 Diagrama de Secuencia Actualizando Pedido de Cliente 47
3.5.8 DER 48
3.6 Generando Valuación CU03.02 49
3.6.1 Especificación 49
3.6.2 Gráfico de Caso de uso 50
3.6.3 Diagrama de Clases 51
3.6.4 Diagrama de Secuencia 52
3.6.5 DER 53

4. ASPECTOS TÉCNICOS QUE PROVEE EL SISTEMA DE INFORMACIÓN 55

4.1 Gestión de Perfiles de Usuario 55
4.2 Gestión de LogIn / LogOut del Sistema 62
4.3 Gestión de Múltiples Idiomas 65
4.4 Gestión de Bitácora 68
4.5 Gestión de Backup 71
4.6 Gestión de Dígitos Verificadores 73
4.7 Gestión de Encriptado 78
4.8 Esquema de persistencia 79

5. MAPA DE NAVEGACIÓN 81

6. INTERFACES DE USUARIO 82

6.1 P1-Principal 82
6.2 P11-LogIn 82
6.3 P1111-Consultar Bitácora 83
6.4 P1112-Idioma 83
6.5 P11131-Perfiles 84
6.6 P111311-Nuevo Perfil 84
6.7 P11132-Usuarios 85
6.8 P1114-Generar BackUp 85
6.9 P1121-Proveedores 86
6.10 P1122-Nota de Pedido 86
6.11 P1123-Remito de Compra 87
6.12 P1131-Cliente 87
6.13 P1132-Pedido de Cliente 88
6.14 P1133-Remito 88
6.15 P1134-Generar Entrega 89
6.16 P1135-Autorizar Entrega 89
6.17 P1136-Stock 90
6.18 P1141-Articulo 90
6.19 P1142-Lista de Precios 91
6.20 P1143-Valuacion de Inventario 91
6.21 P115-Cambiar Idioma 92
6.22 P116-Ayuda 92

7. DIAGRAMA GLOBAL DE CLASES DEL SISTEMA 93

7.1 Capa: BE 93
7.2 Capa: BLL Técnica 94
7.3 Capa: BLL Negocio 95

7.4 CAPA: DAL 96
7.5 CAPA: SEGURIDAD 97

8. ARQUITECTURA 98

9. DER GLOBAL DEL SISTEMA 99

Descripción Global del Producto

1. Descripción Global del Producto

1.1 Propósito

El sistema **Auzonia** estará directamente vinculado con tres procesos estratégicos del negocio, considerados críticos para el desempeño general de la compañía: **Gestión de Compras**, **Gestión de Entrega** de Mercadería y **Valuación de Inventarios**.

Gestión de Compras

En la actualidad, para realizar un pedido de mercadería al proveedor, los responsables de la gestión (Jefe de Ventas y Gerente General) deben revisar manualmente diversos documentos asociados tanto a pedidos de clientes como a órdenes de compra previas. Este procedimiento manual es necesario para proyectar la demanda futura de cada producto y evitar la adquisición de cantidades excesivas. Sin embargo, el proceso resulta lento, repetitivo y propenso a errores de estimación, ya que implica la verificación de múltiples documentos de distinta naturaleza.

El sistema Auzonia automatizará este flujo, permitiendo parametrizar las cantidades estimadas a solicitar para cada artículo en función de la demanda registrada, el historial de ventas y los niveles de stock. De esta manera, la gestión de compras será más ágil, precisa y eficiente.

Asimismo, cuando el proveedor envíe un pedido, el sistema permitirá registrar su recepción de manera que el ingreso de mercadería se refleje automáticamente en el stock y se actualicen los pedidos al proveedor en curso, descontando las cantidades recibidas para cada artículo. Si bien el sistema actual contempla el ingreso de mercadería, la actualización de pedidos previos se realiza manualmente, lo que aumenta la probabilidad de errores operativos. Auzonia resolverá este problema mediante una integración automatizada de ambos procesos, minimizando el margen de error humano y optimizando los tiempos de gestión.

Entrega de Mercadería

Actualmente, los vendedores organizan los pedidos mediante notas confeccionadas manualmente, en las cuales detallan los artículos solicitados por cada cliente, las cantidades, los datos de facturación, la mercadería reservada y otra información relevante.

Este procedimiento manual genera inconsistencias, duplicidades y errores en la carga de datos, afectando la precisión y trazabilidad de los pedidos.
El sistema Auzonia permitirá digitalizar y centralizar la gestión de pedidos, facilitando su organización y reduciendo el margen de error de los usuarios. Cada pedido quedará registrado en forma estructurada, garantizando la integridad de la información y la disponibilidad en tiempo real para las distintas áreas involucradas.

Cuando un vendedor planifica un recorrido para entregar la mercadería a sus clientes, debe solicitar al Jefe de Ventas la autorización para el retiro de los artículos correspondientes. Dicho jefe debe evaluar el stock disponible, la mercadería reservada y la pendiente de recepción de proveedores antes de aprobar la cantidad a entregar. En el sistema actual, este proceso resulta tedioso y poco eficiente, debido a la falta de visibilidad consolidada del estado de cada artículo.

El nuevo sistema Auzonia automatizará la validación de stock y reservas, permitiendo al Jefe de Ventas autorizar o ajustar entregas de manera ágil y segura, con la información actualizada al instante.

Finalmente, la coordinación de entregas involucra a múltiples actores —vendedor, jefe de ventas, encargado de depósito y jefe de playa—, lo cual, en el modelo actual, genera retrasos, descoordinación e inconsistencias cuando se gestionan simultáneamente varios pedidos. Auzonia integrará estos procesos en un flujo automatizado de tareas, garantizando una gestión eficiente, trazable y sincronizada entre todas las áreas participantes.

Valuación de Inventario

Desde el punto de vista contable y financiero, el Gerente General necesita disponer de una herramienta confiable para valorizar la mercadería existente en los depósitos. El sistema Auzonia deberá incorporar un módulo de Valuación de Inventarios que permita calcular la valuación bajo diferentes modalidades de gestión: **FIFO (First In, First Out) y LIFO (Last In, First Out)**.
Para garantizar la precisión de los cálculos, el sistema deberá posibilitar la carga, actualización y mantenimiento de listas de precios vigentes, además de conservar un historial completo de modificaciones, de modo que cada variación pueda ser auditada y trazada.
De esta forma, Auzonia proporcionará al área gerencial información actualizada, verificable y alineada con los criterios contables y normativos, optimizando la toma de decisiones estratégicas relacionadas con costos, márgenes y rentabilidad.

1.2 Descripción Funcional del Sistema Auzonia

A continuación, se detalla la **descripción funcional del sistema a nivel global**, organizada según las principales **gestiones organizacionales** incluidas dentro del alcance del sistema.

1.2.1 Gestión de Compras

- **Estimación de cantidades a solicitar:**

  El sistema permitirá parametrizar cantidades estimadas de compra para cada artículo. A partir de estos valores, realizará los cálculos necesarios considerando los pedidos de clientes pendientes de entrega y los pedidos a

proveedores aún no recibidos, a fin de optimizar la planificación de compras y evitar excedentes de stock.

- **Generación de pedidos a proveedor:**

  Una vez validadas las cantidades a solicitar, el sistema deberá emitir e imprimir automáticamente la orden de pedido al proveedor, registrando el documento de forma interna para garantizar su trazabilidad, control y posterior seguimiento logístico.

- **Ingreso de remitos de proveedor:**

  Al momento de recibir la mercadería, el sistema deberá permitir la carga del remito correspondiente, actualizando en tiempo real el stock disponible y los pedidos a proveedores pendientes, descontando automáticamente las cantidades ingresadas por cada artículo.

1.2.2 Gestión de Entregas

- **Generación de pedidos de cliente:**

  El sistema deberá permitir la carga estructurada de pedidos de clientes, registrando de manera completa y validada los datos esenciales: fecha de pedido, identificación del cliente, domicilio de entrega, artículos solicitados, cantidades y observaciones adicionales. Cada pedido quedará asociado a un número único que garantice su trazabilidad y control posterior.

- **Organización de pedidos por vendedor:**

  Cada vendedor deberá acceder a una vista personalizada que le permita visualizar y gestionar todos sus pedidos pendientes de entrega, junto con su estado operativo actualizado (pendiente, en preparación, entregado, anulado, etc.). Esta funcionalidad ofrecerá transparencia, seguimiento y priorización eficiente dentro del flujo comercial.

- **Gestión integral de entregas:**

  El sistema deberá coordinar y automatizar la comunicación entre los distintos participantes del proceso —vendedor, jefe de ventas, encargado de depósito y jefe de playa— abarcando desde la preparación del pedido hasta la emisión del remito de entrega. La automatización de esta secuencia permitirá minimizar errores, reducir tiempos operativos y optimizar la eficiencia logística general.

1.2.3 Valuación de Inventario

- **Carga y actualización de listas de precios:**

  El sistema deberá permitir la carga y actualización de los precios de compra por artículo, conservando un historial detallado de todas las modificaciones para garantizar trazabilidad, auditoría y control contable.

- **Cálculo de valuación de inventario:**

  Con el objetivo de brindar soporte contable y financiero, el sistema deberá ofrecer la valorización de los activos a una fecha determinada mediante los siguientes métodos reconocidos:

  - FIFO (First In, First Out)
  - LIFO (Last In, First Out)

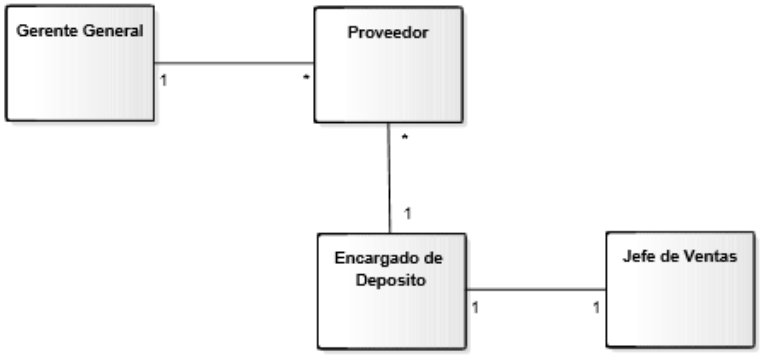
  Esta funcionalidad permitirá al área gerencial obtener información precisa sobre el valor del inventario, facilitando la toma de decisiones estratégicas basadas en datos contables confiables.

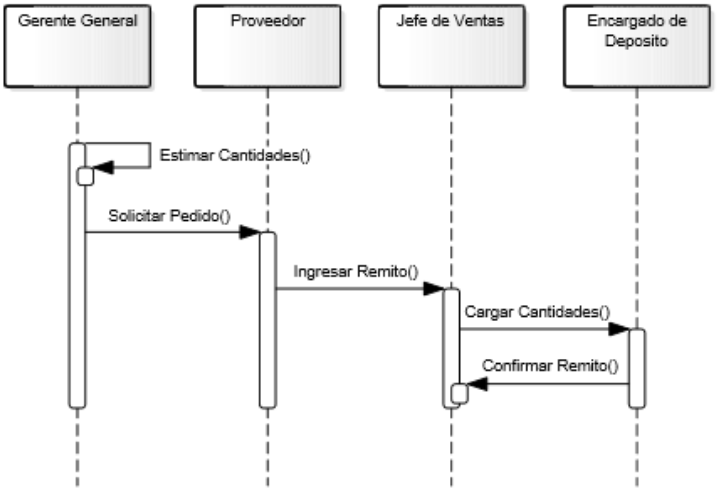
Especificación Funcional

2. Especificación Funcional

2.1 Especificación proceso de Compras

2.1.1 Identificación de Roles intervinientes

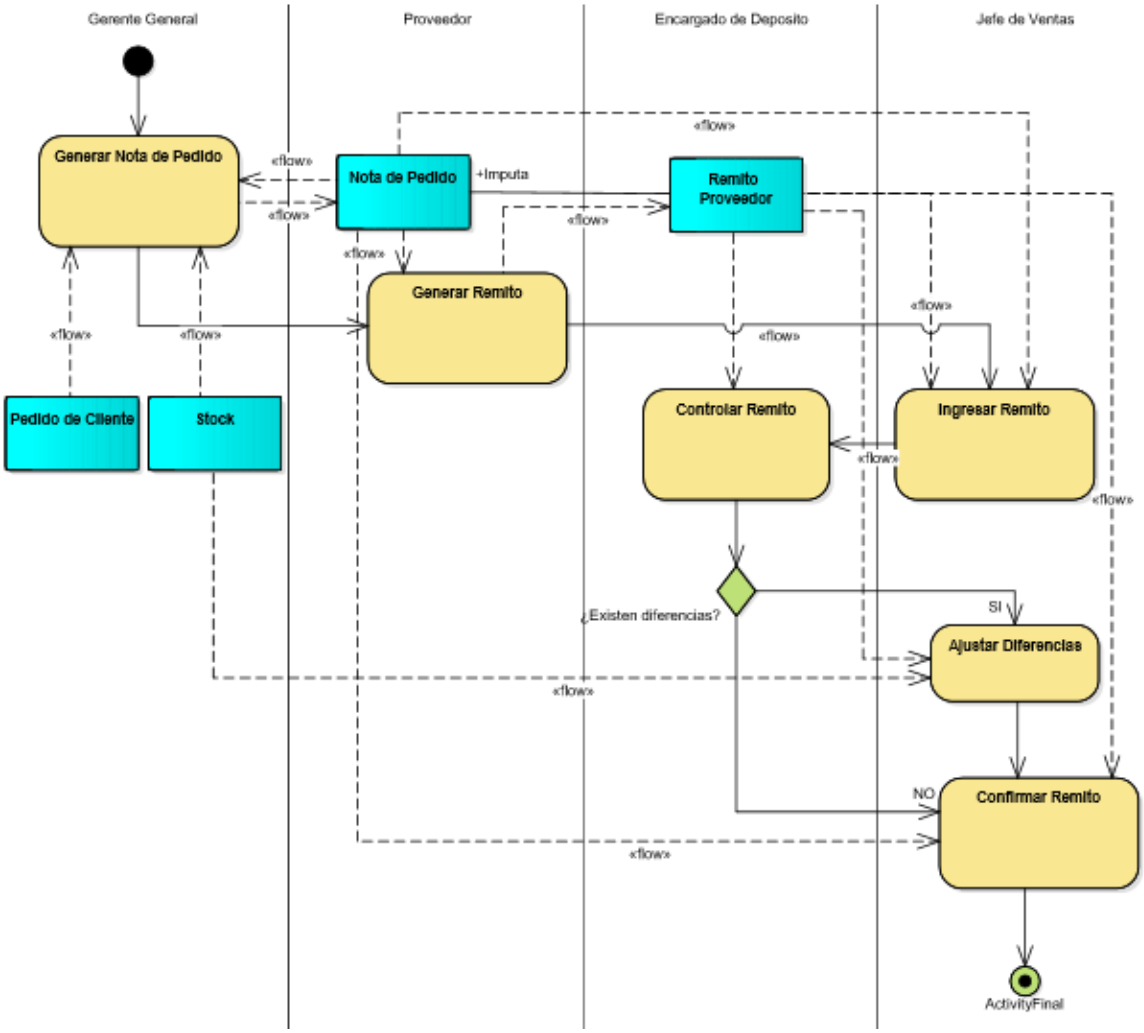




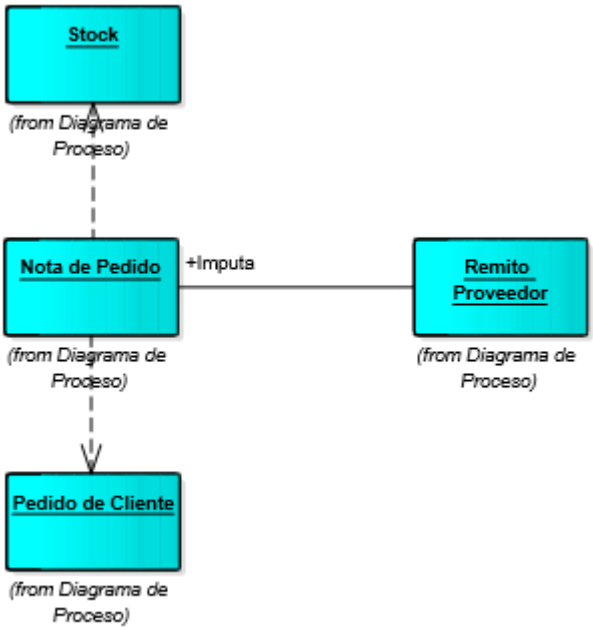
2.1.2 Descripción funcional del proceso: Entrada / Comportamiento / Salida

1. **Generación de la nota de pedido:**
   El Gerente General elabora la nota de pedido, estimando las cantidades a solicitar en función de los pedidos anteriores, pedidos de clientes vigentes y el stock mínimo establecido para cada artículo. La nota debe detallar las cantidades solicitadas, la fecha de emisión y un número identificatorio único, garantizando trazabilidad y control durante todo el proceso de compra.
2. **Emisión del remito por el proveedor:**
   El proveedor recibe la nota de pedido y genera el remito correspondiente con las cantidades a enviar en base a su disponibilidad.
3. **Ingreso del remito al sistema:**
   El Jefe de Ventas registra el remito del proveedor en el sistema, asegurando que la información quede documentada para su seguimiento.
4. **Control físico de mercadería:**
   El Encargado de Depósito verifica la mercadería recibida frente al remito, validando cantidades y condiciones, y registra las cantidades reales ingresadas al stock.
5. **Ajuste de diferencias:**
   El Jefe de Ventas revisa las cantidades enviadas y ajusta cualquier diferencia entre lo solicitado y lo recibido, asegurando la coherencia de datos en el sistema.
6. **Confirmación final:**
   Una vez validadas las cantidades físicas ingresadas al depósito, el Jefe de Ventas confirma la generación final del remito en el sistema, completando así el ciclo de recepción y actualización de stock de manera automatizada y trazable.

2.1.3 Diagrama de Proceso

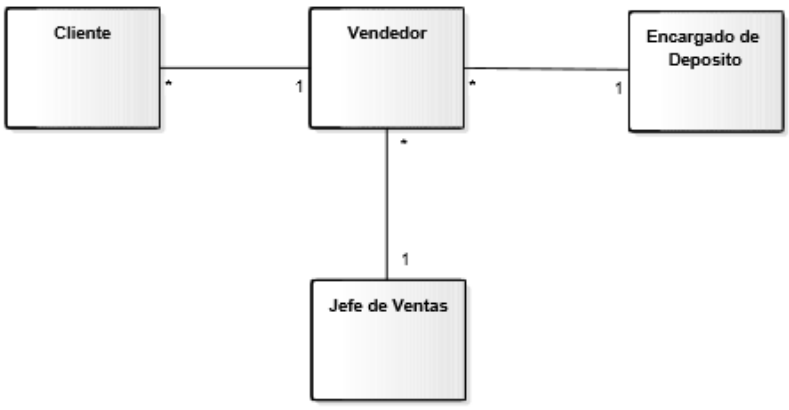


2.1.4 Modelo conceptual



2.2 Especificación proceso de Entrega

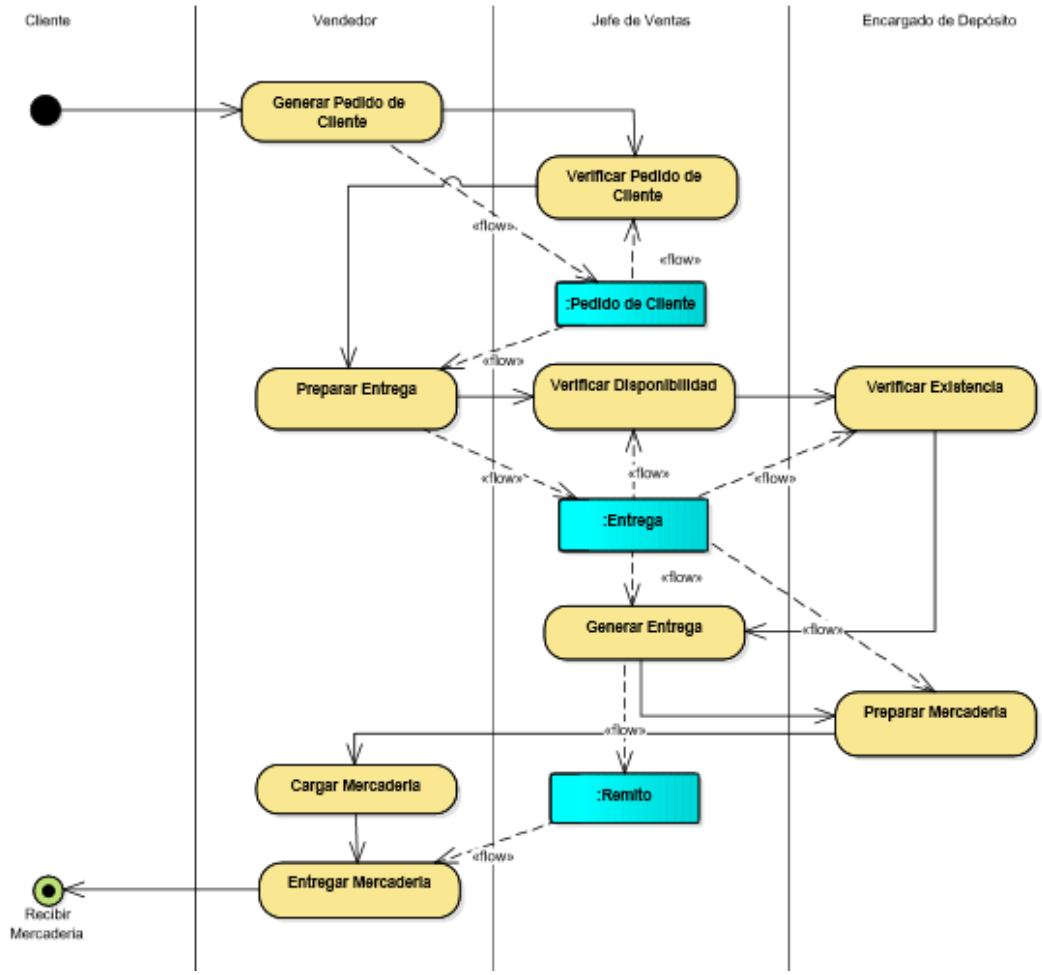
2.2.1 Identificación de Roles intervinientes



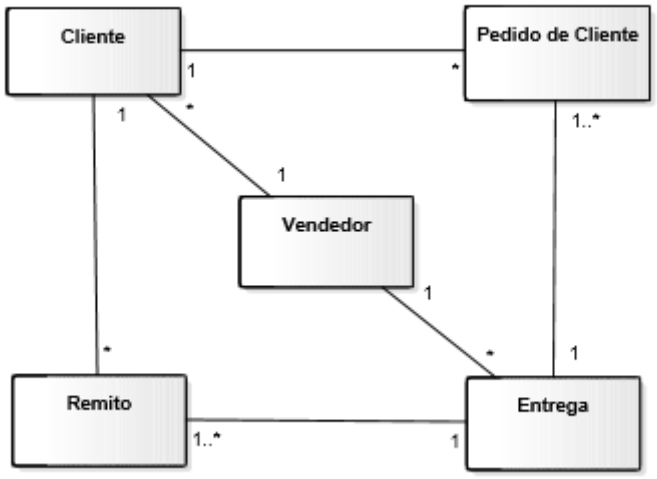
2.2.2 Descripción funcional del proceso: Entrada / Comportamiento / Salida

1. **Recepción de pedido de cliente**
   El cliente realiza un pedido especificando las cantidades requeridas por artículo. El vendedor registra esta solicitud en el sistema mediante la creación de un Pedido de Cliente, detallando los ítems informados.
2. **Revisión Comercial del Pedido**
   El Jefe de Ventas evalúa el pedido para verificar la disponibilidad de mercadería en stock y determinar si es necesario reservar medidas o productos específicos.
3. **Organización de Entregas Pendientes**
   El vendedor analiza los pedidos de cliente pendientes de entrega y planifica la logística correspondiente, agrupando los artículos disponibles para su despacho en el viaje programado.
4. **Autorización de Entrega Comercial**
   El Jefe de Ventas confirma la disponibilidad de los productos en stock y autoriza formalmente la liberación de la mercadería a entregar.
5. **Verificación Física de Stock**
   El Jefe de Depósito realiza el control físico del inventario, corroborando la existencia real de los artículos a despachar.
6. **Generación de Documentación de Entrega**
   Una vez confirmada la disponibilidad, el Jefe de Ventas emite en el sistema la Entrega, donde se detallan las cantidades correspondientes a cada artículo, y genera los Remitos asociados para su trazabilidad.
7. **Preparación de la Mercadería**
   El Encargado de Depósito procede a preparar la mercadería conforme al detalle de entrega autorizado.
8. **Carga de la Mercadería**
   El vendedor carga en el vehículo la mercadería preparada por el depósito, asegurando la correspondencia con el remito y el pedido del cliente.
9. **Entrega al Cliente**
   Finalmente, el vendedor realiza la entrega física de la mercadería al cliente, completando el proceso logístico y comercial del pedido.

2.2.3 Diagrama de Proceso

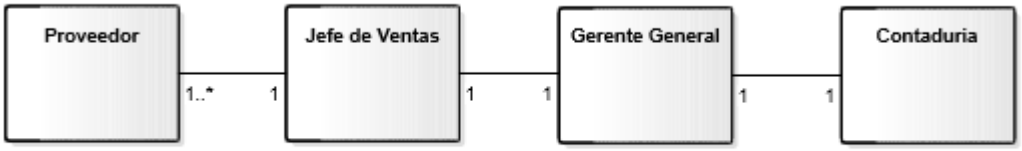


2.2.4 Modelo conceptual



2.3 Especificación proceso de Valuación de Inventario

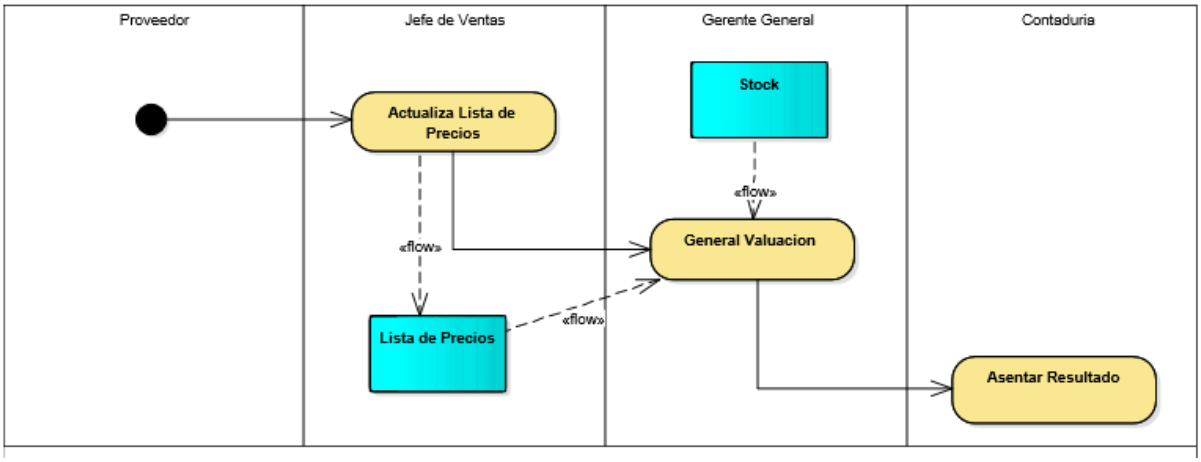
2.3.1 Identificación de Roles intervinientes



2.3.2 Descripción funcional del proceso: Entrada / Comportamiento / Salida

1. El proveedor envía un listado con la lista de precios vigente para cada uno de los artículos y la fecha de validez.
2. El jefe de ventas actualiza el listado.
3. El gerente general valoriza la mercadería.
4. El gerente general informa el resultado a contaduría.

2.3.3 Diagrama de Proceso



2.3.4 Modelo Conceptual



Especificaciones de Casos de Uso

3. Especificaciones de Casos de Uso

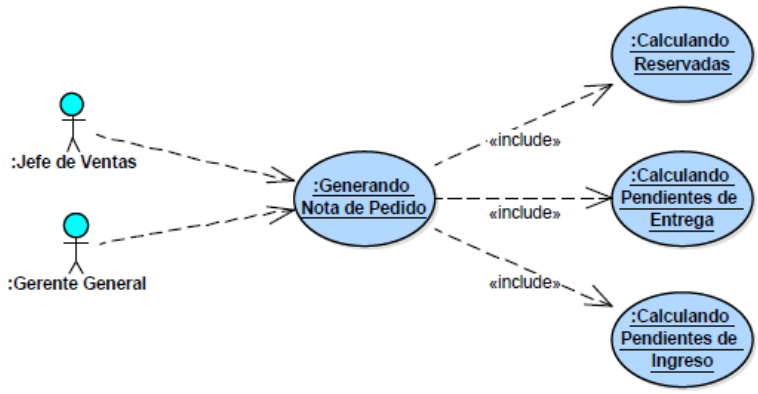
3.1 Generar Nota de Pedido CU01.01

3.1.1 Especificación

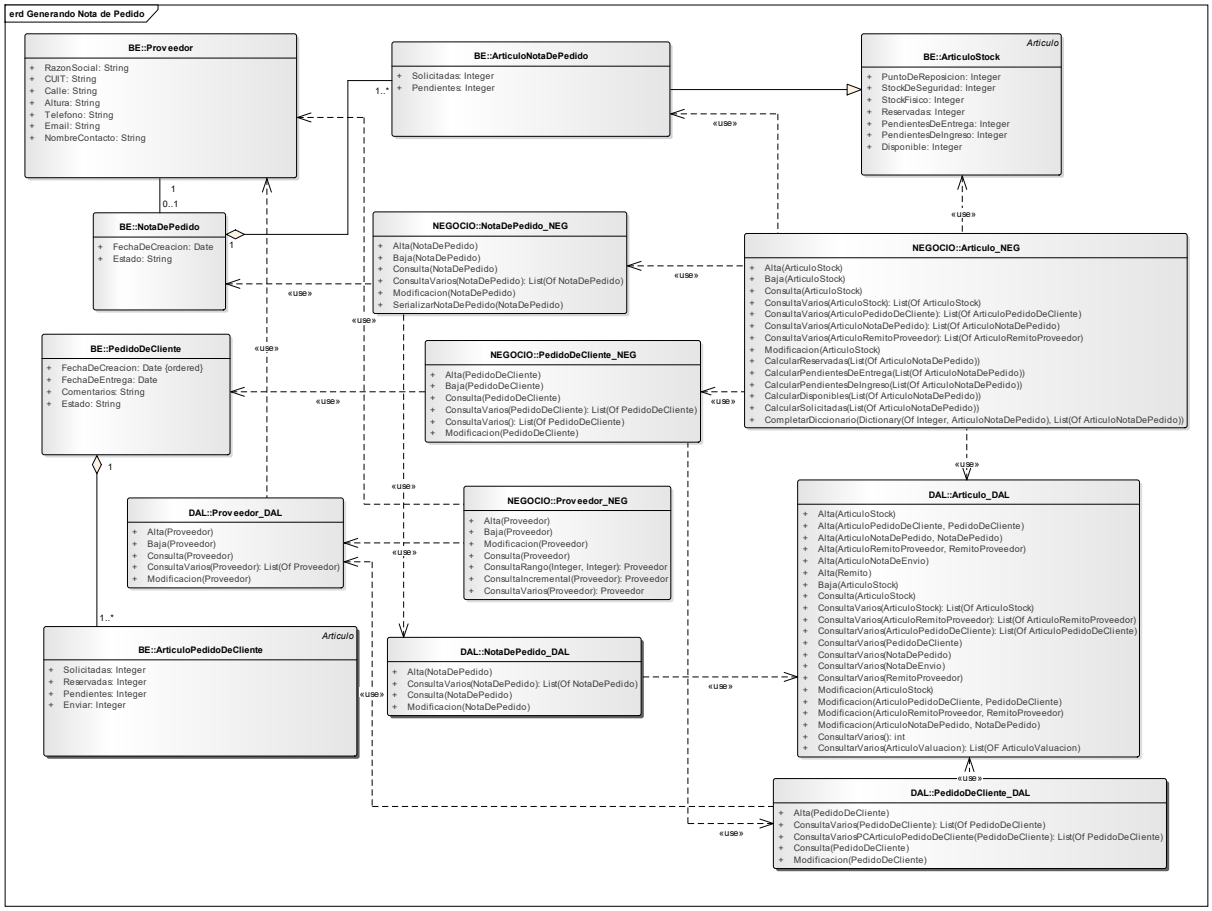
1 - Nombre	Generar Nota de Pedido
Descripción:	Permite realizar un pedido al proveedor. Dicho pedido incluye un listado con los artículos a solicitar y sus respectivas cantidades. Para poder calcular las cantidades a solicitar el sistema facilita al usuario una estimación a pedir teniendo en cuenta los pedidos realizados por los clientes y la mercadería pendiente de entrega por parte de dicho proveedor.
Actores:	Gerente General
2 - Flujo de Eventos	
Disparador/Condición:	El gerente general inicia el proceso de gestionar pedido al proveedor.
Flujo Básico:	
	1. El sistema muestra en pantalla los proveedores disponibles. 2. El usuario selecciona al proveedor al cual se le solicitará el pedido. 3. El sistema muestra un listado con el detalle de todos los artículos relacionados con proveedor seleccionado. Para cada artículo debe detallar los siguientes puntos: Código Descripción Punto de Reposición Stock de Seguridad Stock físico Reservadas Pendientes de Entrega Pendientes de Ingreso Disponibles Solicitadas (El sistema calcula la cantidad estimada) 4. El usuario ajusta las cantidades a solicitar para cada uno de los artículos. 5. El usuario confirma la creación de la nota de pedido. 6. El sistema muestra en pantalla el ingreso exitoso. 8. Fin de caso de uso.
Flujos Alternativos:	

3 - Requerimientos Especiales	
	N/A.
4 - Precondiciones:	Debe existir un al menos un proveedor cargado en el sistema.
	El usuario debe tener permiso para generar la Nota de Pedido.
5 - Postcondiciones:	
	El usuario visualiza en pantalla el detalle de la Nota de Pedido Generada. Este detalle incluye la fecha, el usuario que la generó, un número identificatorio y el listado de artículos con sus respectivas cantidades. La Nota de Pedido queda registrada en la base de datos.
6 - Puntos de Extensión	

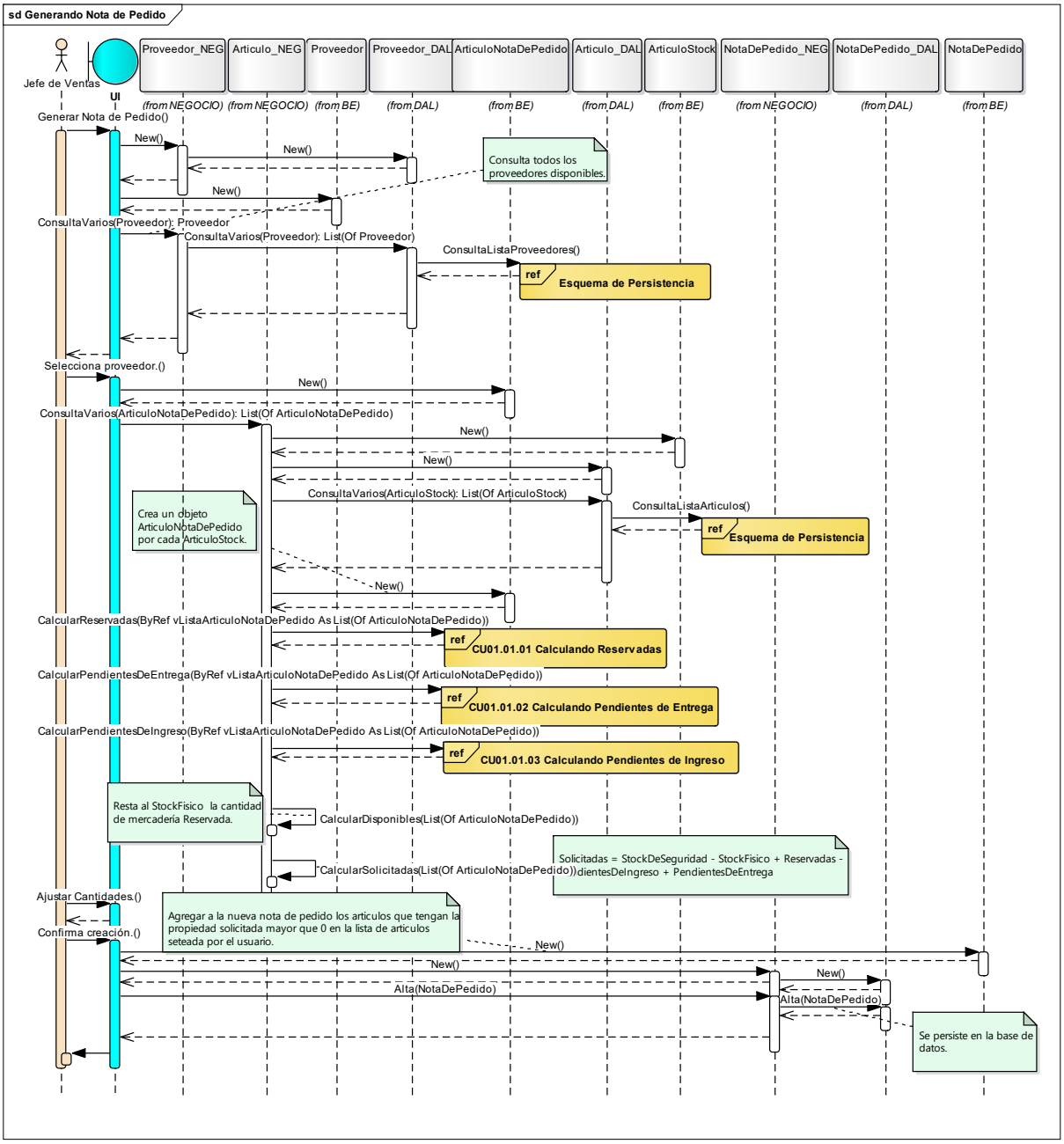
3.1.2 Gráfico de Caso de uso



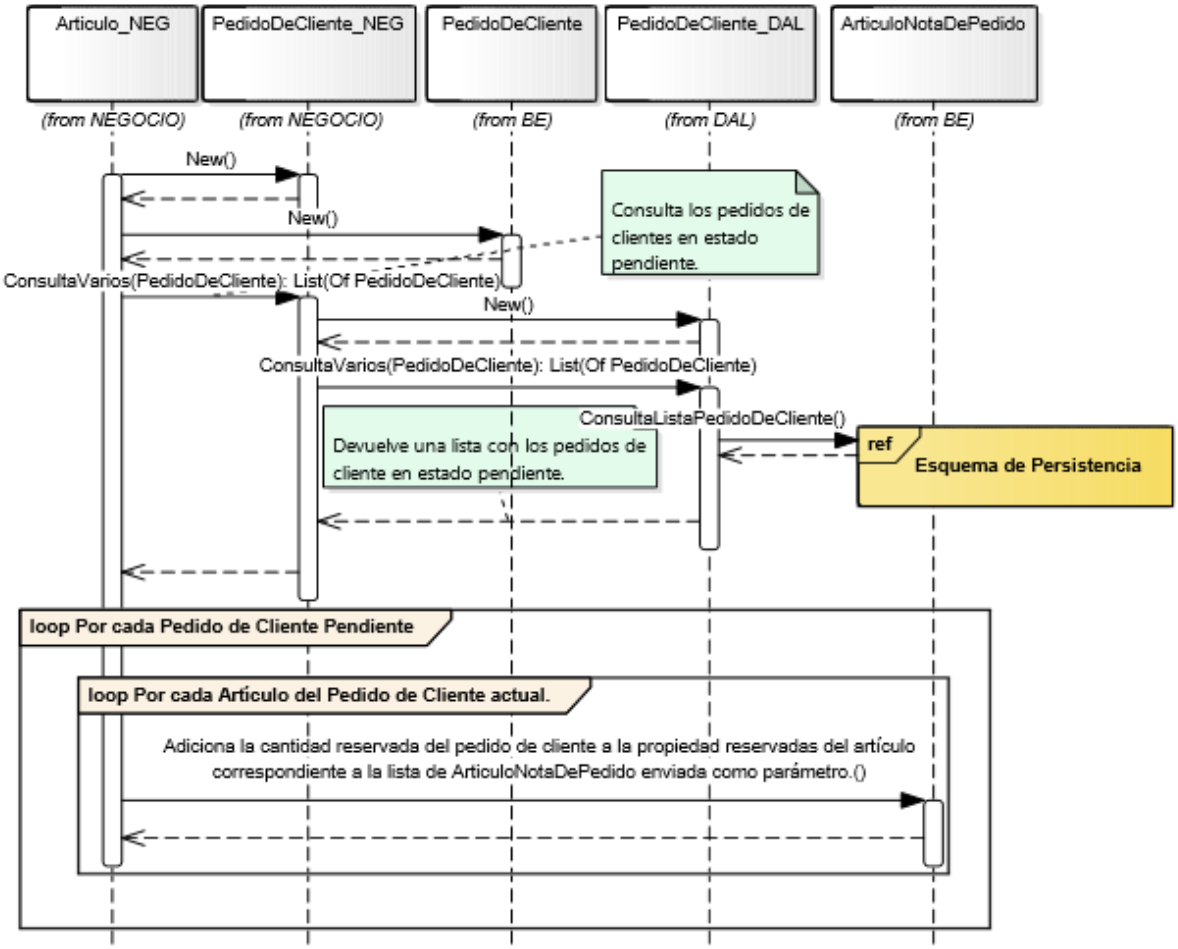
3.1.3 Diagrama de Clases



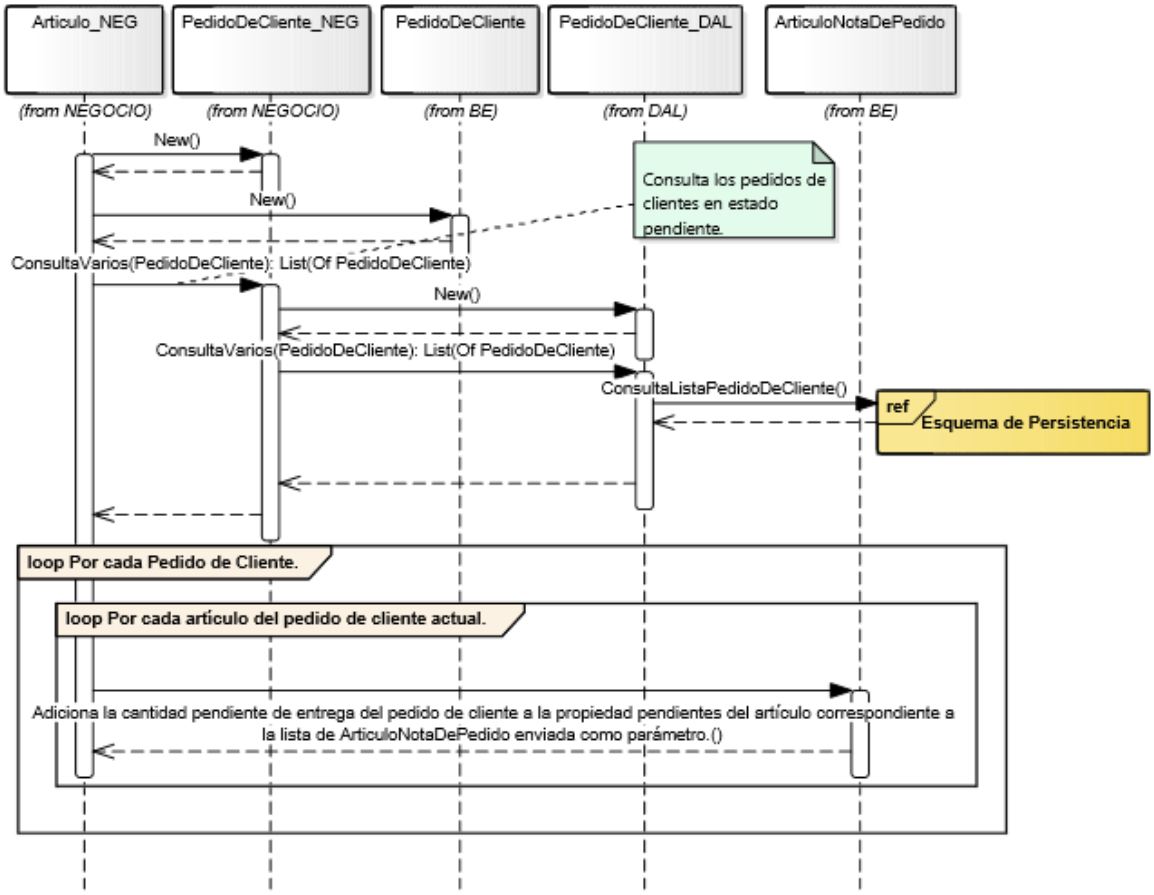
3.1.4 Diagrama de Secuencia – Generar Nota de Pedido



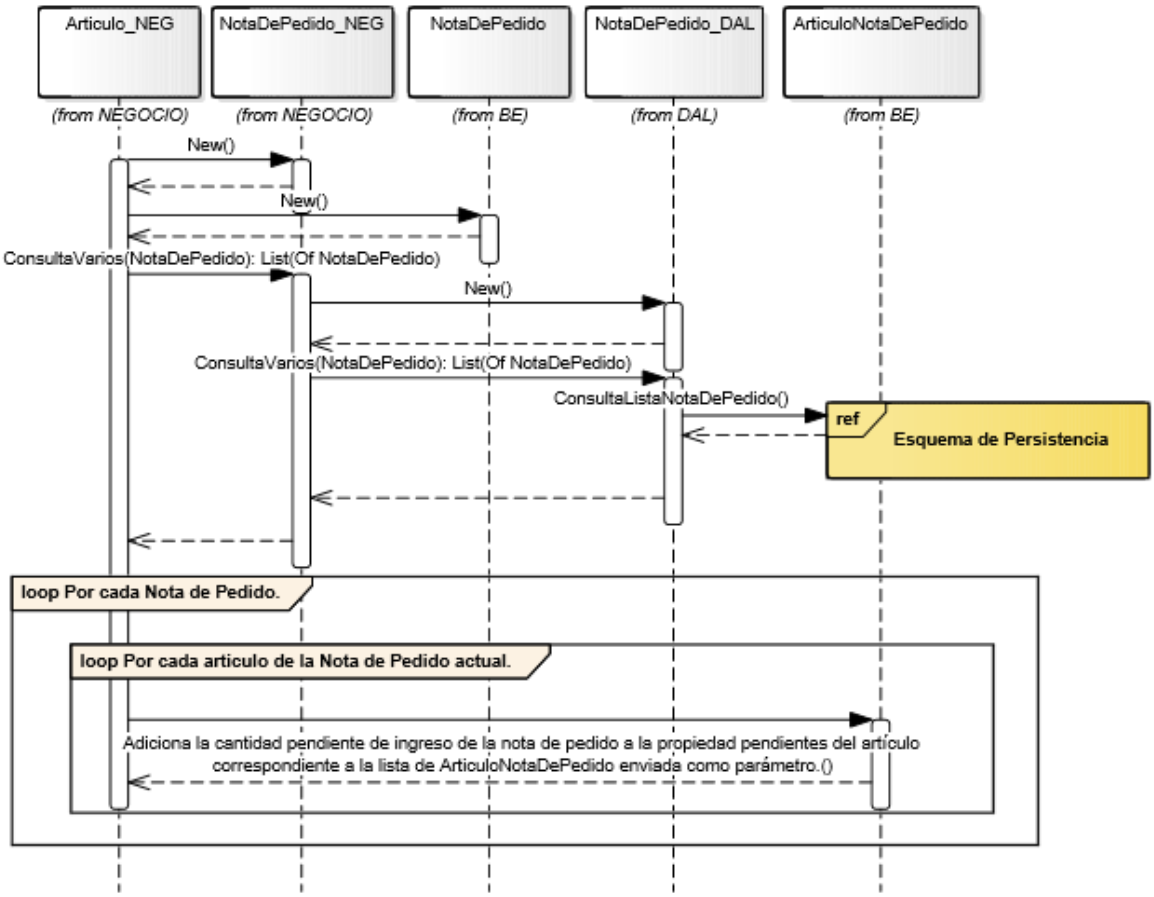
3.1.5 Diagrama de Secuencia – Calcular Reservadas



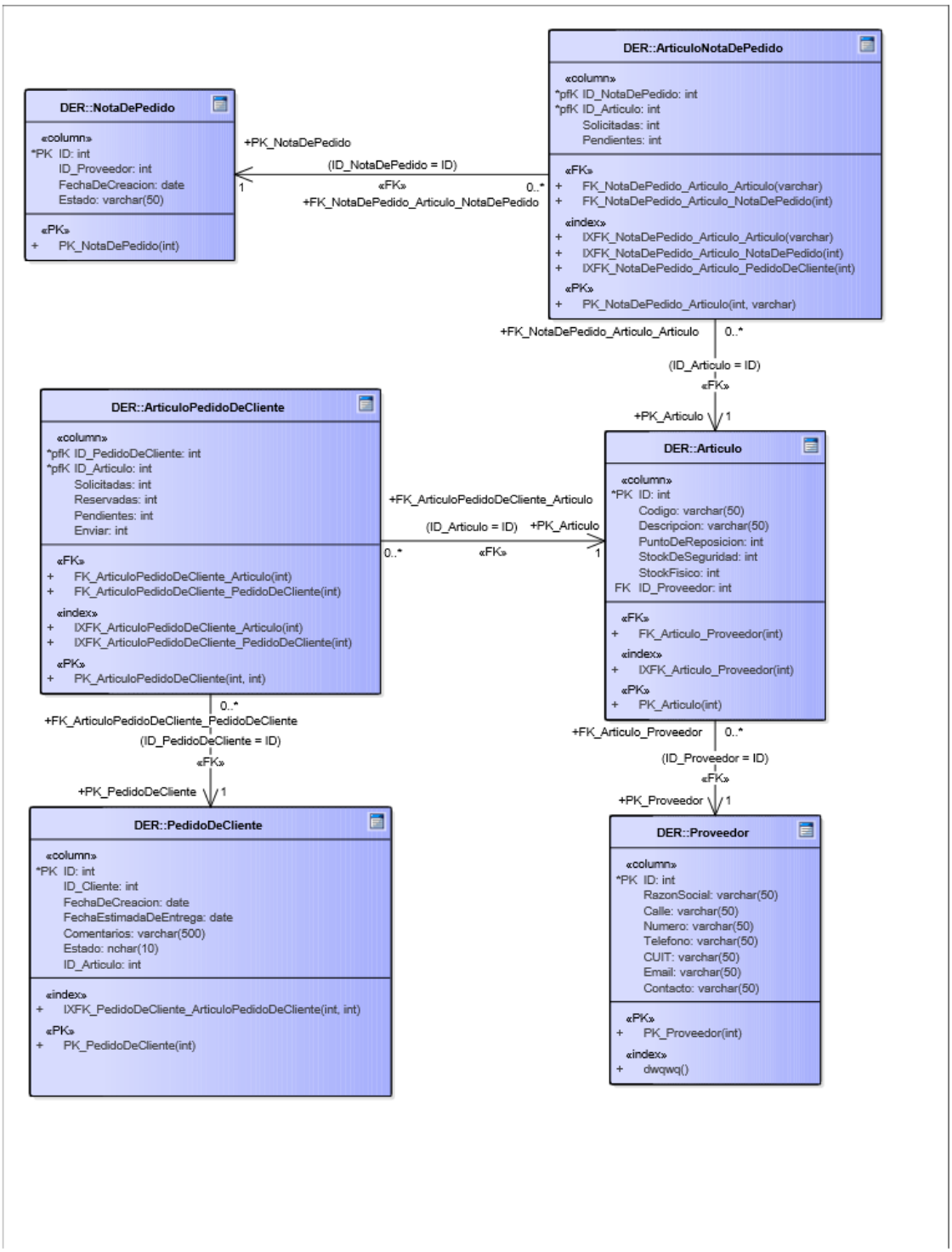
3.1.6 Diagrama de Secuencia – Calculando Pendientes de Entrega



3.1.7 Diagrama de Secuencia – Calculando Pendientes de Ingreso



3.1.8 DER

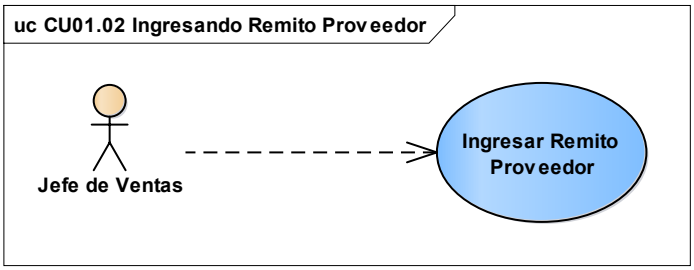

DER::NotaDePedido
«column»
*PK ID: int
ID_Proveedor: int
FechaDeCreacion: date
Estado: varchar(50)
«PK»
+ PK_NotaDePedido(int)
+PK_NotaDePedido
(ID_NotaDePedido = ID)
1
«FK»
0..*
+FK_NotaDePedido_Articulo_NotaDePedido
DER::ArticuloNotaDePedido
«column»
*pfK ID_NotaDePedido: int
*pfK ID_Articulo: int
Solicitadas: int
Pendientes: int
«FK»
+ FK_NotaDePedido_Articulo_Articulo(varchar)
+ FK_NotaDePedido_Articulo_NotaDePedido(int)
«index»
+ IXFK_NotaDePedido_Articulo_Articulo(varchar)
+ IXFK_NotaDePedido_Articulo_NotaDePedido(int)
+ IXFK_NotaDePedido_Articulo_PedidoDeCliente(int)
«PK»
+ PK_NotaDePedido_Articulo(int, varchar)
+FK_NotaDePedido_Articulo_Articulo
0..*
(ID_Articulo = ID)
«FK»
+PK_Articulo
1
DER::ArticuloPedidoDeCliente
«column»
*pfK ID_PedidoDeCliente: int
*pfK ID_Articulo: int
Solicitadas: int
Reservadas: int
Pendientes: int
Enviar: int
«FK»
+ FK_ArticuloPedidoDeCliente_Articulo(int)
+ FK_ArticuloPedidoDeCliente_PedidoDeCliente(int)
«index»
+ IXFK_ArticuloPedidoDeCliente_Articulo(int)
+ IXFK_ArticuloPedidoDeCliente_PedidoDeCliente(int)
«PK»
+ PK_ArticuloPedidoDeCliente(int, int)
+FK_ArticuloPedidoDeCliente_Articulo
(ID_Articulo = ID)
+PK_Articulo
0..*
«FK»
1
DER::Articulo
«column»
*PK ID: int
Codigo: varchar(50)
Descripcion: varchar(50)
PuntoDeReposicion: int
StockDeSeguridad: int
StockFisico: int
FK ID_Proveedor: int
«FK»
+ FK_Articulo_Proveedor(int)
«index»
+ IXFK_Articulo_Proveedor(int)
«PK»
+ PK_Articulo(int)
0..*
+FK_ArticuloPedidoDeCliente_PedidoDeCliente
(ID_PedidoDeCliente = ID)
«FK»
+PK_PedidoDeCliente
1
+FK_Articulo_Proveedor
0..*
(ID_Proveedor = ID)
«FK»
+PK_Proveedor
1
DER::PedidoDeCliente
«column»
*PK ID: int
ID_Cliente: int
FechaDeCreacion: date
FechaEstimadaDeEntrega: date
Comentarios: varchar(500)
Estado: nchar(10)
ID_Articulo: int
«index»
+ IXFK_PedidoDeCliente_ArticuloPedidoDeCliente(int, int)
«PK»
+ PK_PedidoDeCliente(int)
DER::Proveedor
«column»
*PK ID: int
RazonSocial: varchar(50)
Calle: varchar(50)
Numero: varchar(50)
Telefono: varchar(50)
CUIT: varchar(50)
Email: varchar(50)
Contacto: varchar(50)
«PK»
+ PK_Proveedor(int)
«index»
+ dwqwq()

3.2 Ingresar Remito Proveedor CU01.02

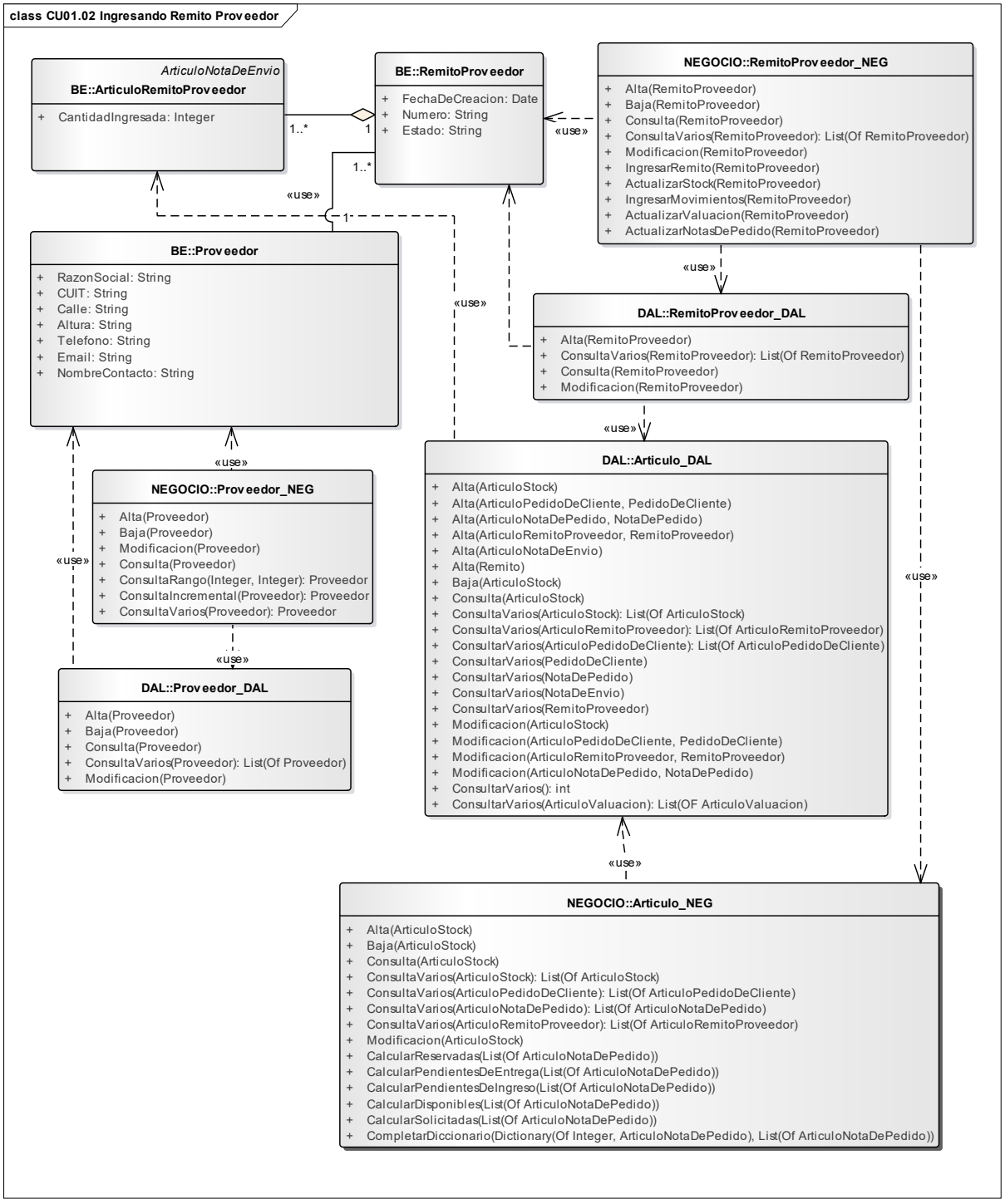
3.2.1 Especificación

1 - Nombre	Ingresar Remito Proveedor
Descripción:	Permite ingresar al sistema el remito generado por el proveedor. Para ello el usuario debe ingresar las cantidades correspondientes para cada uno de los artículos.
Actores:	Jefe de Ventas
2 - Flujo de Eventos	
Disparador/Condición:	El jede de ventas inicia el proceso de ingreso de remito proveedor.
Flujo Básico:	
	1. El sistema muestra en pantalla los proveedores disponibles. 2. El usuario selecciona al proveedor por el cual se ingresará el remito. 3. El sistema muestra un listado con el detalle de todos los artículos relacionados con proveedor seleccionado. 4. El usuario ajusta las cantidades a solicitar para cada uno de los artículos. 5. El usuario confirma la creación del remito de proveedor. 6. El sistema genera el remito de proveedor. 7. Fin de caso de uso.
Flujos Alternativos:	
	N/A
3 - Requerimientos Especiales	
	N/A
4 - Precondiciones:	
	El usuario debe tener permiso para generar la Remito de Proveedor.
5 - Postcondiciones:	
	El usuario visualiza en pantalla la confirmación del Remito Proveedor generado correctamente. El Remito de Proveedor queda registrado en la base de datos.
6 - Puntos de Extensión	
	N/A

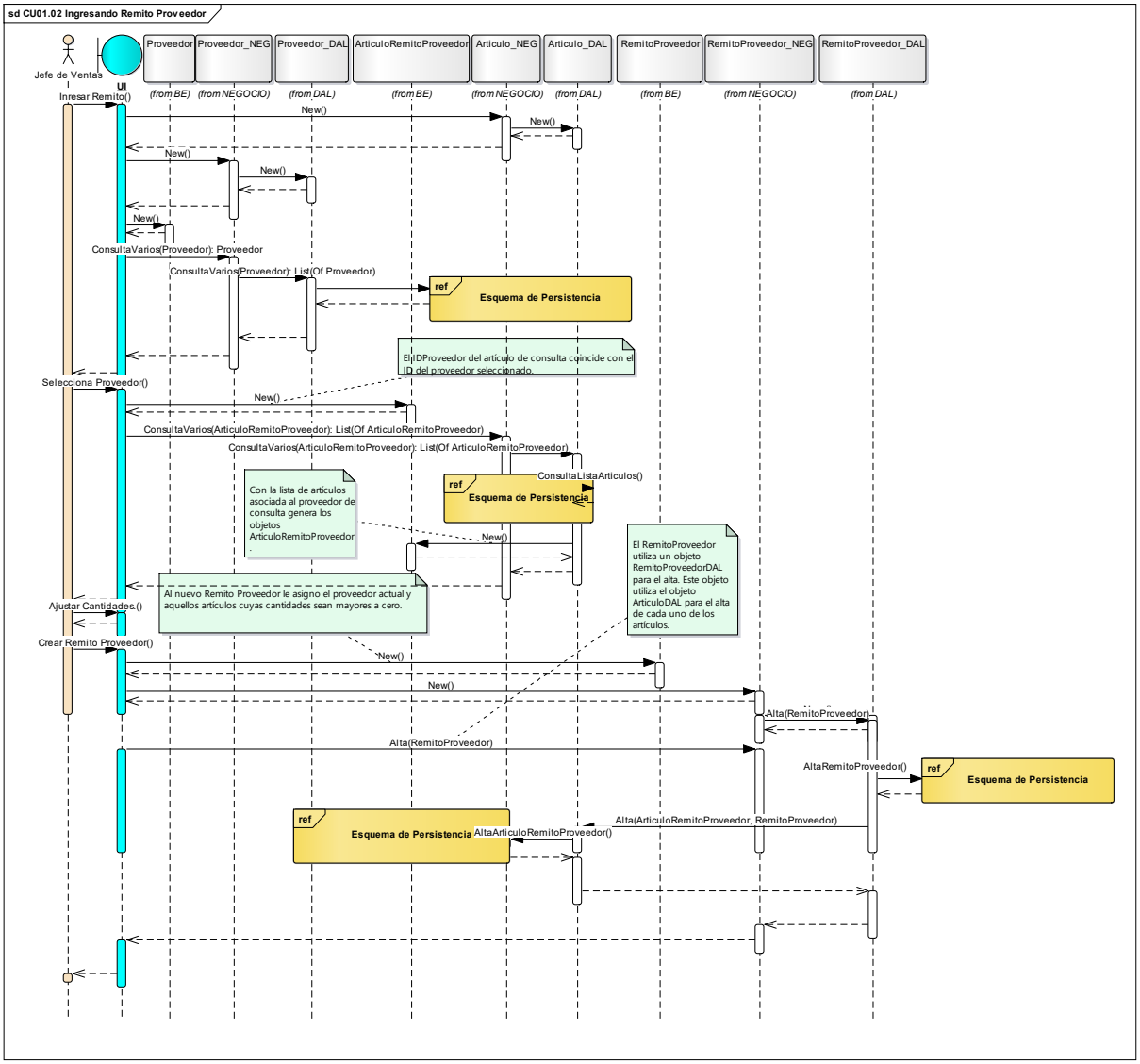
3.2.2 Gráfico de Caso de uso



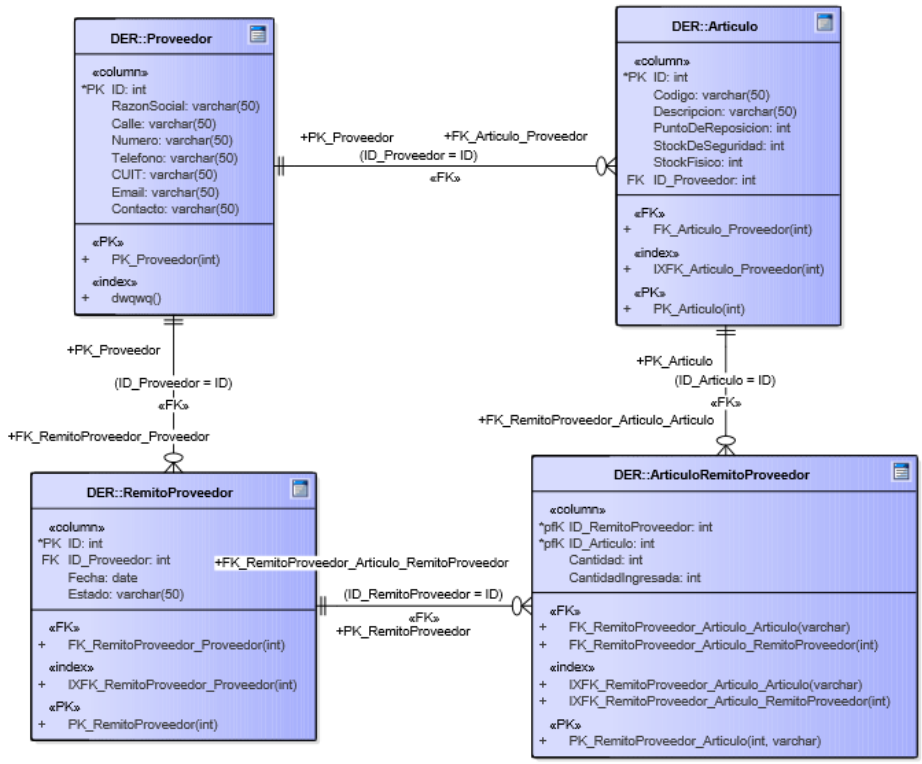
3.2.3 Diagrama de Clases



3.2.4 Diagrama de Secuencia



3.2.5 DER

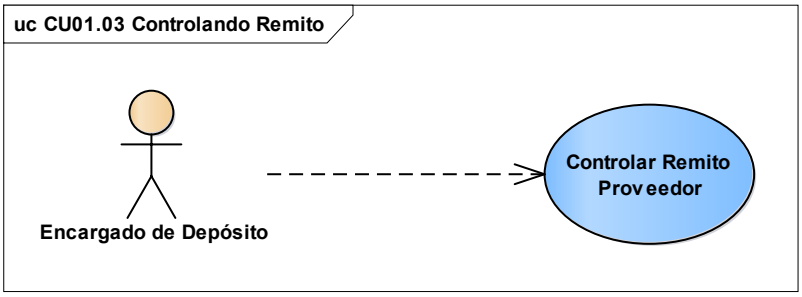

DER::Proveedor
«column»
*PK ID: int
RazonSocial: varchar(50)
Calle: varchar(50)
Numero: varchar(50)
Telefono: varchar(50)
CUIT: varchar(50)
Email: varchar(50)
Contacto: varchar(50)
«PK»
+ PK_Proveedor(int)
«index»
+ dwqwq()
DER::Articulo
«column»
*PK ID: int
Codigo: varchar(50)
Descripcion: varchar(50)
PuntoDeReposicion: int
StockDeSeguridad: int
StockFisico: int
FK ID_Proveedor: int
«FK»
+ FK_Articulo_Proveedor(int)
«index»
+ IXFK_Articulo_Proveedor(int)
«PK»
+ PK_Articulo(int)
+PK_Proveedor
(ID_Proveedor = ID)
«FK»
+FK_Articulo_Proveedor
+PK_Proveedor
(ID_Proveedor = ID)
«FK»
+FK_RemitoProveedor_Proveedor
+PK_Articulo
(ID_Articulo = ID)
«FK»
+FK_RemitoProveedor_Articulo_Articulo
DER::RemitoProveedor
«column»
*PK ID: int
FK ID_Proveedor: int
Fecha: date
Estado: varchar(50)
«FK»
+ FK_RemitoProveedor_Proveedor(int)
«index»
+ IXFK_RemitoProveedor_Proveedor(int)
«PK»
+ PK_RemitoProveedor(int)
+FK_RemitoProveedor_Articulo_RemitoProveedor
(ID_RemitoProveedor = ID)
«FK»
+PK_RemitoProveedor
DER::ArticuloRemitoProveedor
«column»
*pfK ID_RemitoProveedor: int
*pfK ID_Articulo: int
Cantidad: int
CantidadIngresada: int
«FK»
+ FK_RemitoProveedor_Articulo_Articulo(varchar)
+ FK_RemitoProveedor_Articulo_RemitoProveedor(int)
«index»
+ IXFK_RemitoProveedor_Articulo_Articulo(varchar)
+ IXFK_RemitoProveedor_Articulo_RemitoProveedor(int)
«PK»
+ PK_RemitoProveedor_Articulo(int, varchar)

3.3 Controlando Remito CU01.03

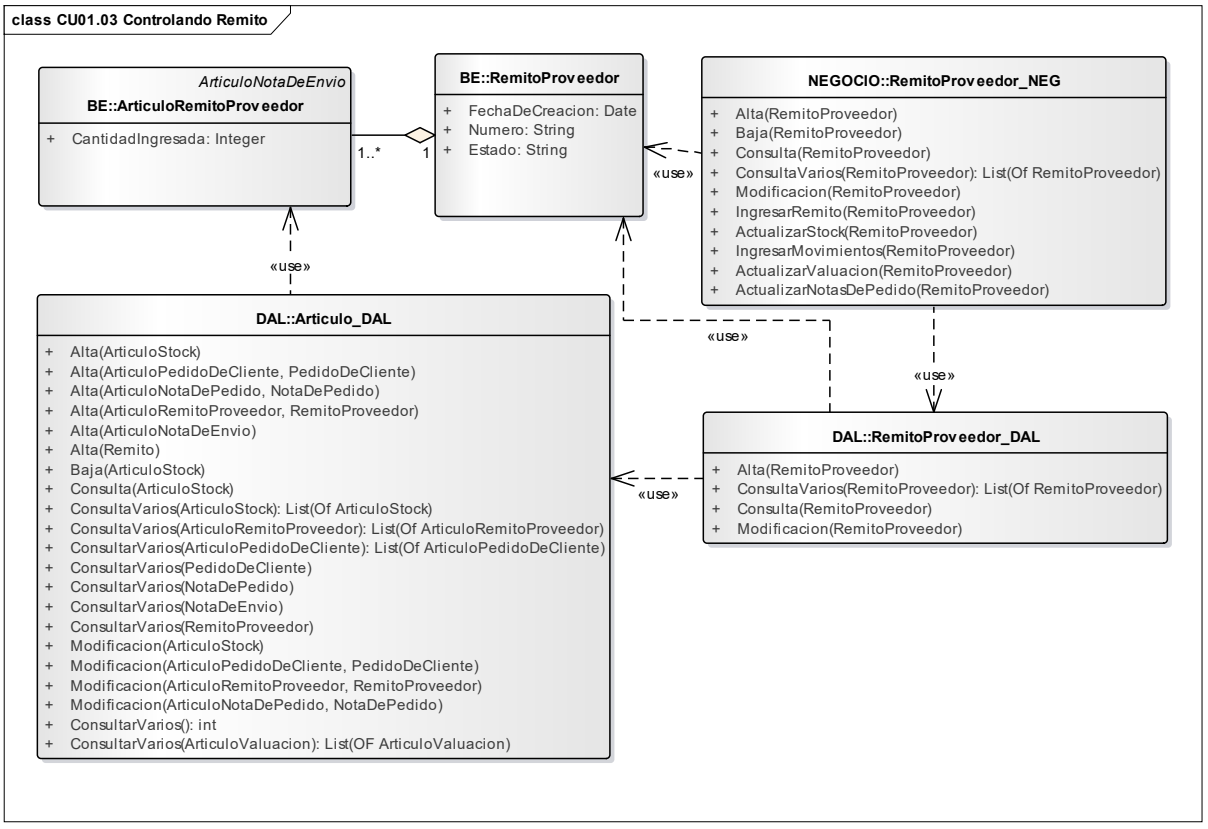
3.3.1 Especificación

1 - Nombre	Controlando Remito Proveedor
Descripción:	Permite al Encargado de Depósito controlar la existencia física de la mercadería recibida.
Actores:	Encargado de Depósito
2 - Flujo de Eventos	
Disparador/Condición:	El encargado de depósito inicia el proceso de controlar remito proveedor.
Flujo Básico:	
	1. El sistema muestra en pantalla los remitos de proveedor en estado Ingresados. 2. El usuario selecciona el remito a controlar. 3. El sistema muestra un listado con el detalle de todos los artículos relacionados con el remito proveedor seleccionado. 4. El usuario ajusta las cantidades físicas para cada uno de los artículos. 5. El usuario confirma la verificación. 6. El sistema guarda el remito de proveedor con estado Revisado. 7. Fin de caso de uso.
Flujos Alternativos:	
	N/A
3 - Requerimientos Especiales	
	N/A
4 - Precondiciones:	
	El usuario debe tener permiso para controlar el Remito de Proveedor.
5 - Postcondiciones:	
	El usuario visualiza en pantalla la confirmación del Remito Proveedor actualizado correctamente. El Remito de Proveedor queda registrado en la base de datos.
6 - Puntos de Extensión	
	N/A

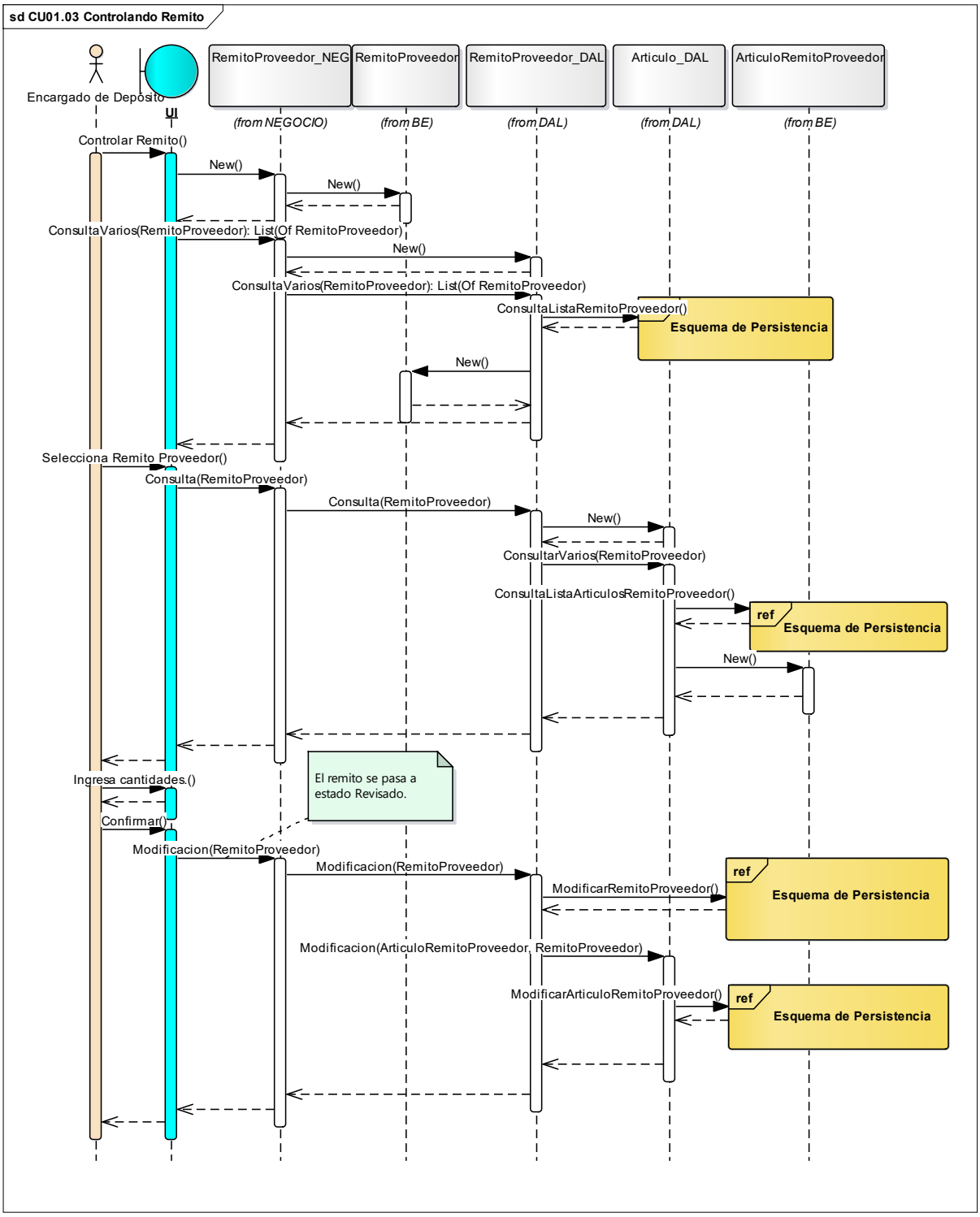
3.3.2 Gráfico de Caso de uso



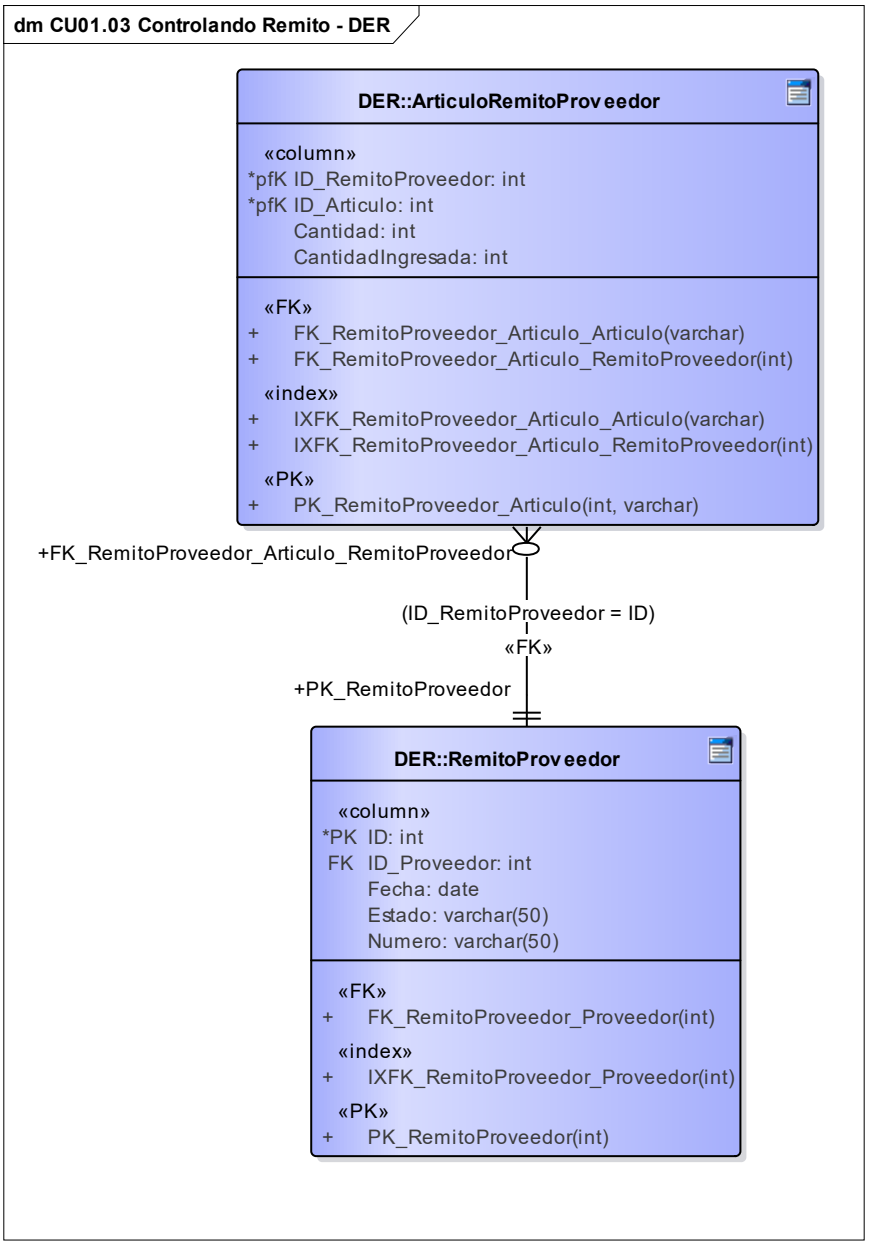
3.3.3 Diagrama de Clases



3.3.4 Diagrama de Secuencia



3.3.5 DER

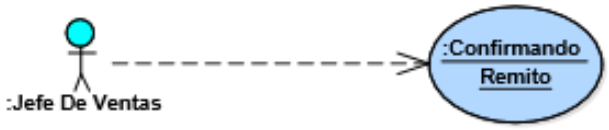

dm CU01.03 Controlando Remito - DER
DER::ArticuloRemitoProveedor
«column»
*pfK ID_RemitoProveedor: int
*pfK ID_Articulo: int
Cantidad: int
CantidadIngresada: int
«FK»
+ FK_RemitoProveedor_Articulo_Articulo(varchar)
+ FK_RemitoProveedor_Articulo_RemitoProveedor(int)
«index»
+ IXFK_RemitoProveedor_Articulo_Articulo(varchar)
+ IXFK_RemitoProveedor_Articulo_RemitoProveedor(int)
«PK»
+ PK_RemitoProveedor_Articulo(int, varchar)
+FK_RemitoProveedor_Articulo_RemitoProveedor
(ID_RemitoProveedor = ID)
«FK»
+PK_RemitoProveedor
DER::RemitoProveedor
«column»
*PK ID: int
FK ID_Proveedor: int
Fecha: date
Estado: varchar(50)
Numero: varchar(50)
«FK»
+ FK_RemitoProveedor_Proveedor(int)
«index»
+ IXFK_RemitoProveedor_Proveedor(int)
«PK»
+ PK_RemitoProveedor(int)

3.4 Confirmando Remito Proveedor CU01.04

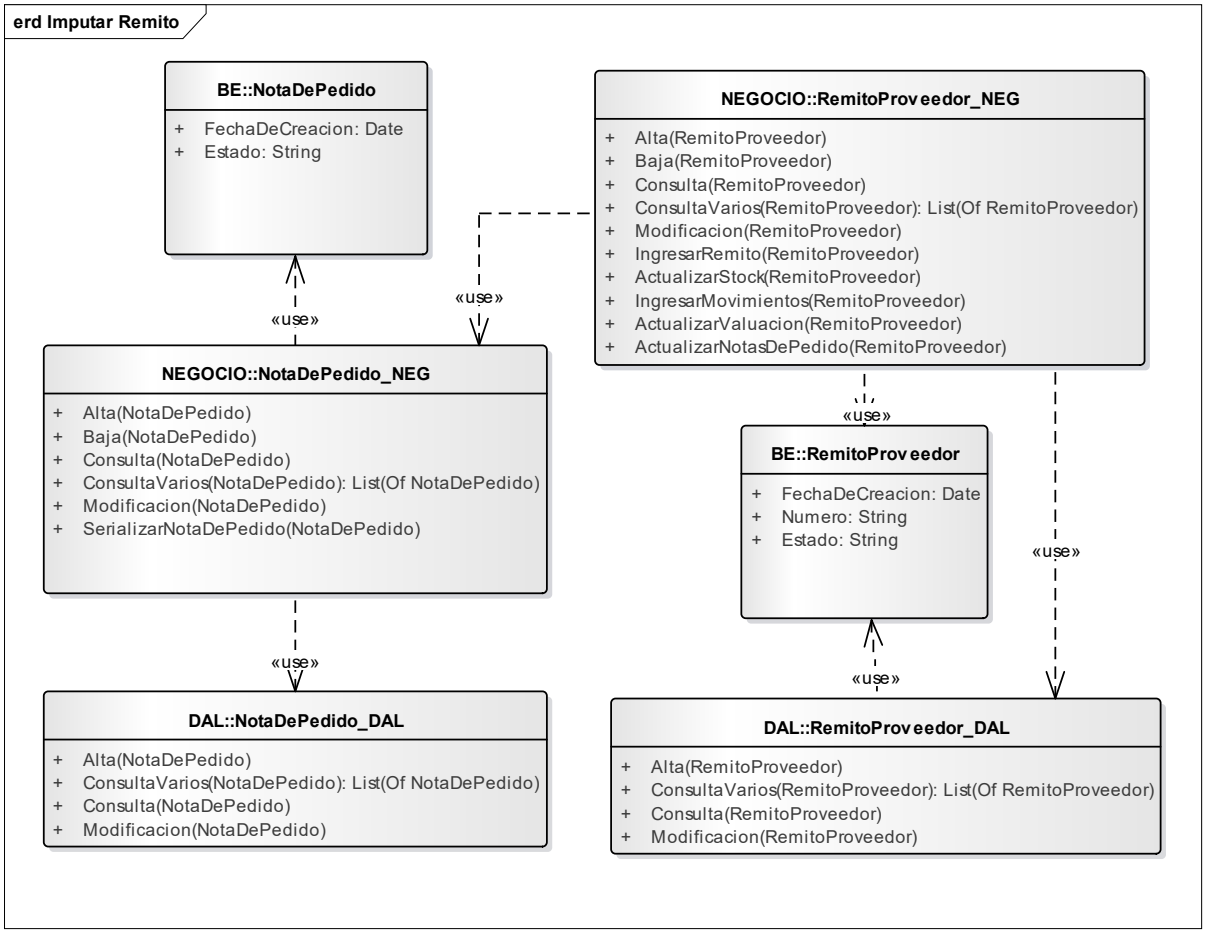
3.4.1 Especificación

1 - Nombre	Confirmando Remito Proveedor
Descripción:	Permite al Jefe de Ventas confirmar las cantidades previamente verificadas por el Encargado de Depósito.
Actores:	Encargado de Depósito
2 - Flujo de Eventos	
Disparador/Condición:	El jefe de ventas inicia el proceso de confirmar remito proveedor.
Flujo Básico:	
	1. El sistema muestra en pantalla los remitos de proveedor en estado Revisado. 2. El usuario selecciona el remito a controlar. 3. El sistema muestra un listado con el detalle de todos los artículos relacionados con el remito proveedor seleccionado. 4. El usuario ajusta las cantidades físicas para cada uno de los artículos. 5. El usuario confirma el ingreso. 6. El sistema guarda el remito de proveedor con estado Ingresado. 7. Fin de caso de uso.
Flujos Alternativos:	
	N/A
3 - Requerimientos Especiales	
	N/A
4 - Precondiciones:	
	El usuario debe tener permiso para ingresar el Remito de Proveedor.
5 - Postcondiciones:	
	El usuario visualiza en pantalla la confirmación del Remito Proveedor actualizado correctamente. El Remito de Proveedor queda registrado en la base de datos.
6 - Puntos de Extensión	
	N/A

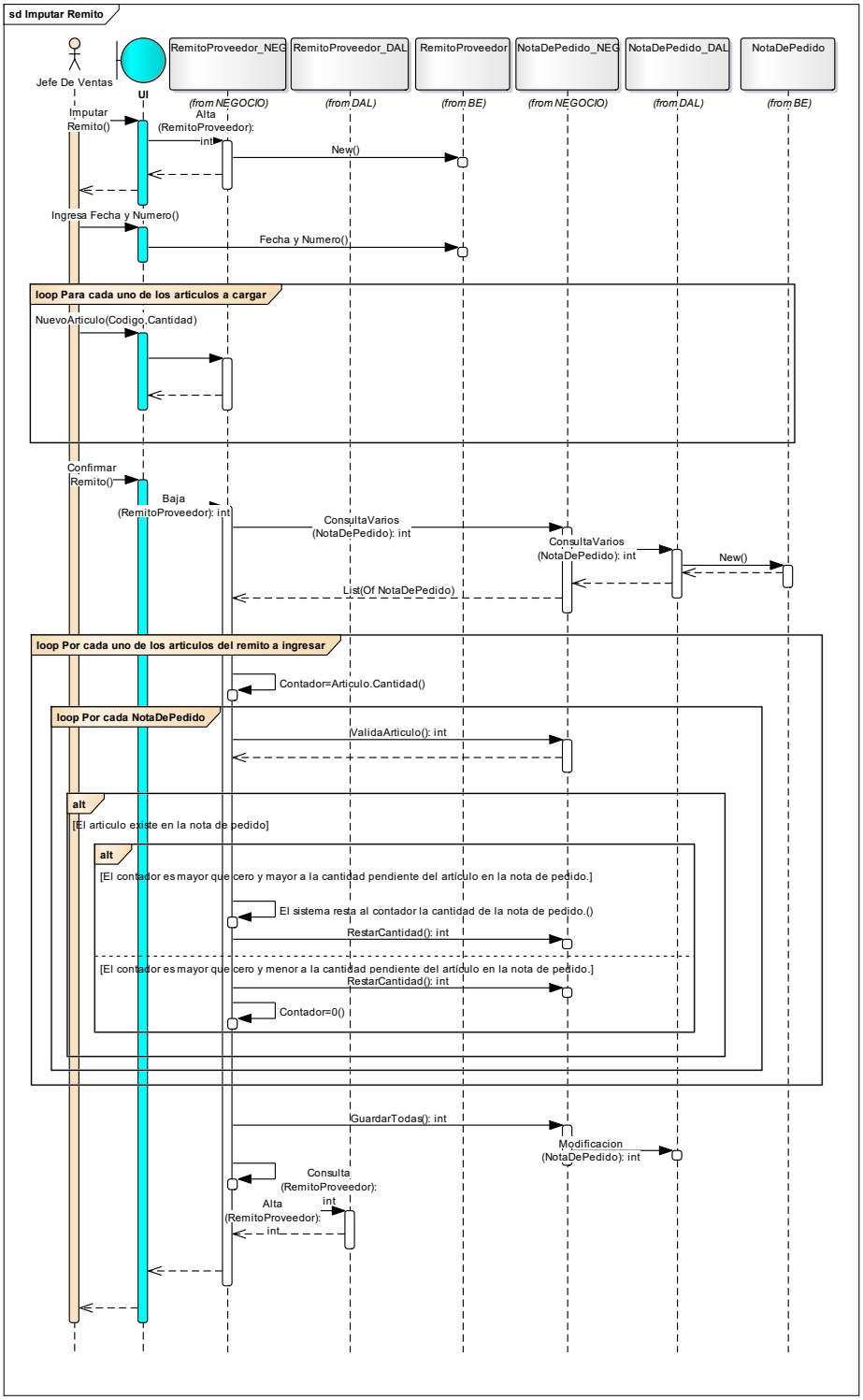
3.4.2 Gráfico de Caso de uso



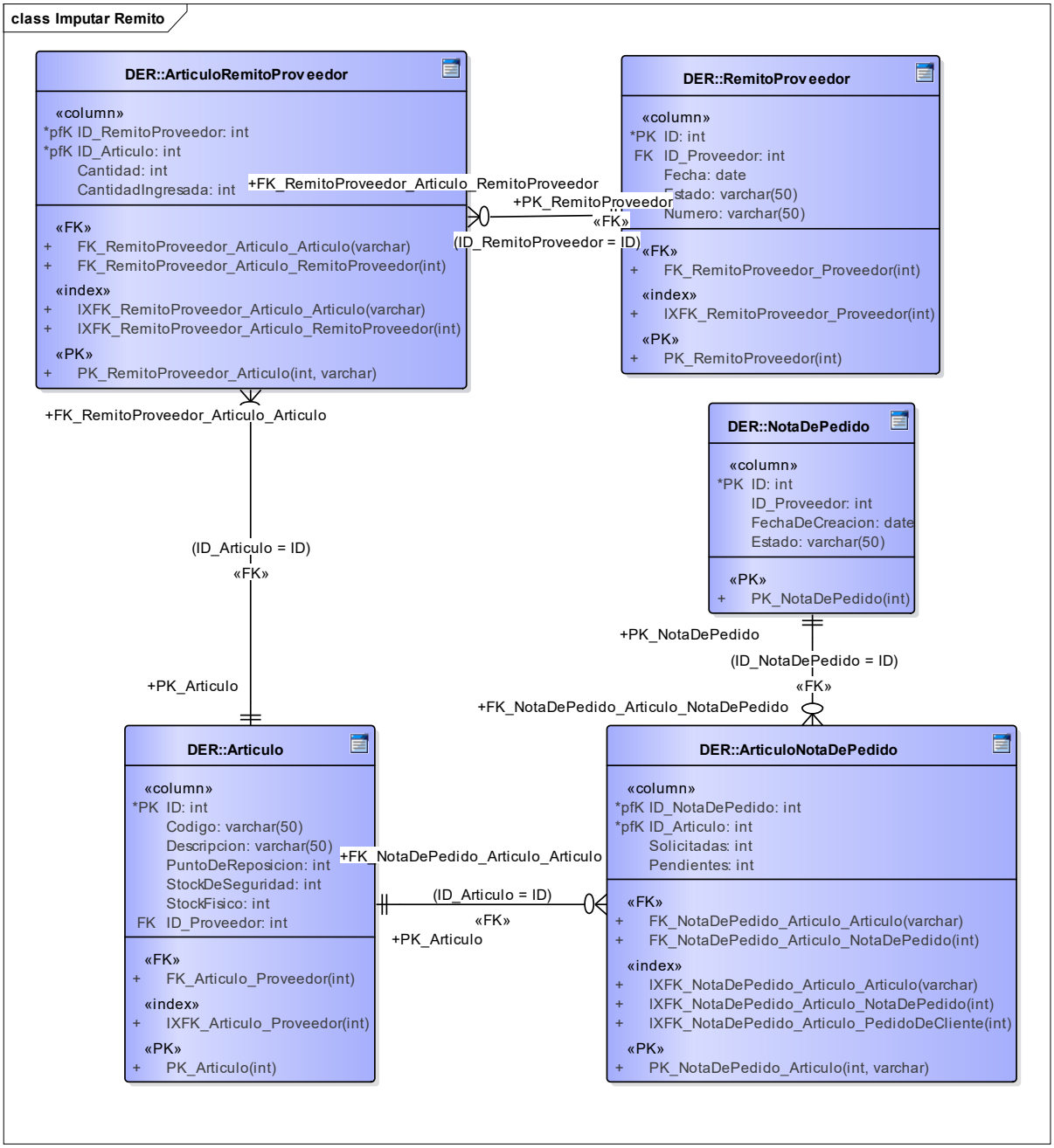
3.4.3 Diagrama de Clases



3.4.4 Diagrama de Secuencia



3.4.5 DER


class Imputar Remito
DER::ArticuloRemitoProveedor
«column»
*pfK ID_RemitoProveedor: int
*pfK ID_Articulo: int
Cantidad: int
CantidadIngresada: int
«FK»
+ FK_RemitoProveedor_Articulo_Articulo(varchar)
+ FK_RemitoProveedor_Articulo_RemitoProveedor(int)
«index»
+ IXFK_RemitoProveedor_Articulo_Articulo(varchar)
+ IXFK_RemitoProveedor_Articulo_RemitoProveedor(int)
«PK»
+ PK_RemitoProveedor_Articulo(int, varchar)
+FK_RemitoProveedor_Articulo_RemitoProveedor
+PK_RemitoProveedor
«FK»
(ID_RemitoProveedor = ID)
DER::RemitoProveedor
«column»
*PK ID: int
FK ID_Proveedor: int
Fecha: date
Estado: varchar(50)
Numero: varchar(50)
«FK»
+ FK_RemitoProveedor_Proveedor(int)
«index»
+ IXFK_RemitoProveedor_Proveedor(int)
«PK»
+ PK_RemitoProveedor(int)
+FK_RemitoProveedor_Articulo_Articulo
(ID_Articulo = ID)
«FK»
+PK_Articulo
DER::NotaDePedido
«column»
*PK ID: int
ID_Proveedor: int
FechaDeCreacion: date
Estado: varchar(50)
«PK»
+ PK_NotaDePedido(int)
+PK_NotaDePedido
(ID_NotaDePedido = ID)
«FK»
+FK_NotaDePedido_Articulo_NotaDePedido
DER::Articulo
«column»
*PK ID: int
Codigo: varchar(50)
Descripcion: varchar(50)
PuntoDeReposicion: int
StockDeSeguridad: int
StockFisico: int
FK ID_Proveedor: int
«FK»
+ FK_Articulo_Proveedor(int)
«index»
+ IXFK_Articulo_Proveedor(int)
«PK»
+ PK_Articulo(int)
+FK_NotaDePedido_Articulo_Articulo
(ID_Articulo = ID)
«FK»
+PK_Articulo
DER::ArticuloNotaDePedido
«column»
*pfK ID_NotaDePedido: int
*pfK ID_Articulo: int
Solicitadas: int
Pendientes: int
«FK»
+ FK_NotaDePedido_Articulo_Articulo(varchar)
+ FK_NotaDePedido_Articulo_NotaDePedido(int)
«index»
+ IXFK_NotaDePedido_Articulo_Articulo(varchar)
+ IXFK_NotaDePedido_Articulo_NotaDePedido(int)
+ IXFK_NotaDePedido_Articulo_PedidoDeCliente(int)
«PK»
+ PK_NotaDePedido_Articulo(int, varchar)

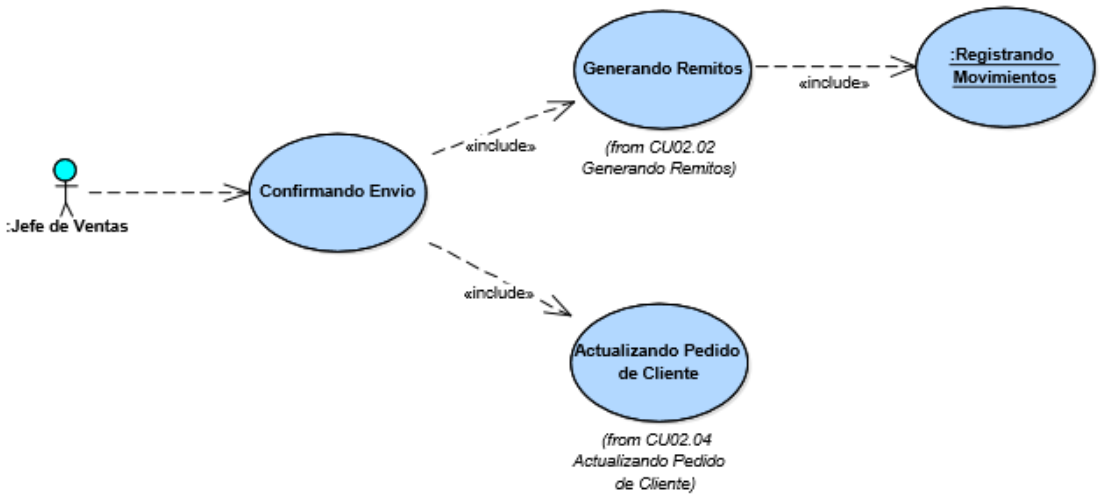
3.5 Confirmando Envío CU02.01

3.5.1 Especificación

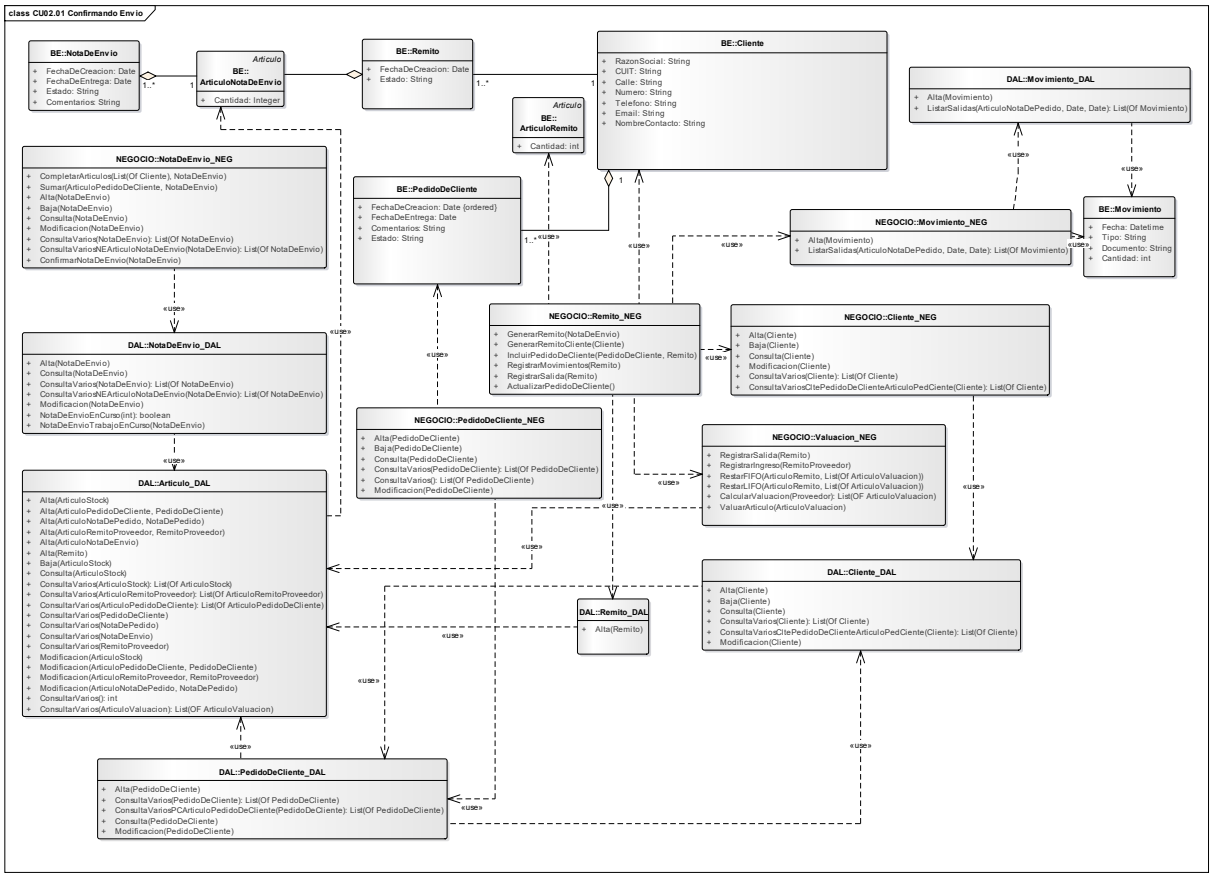
1 - Nombre	Confirmando Envío
Descripción:	En esta instancia los vendedores ya han confirmado las cantidades a enviar, el jefe de ventas autorizó dichas cantidades y el encargado de depósito verificó la existencia física de esa mercadería. La confirmación del envío genera los remitos correspondientes para cada uno de los clientes, la nota de envío queda registrada en estado confirmada para que el jefe de playa pueda realizar el control correspondiente. Además se actualizan las cantidades pendientes de entrega en los pedidos de los clientes, se registran los movimientos de salida de la mercadería y se actualizan las tablas de valorización de mercadería.
Actores:	Jefe de Ventas
2 - Flujo de Eventos	
Disparador/Condición:	El jefe de ventas inicia el proceso de confirmar mercadería.
Flujo Básico:	
	1. El sistema muestra en pantalla las notas de pedido que se encuentran en estado *Revisada.* 2. El usuario selecciona la nota de envío a confirmar. 3. El sistema muestra un listado con el detalle de todos los artículos relacionados con la nota de envío seleccionada. Para cada artículo debe detallar los siguientes puntos: ID Código Descripción Cantidad 4. El usuario completa los comentarios relacionados a la nota de envío. 5. El usuario confirma la nota de envío. 6. El sistema muestra en pantalla que la actualización ha sido realizada. 7. Fin de caso de uso.
Flujos Alternativos:	
	N/A
3 - Requerimientos Especiales	
	N/A
4 - Precondiciones:	
	El usuario debe tener permiso para generar la Nota de Pedido.

5 - Postcondiciones:	
	Se generaron los *Remitos* correspondientes a cada vendedor. Se registraron los *Movimientos* de mercadería. Se actualizo la tabla de *Valuación* de mercadería (FIFO|LIFO). Se actualizaron las *Notas de Pedido* relacionadas al envío actual. La actualización de la *Nota de Envío* queda registrada en la base de datos. El usuario visualizó en pantalla el resultado de la actualización de la nota de envío.
6 - Puntos de Extensión	
	N/A

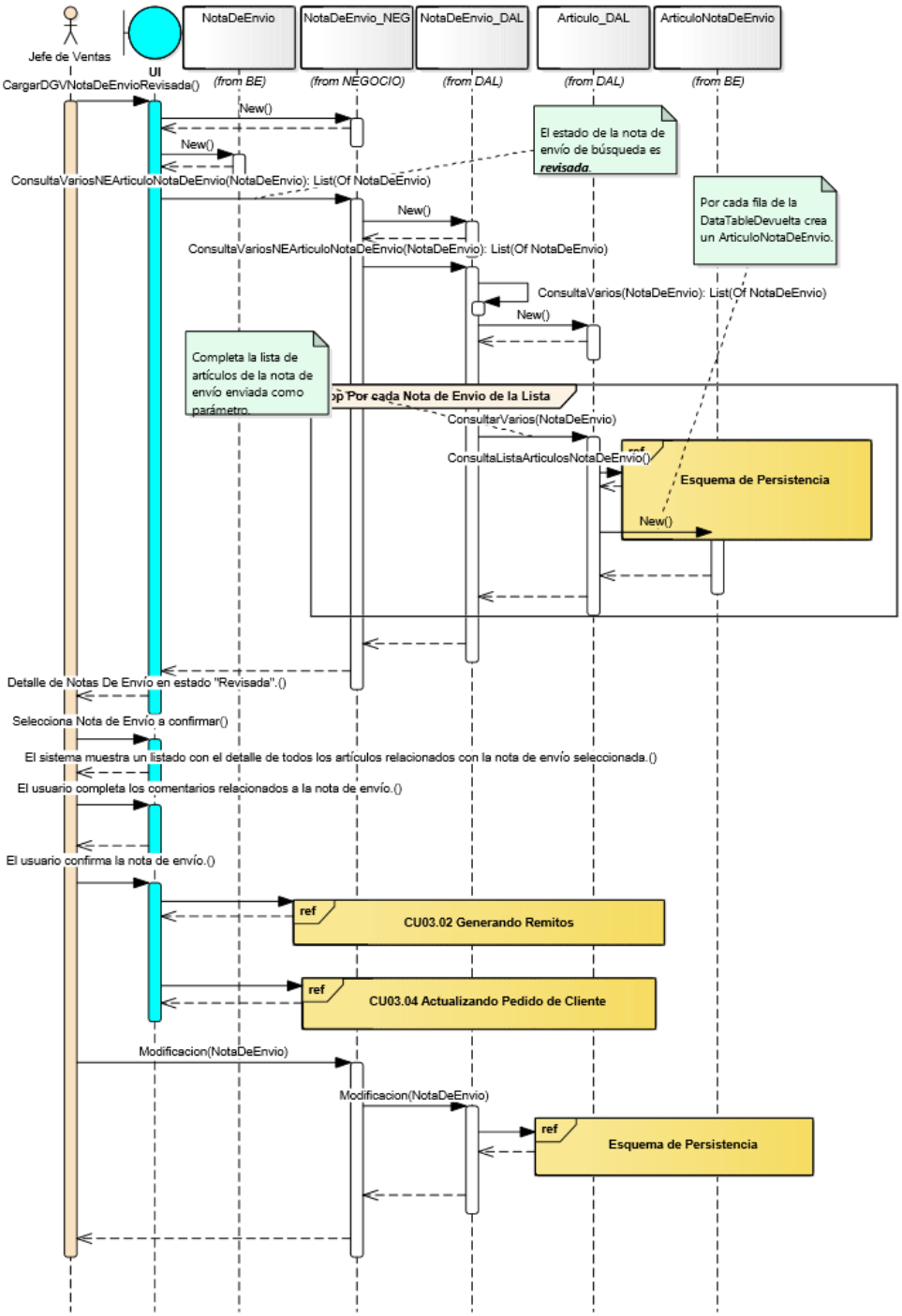
3.5.2 Gráfico de Caso de uso



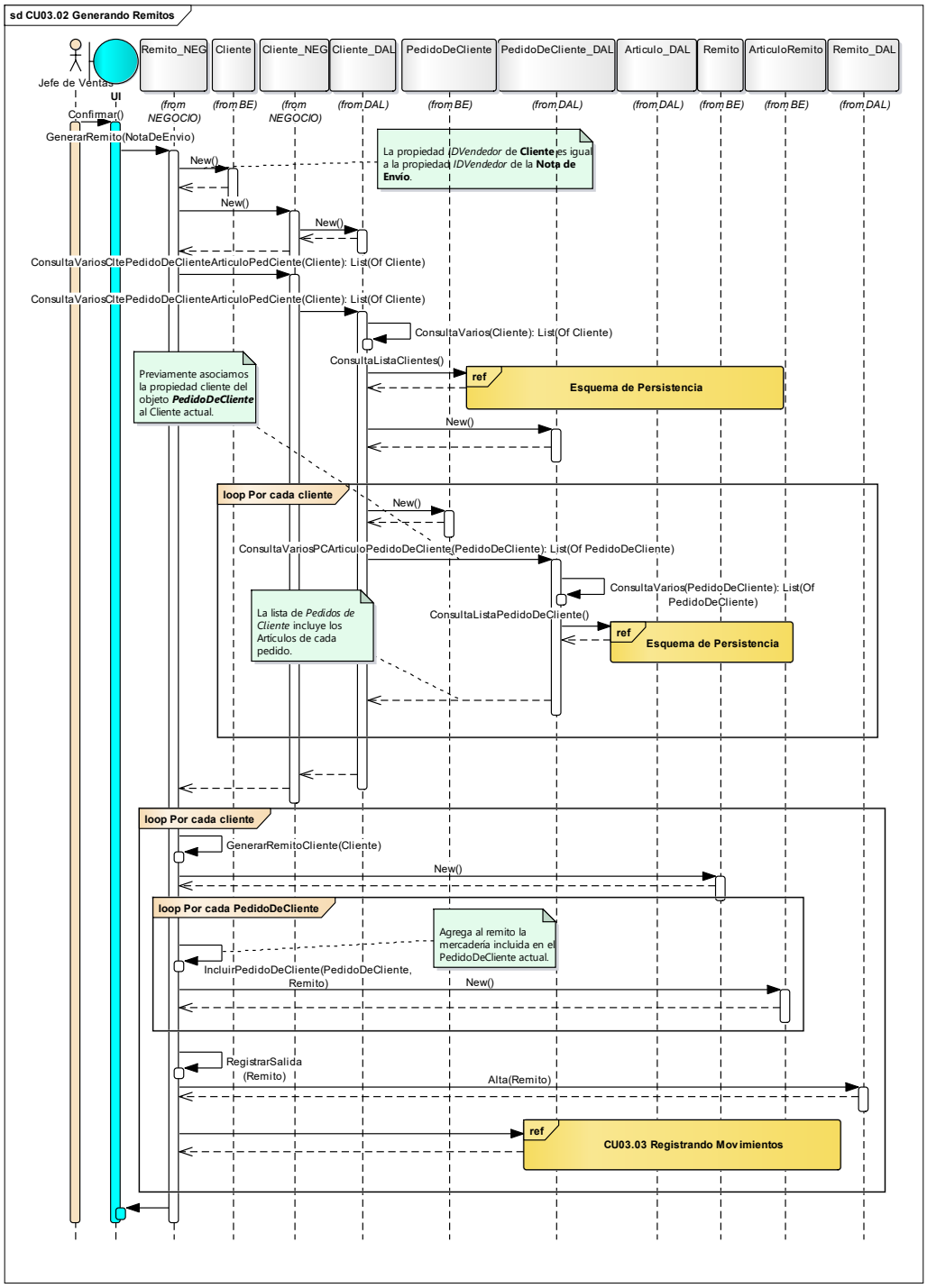
3.5.3 Diagrama de Clases



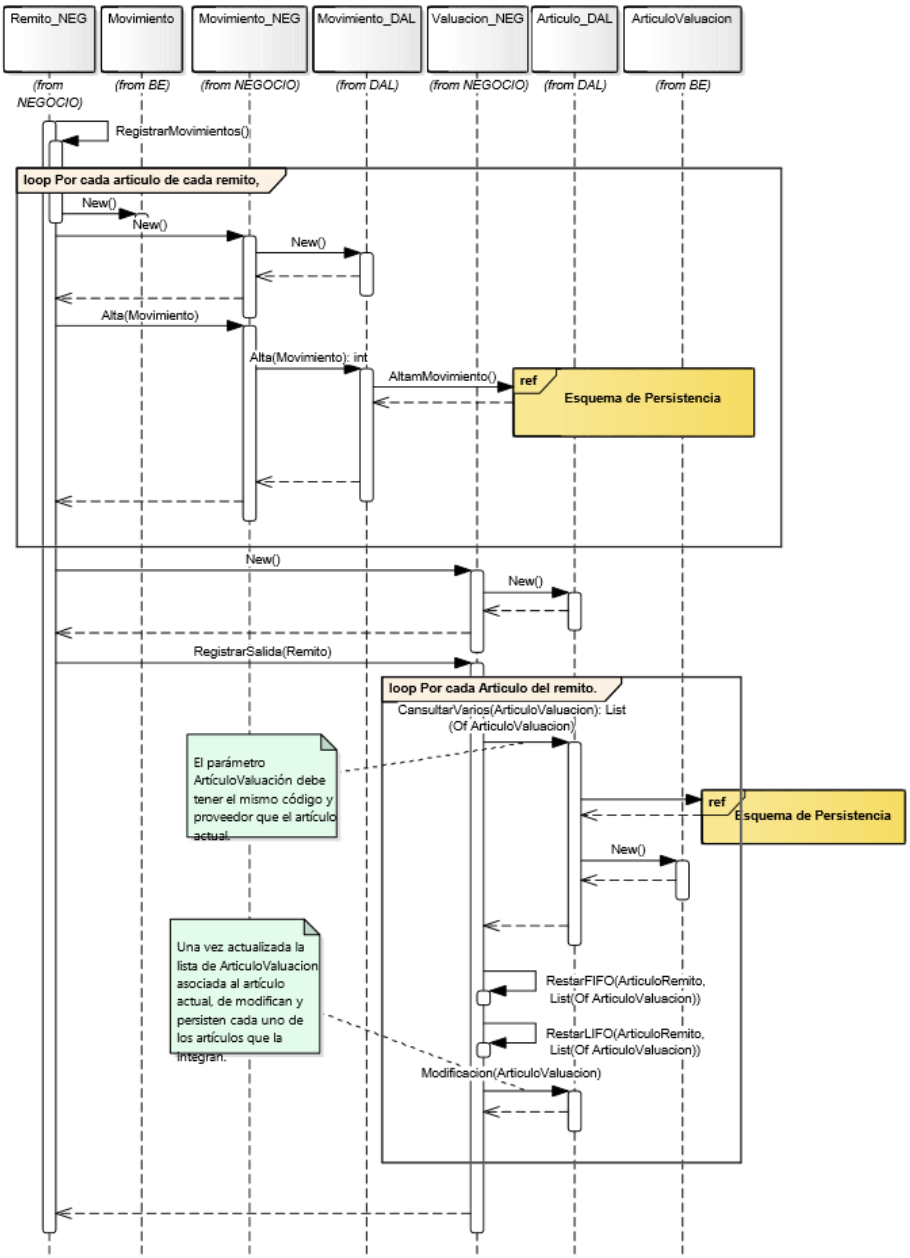
3.5.4 Diagrama de Secuencia Confirmando Envío



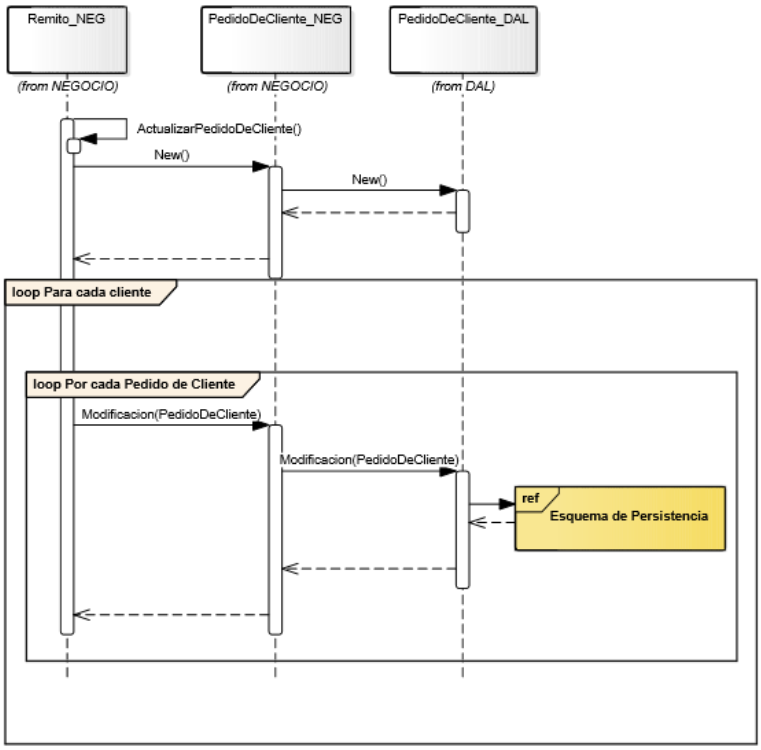
3.5.5 Diagrama de Secuencia Generando Remitos



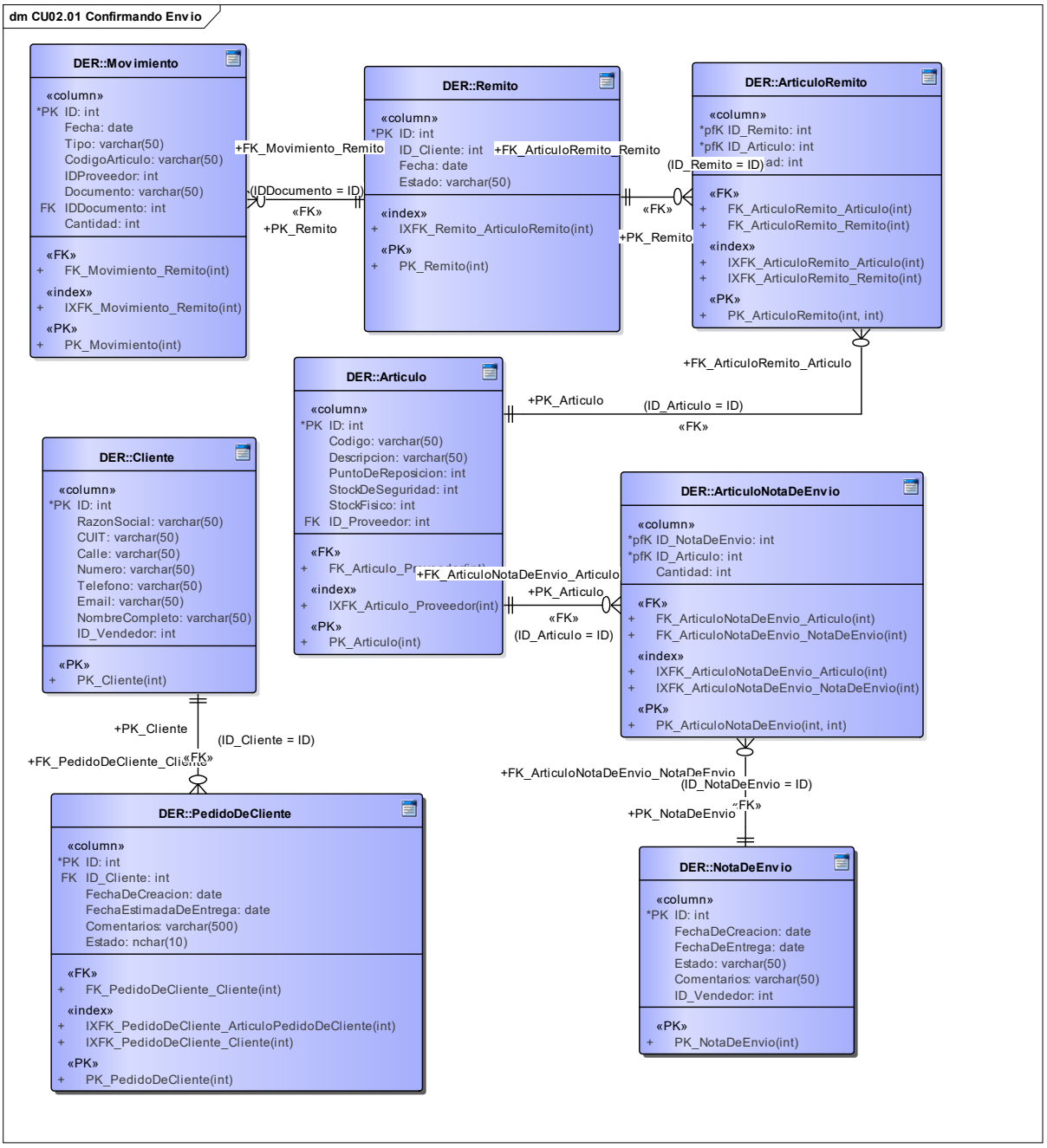
3.5.6 Diagrama de Secuencia Registrando Movimientos



3.5.7 Diagrama de Secuencia Actualizando Pedido de Cliente



3.5.8 DER


dm CU02.01 Confirmando Envio
DER::Movimiento
«column»
*PK ID: int
Fecha: date
Tipo: varchar(50)
CodigoArticulo: varchar(50)
IDProveedor: int
Documento: varchar(50)
FK IDDocumento: int
Cantidad: int
«FK»
+ FK_Movimiento_Remito(int)
«index»
+ IXFK_Movimiento_Remito(int)
«PK»
+ PK_Movimiento(int)
+FK_Movimiento_Remito
(IDDocumento = ID)
«FK»
+PK_Remito
DER::Remito
«column»
*PK ID: int
ID_Cliente: int
Fecha: date
Estado: varchar(50)
«index»
+ IXFK_Remito_ArticuloRemito(int)
«PK»
+ PK_Remito(int)
+FK_ArticuloRemito_Remito
(ID_Remito = ID)
«FK»
+PK_Remito
DER::ArticuloRemito
«column»
*pfK ID_Remito: int
*pfK ID_Articulo: int
ad: int
«FK»
+ FK_ArticuloRemito_Articulo(int)
+ FK_ArticuloRemito_Remito(int)
«index»
+ IXFK_ArticuloRemito_Articulo(int)
+ IXFK_ArticuloRemito_Remito(int)
«PK»
+ PK_ArticuloRemito(int, int)
+FK_ArticuloRemito_Articulo
DER::Articulo
«column»
*PK ID: int
Codigo: varchar(50)
Descripcion: varchar(50)
PuntoDeReposicion: int
StockDeSeguridad: int
StockFisico: int
FK ID_Proveedor: int
«FK»
+ FK_Articulo_Pr
«index»
+ IXFK_Articulo_Proveedor(int)
«PK»
+ PK_Articulo(int)
+PK_Articulo
(ID_Articulo = ID)
«FK»
+FK_ArticuloNotaDeEnvio_Articulo
+PK_Articulo
(ID_Articulo = ID)
«FK»
DER::Cliente
«column»
*PK ID: int
RazonSocial: varchar(50)
CUIT: varchar(50)
Calle: varchar(50)
Numero: varchar(50)
Telefono: varchar(50)
Email: varchar(50)
NombreCompleto: varchar(50)
ID_Vendedor: int
«PK»
+ PK_Cliente(int)
+PK_Cliente
(ID_Cliente = ID)
«FK»
+FK_PedidoDeCliente_Cliente
DER::ArticuloNotaDeEnvio
«column»
*pfK ID_NotaDeEnvio: int
*pfK ID_Articulo: int
Cantidad: int
«FK»
+ FK_ArticuloNotaDeEnvio_Articulo(int)
+ FK_ArticuloNotaDeEnvio_NotaDeEnvio(int)
«index»
+ IXFK_ArticuloNotaDeEnvio_Articulo(int)
+ IXFK_ArticuloNotaDeEnvio_NotaDeEnvio(int)
«PK»
+ PK_ArticuloNotaDeEnvio(int, int)
+FK_ArticuloNotaDeEnvio_NotaDeEnvio
(ID_NotaDeEnvio = ID)
«FK»
+PK_NotaDeEnvio
DER::PedidoDeCliente
«column»
*PK ID: int
FK ID_Cliente: int
FechaDeCreacion: date
FechaEstimadaDeEntrega: date
Comentarios: varchar(500)
Estado: nchar(10)
«FK»
+ FK_PedidoDeCliente_Cliente(int)
«index»
+ IXFK_PedidoDeCliente_ArticuloPedidoDeCliente(int)
+ IXFK_PedidoDeCliente_Cliente(int)
«PK»
+ PK_PedidoDeCliente(int)
DER::NotaDeEnvio
«column»
*PK ID: int
FechaDeCreacion: date
FechaDeEntrega: date
Estado: varchar(50)
Comentarios: varchar(50)
ID_Vendedor: int
«PK»
+ PK_NotaDeEnvio(int)

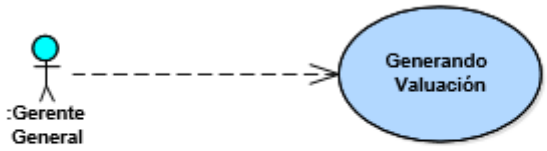
3.6 Generando Valuación CU03.02

3.6.1 Especificación

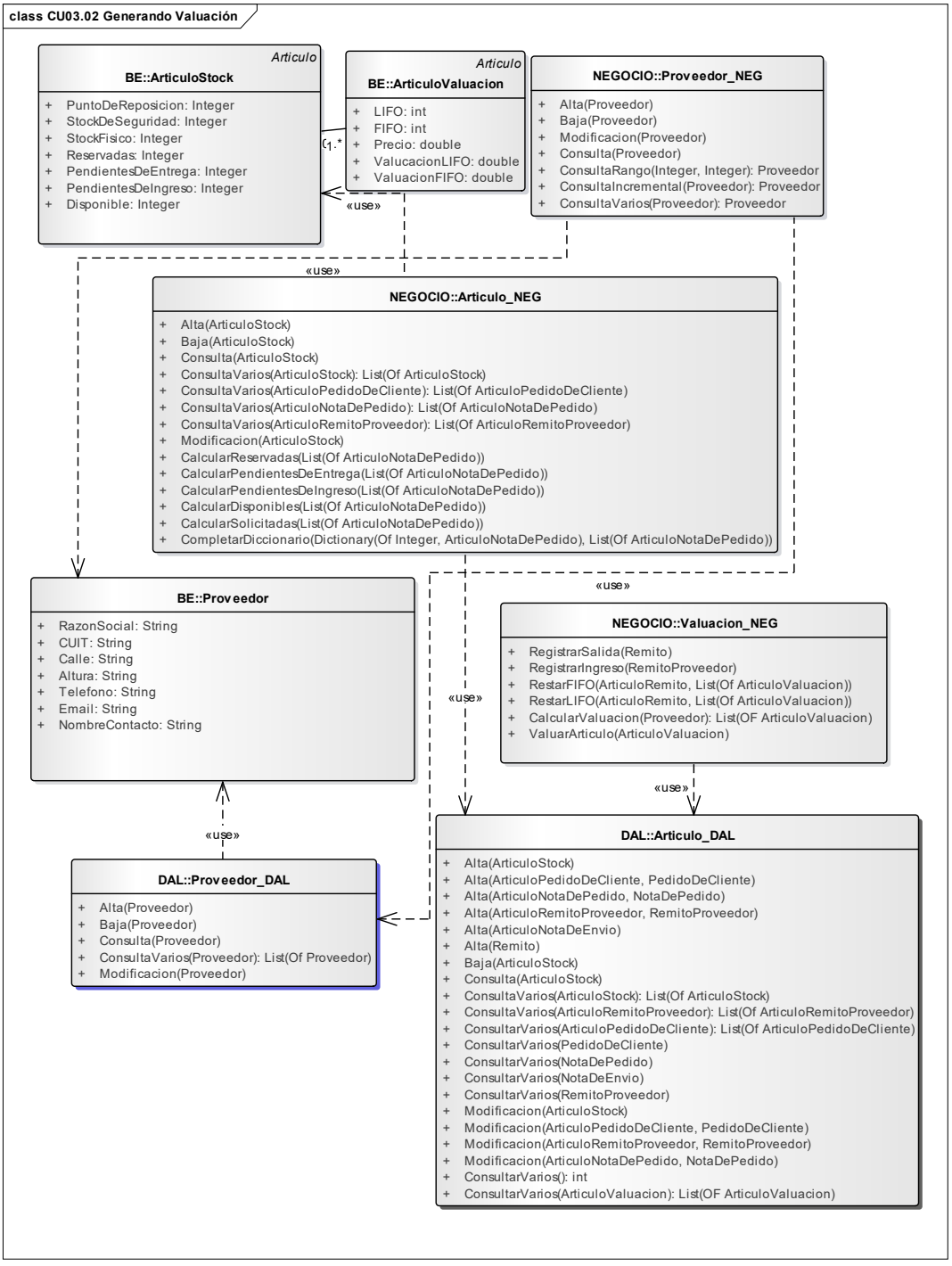
1 - Nombre:	Generar Valuación
Descripción:	Permite obtener la valorización del Precio Promedio Ponderado(PPP) bajo alguna de las siguientes modalidades: First In First Out(FIFO)-Last In First Out(LIFO)
Actores:	Gerente General
2 - Flujo de Eventos	
Disparador/Condición:	El Gerente General inicia el proceso de valuación de la mercadería en existencia para informar a contaduría o tomar decisiones estratégicas.
Flujo Básico:	
	1. El usuario inicia el proceso de valuación de inventario. 2. El sistema muestra un listado con los proveedores disponibles. 3. El usuario selecciona el proveedor para los artículos considerados en la valuación. 4. El sistema muestra en pantalla un detalle con la valuación de cada uno de los artículos y la valuación del monto total de la mercadería. 5. Fin de caso de uso.
Flujos Alternativos:	
3 - Requerimientos Especiales	
4 - Precondiciones:	
	N/A
5 - Postcondiciones:	
	El sistema devuelve un listado de cada uno de los artículos con su valorización. El sistema devuelve el monto total de valorización, dicho monto es la sumatoria de los montos asociados a cada uno de los artículos.
6 - Puntos de Extensión	

	N/A

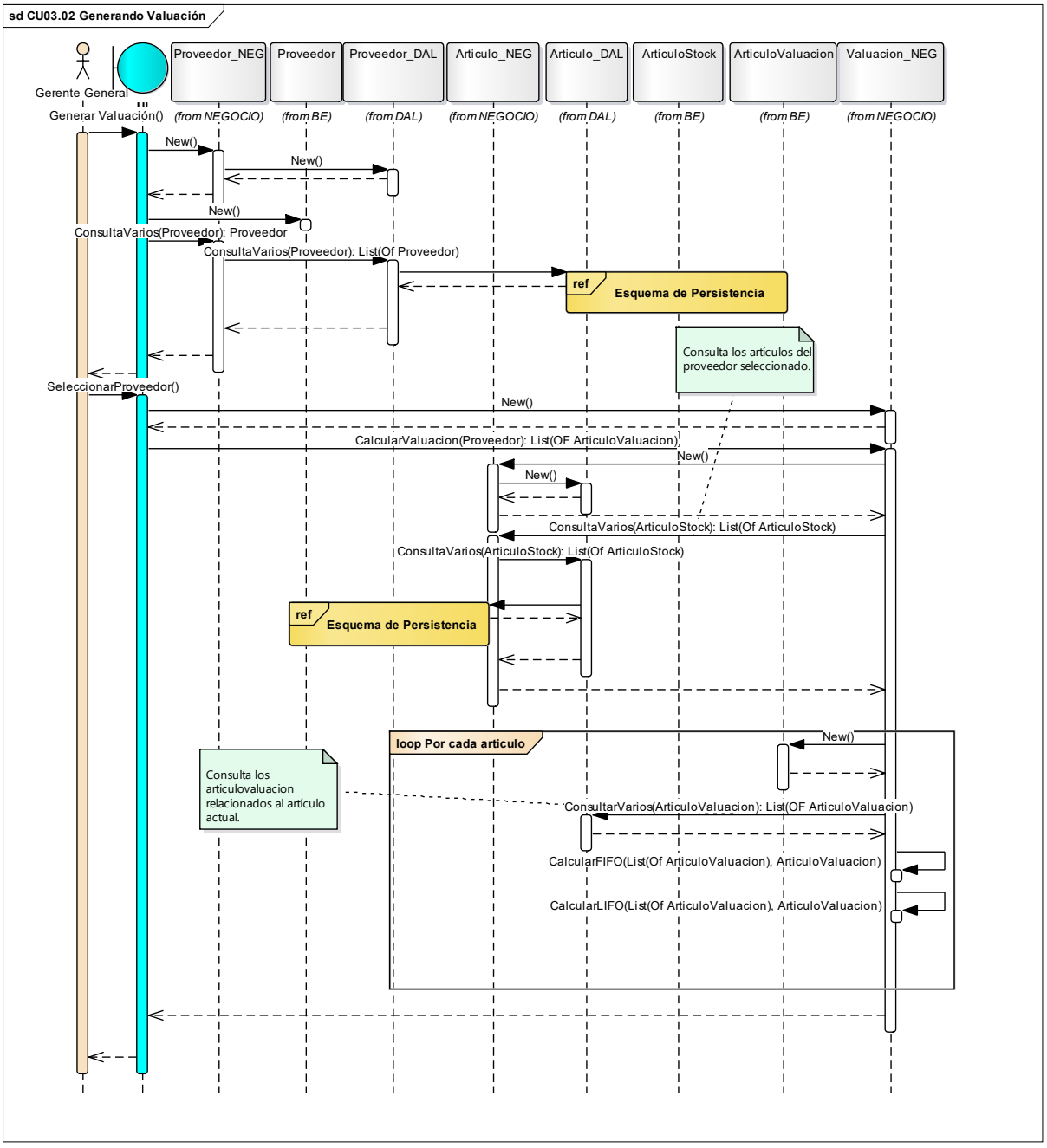
3.6.2 Gráfico de Caso de uso



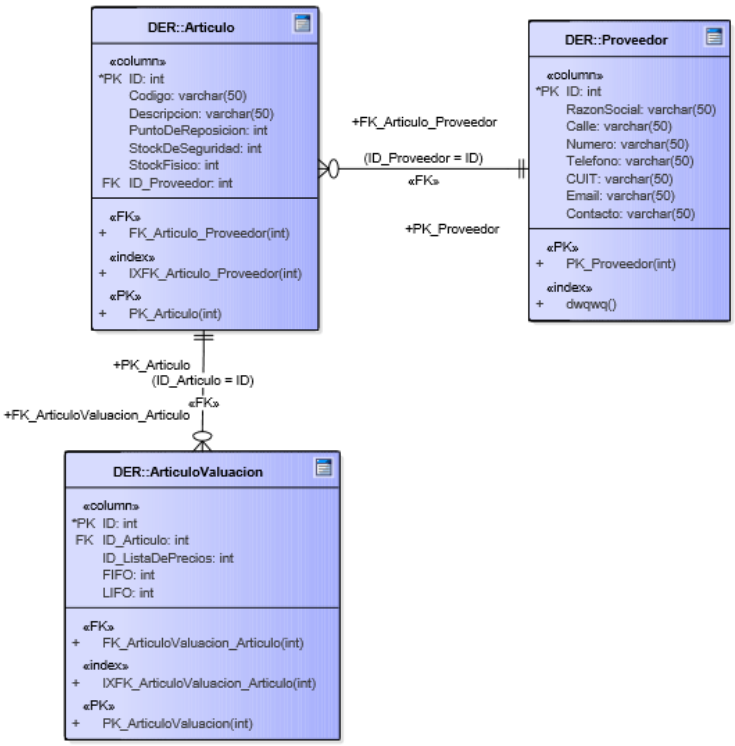
3.6.3 Diagrama de Clases



3.6.4 Diagrama de Secuencia



3.6.5 DER


DER::Articulo
«column»
*PK ID: int
Codigo: varchar(50)
Descripcion: varchar(50)
PuntoDeReposicion: int
StockDeSeguridad: int
StockFisico: int
FK ID_Proveedor: int
«FK»
+ FK_Articulo_Proveedor(int)
«index»
+ IXFK_Articulo_Proveedor(int)
«PK»
+ PK_Articulo(int)
+FK_Articulo_Proveedor
(ID_Proveedor = ID)
«FK»
+PK_Proveedor
DER::Proveedor
«column»
*PK ID: int
RazonSocial: varchar(50)
Calle: varchar(50)
Numero: varchar(50)
Telefono: varchar(50)
CUIT: varchar(50)
Email: varchar(50)
Contacto: varchar(50)
«PK»
+ PK_Proveedor(int)
«index»
+ dwqwq()
+PK_Articulo
(ID_Articulo = ID)
«FK»
+FK_ArticuloValuacion_Articulo
DER::ArticuloValuacion
«column»
*PK ID: int
FK ID_Articulo: int
ID_ListaDePrecios: int
FIFO: int
LIFO: int
«FK»
+ FK_ArticuloValuacion_Articulo(int)
«index»
+ IXFK_ArticuloValuacion_Articulo(int)
«PK»
+ PK_ArticuloValuacion(int)

Aspectos técnicos que provee el sistema de información

4. Aspectos técnicos que provee el sistema de información

4.1 Gestión de Perfiles de Usuario

Objetivo

El sistema deberá permitir la administración integral de accesos y funcionalidades para cada usuario, garantizando un esquema de seguridad flexible y escalable. Las restricciones de uso o visualización de datos se definirán según el perfil de cada usuario, configurado por el Administrador del Sistema. Para ello, el sistema ofrecerá la posibilidad de crear **perfiles de usuario**, los cuales agruparán uno o varios permisos atómicos, permitiendo una asignación granular y coherente con la estructura organizativa.

Descripción funcional

La administración de permisos se implementará mediante un módulo de seguridad basado en el modelo Usuario–Familia–Patente, estructurado de la siguiente forma:

- Usuario: Representa a la persona que interactúa con el sistema. A cada usuario se le podrán asignar permisos individuales o por herencia, a través de familias de permisos.
- Familia: Conjunto lógico de permisos asociados a una funcionalidad o proceso del negocio. Cada familia deberá contener al menos una patente (permiso atómico), pudiendo además incluir otras familias. Su denominación deberá ser descriptiva, reflejando la función o área que agrupa.
- Patente: Constituye el permiso más elemental y específico que puede otorgarse a un usuario. Cada patente estará vinculada a una funcionalidad puntual del sistema, como una operación CRUD o una acción de control.

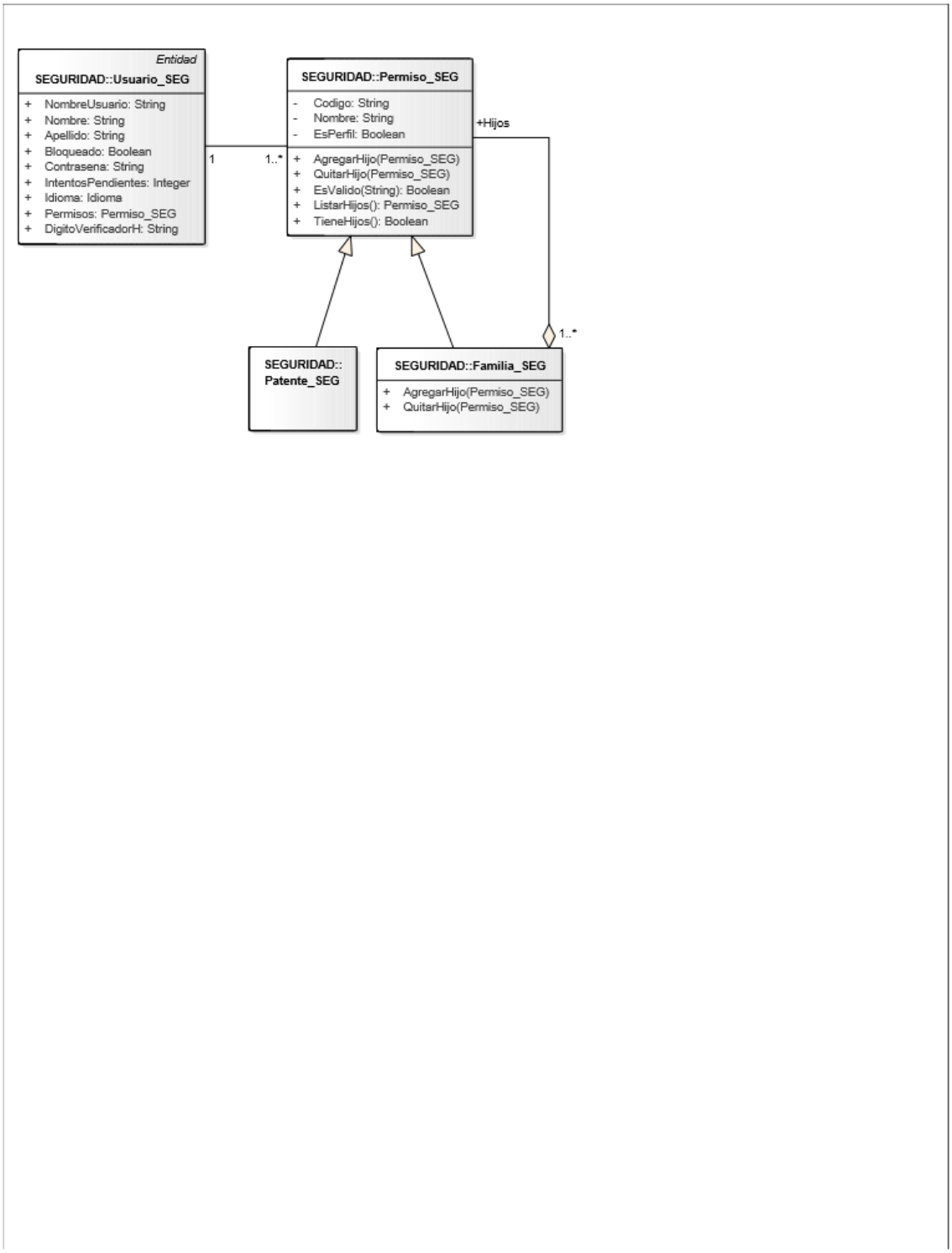
Comportamiento y diseño

Una vez creada una familia, será posible **asignarle patentes y/o subfamilias**, de modo que todos los usuarios asociados hereden automáticamente los permisos correspondientes. Para soportar esta estructura jerárquica y reutilizable, el sistema implementará el **patrón de diseño *Composite***.
Este patrón permite construir objetos complejos (*Permiso*) a partir de otros más simples y homogéneos (*Familia* y *Patente*), mediante una composición recursiva organizada en una **estructura de árbol**, facilitando la gestión, extensión y trazabilidad del modelo de seguridad.

Listado de patentes definidas en el sistema

Código	Permiso
PUSU001	Alta Usuario
PUSU002	Baja Usuario
PUSU003	Modificación Usuario
PUSU004	Consulta Usuario
PFAM001	Alta Familia
PFAM002	Baja Familia
PFAM003	Modificación Familia
PFAM004	Consulta Familia
PBAC001	Realizar BackUp
PRES001	Realizar Restore
PCOM000	Acceso a módulo Compras
PVEN000	Acceso a módulo Ventas
PCON000	Acceso a módulo Contabilidad
PSIS000	Acceso a módulo Sistema

Diagrama de clases



DER

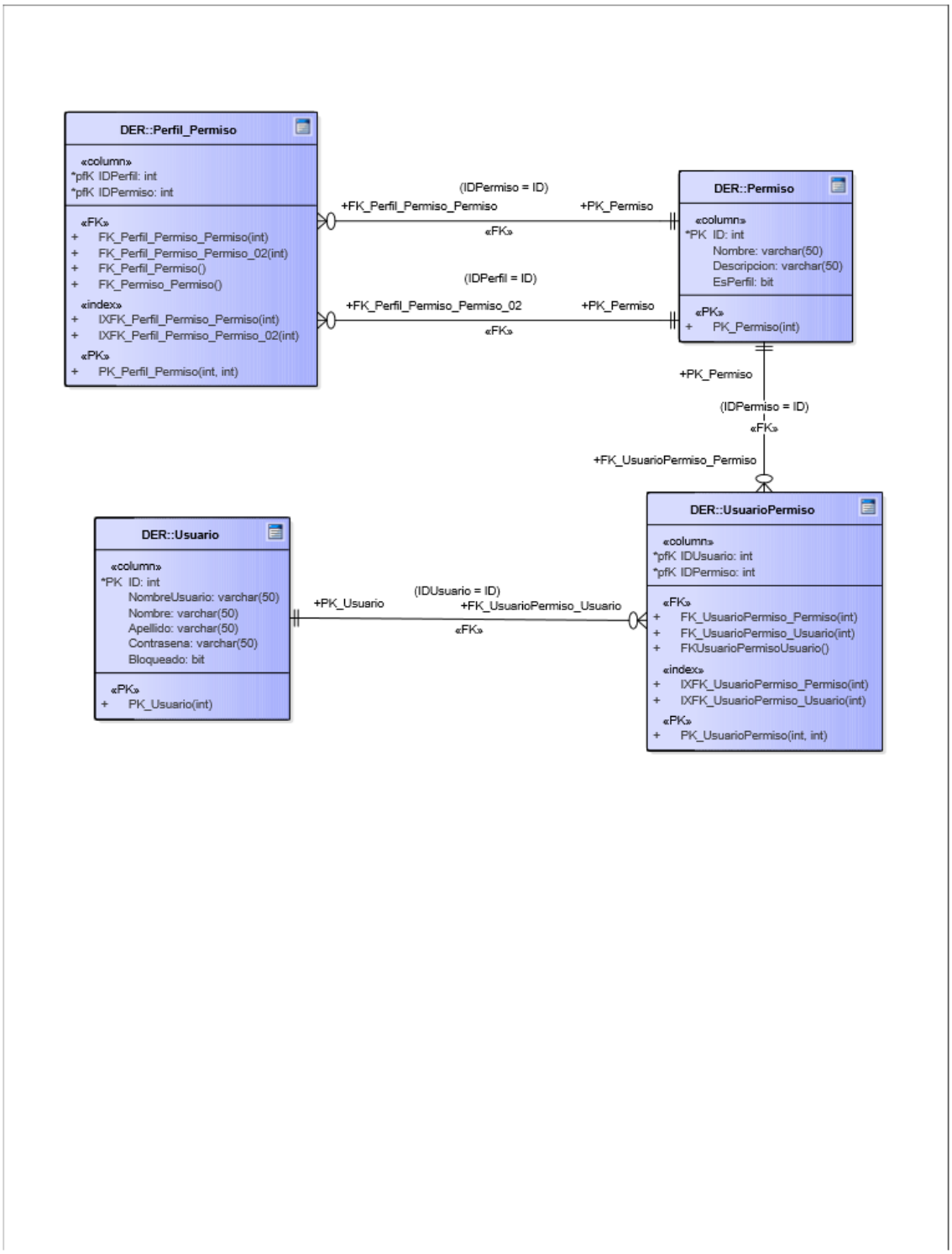
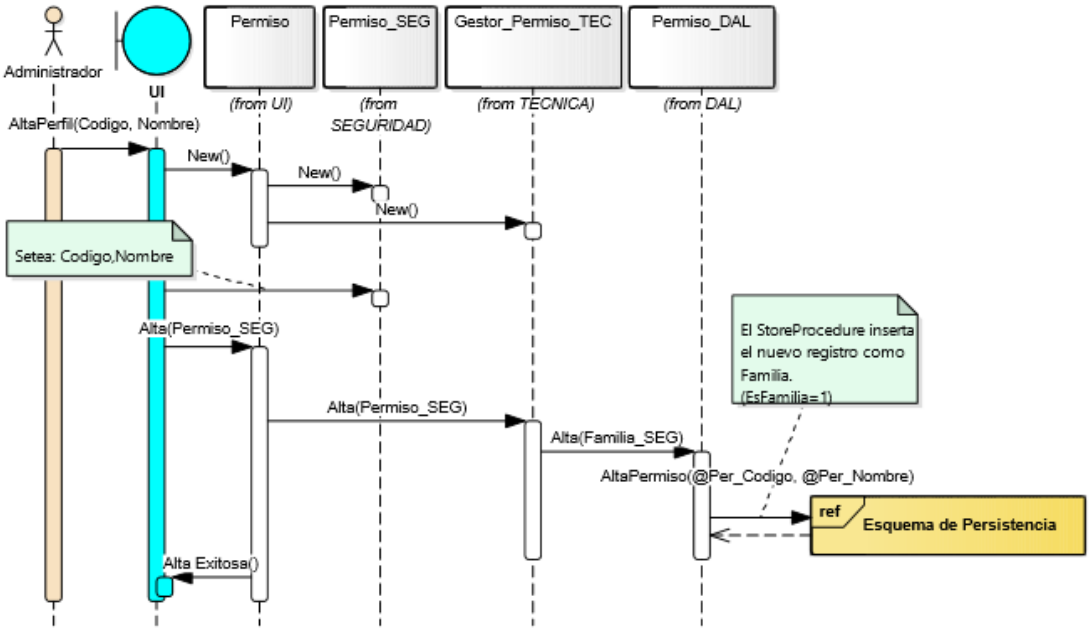
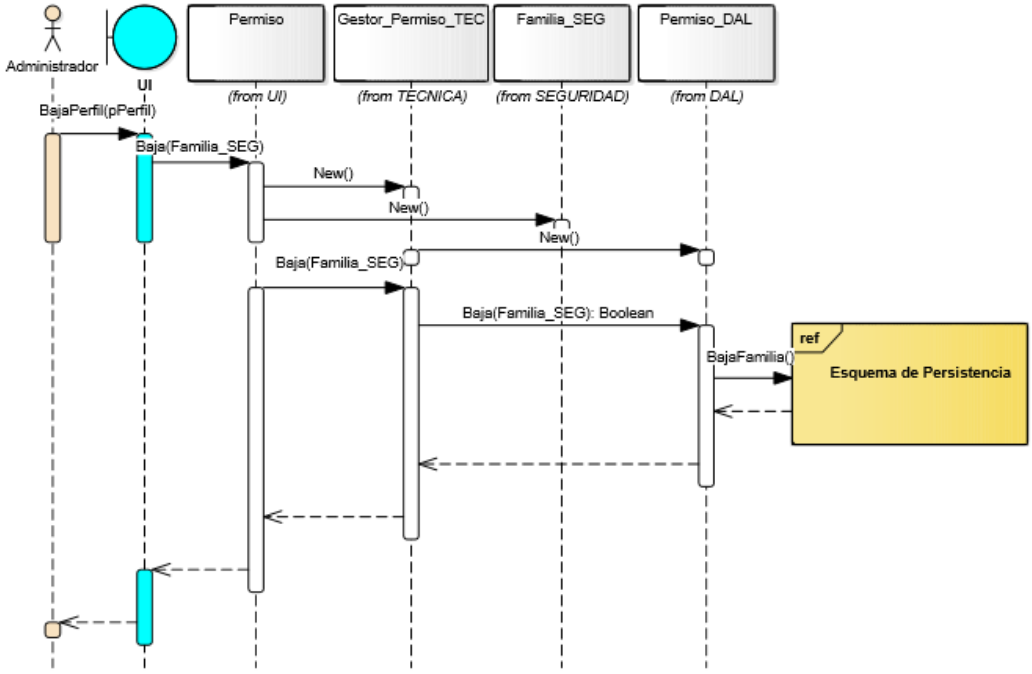

DER::Perfil_Permiso
«column»
*pfK IDPerfil: int
*pfK IDPermiso: int
«FK»
+ FK_Perfil_Permiso_Permiso(int)
+ FK_Perfil_Permiso_Permiso_02(int)
+ FK_Perfil_Permiso()
+ FK_Permiso_Permiso()
«index»
+ IXFK_Perfil_Permiso_Permiso(int)
+ IXFK_Perfil_Permiso_Permiso_02(int)
«PK»
+ PK_Perfil_Permiso(int, int)
(IDPermiso = ID)
+FK_Perfil_Permiso_Permiso
+PK_Permiso
«FK»
(IDPerfil = ID)
+FK_Perfil_Permiso_Permiso_02
+PK_Permiso
«FK»
DER::Permiso
«column»
*PK ID: int
Nombre: varchar(50)
Descripcion: varchar(50)
EsPerfil: bit
«PK»
+ PK_Permiso(int)
+PK_Permiso
(IDPermiso = ID)
«FK»
+FK_UsuarioPermiso_Permiso
DER::Usuario
«column»
*PK ID: int
NombreUsuario: varchar(50)
Nombre: varchar(50)
Apellido: varchar(50)
Contrasena: varchar(50)
Bloqueado: bit
«PK»
+ PK_Usuario(int)
+PK_Usuario
(IDUsuario = ID)
+FK_UsuarioPermiso_Usuario
«FK»
DER::UsuarioPermiso
«column»
*pfK IDUsuario: int
*pfK IDPermiso: int
«FK»
+ FK_UsuarioPermiso_Permiso(int)
+ FK_UsuarioPermiso_Usuario(int)
+ FKUsuarioPermisoUsuario()
«index»
+ IXFK_UsuarioPermiso_Permiso(int)
+ IXFK_UsuarioPermiso_Usuario(int)
«PK»
+ PK_UsuarioPermiso(int, int)

Diagrama de Secuencia

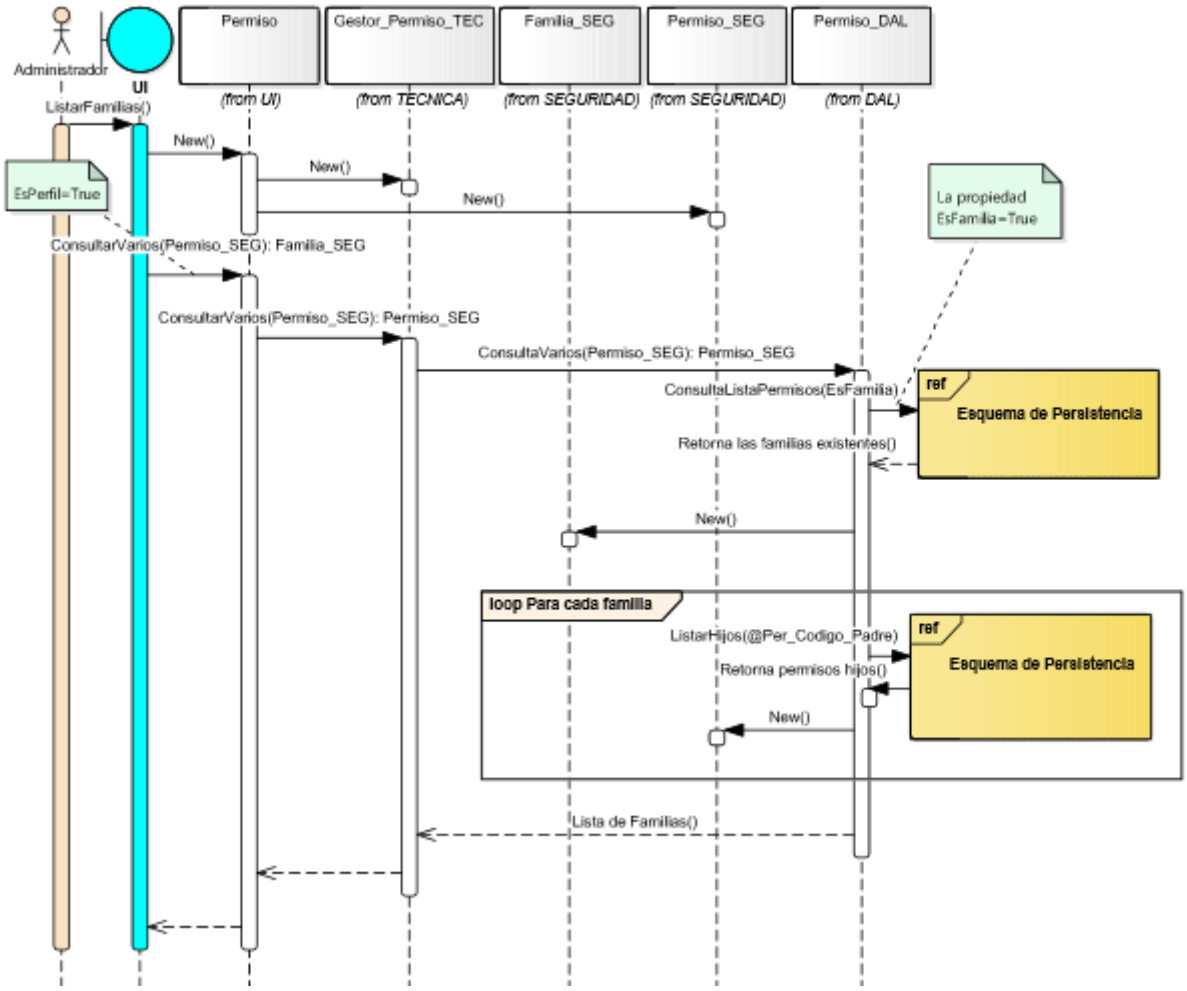
Alta Familia



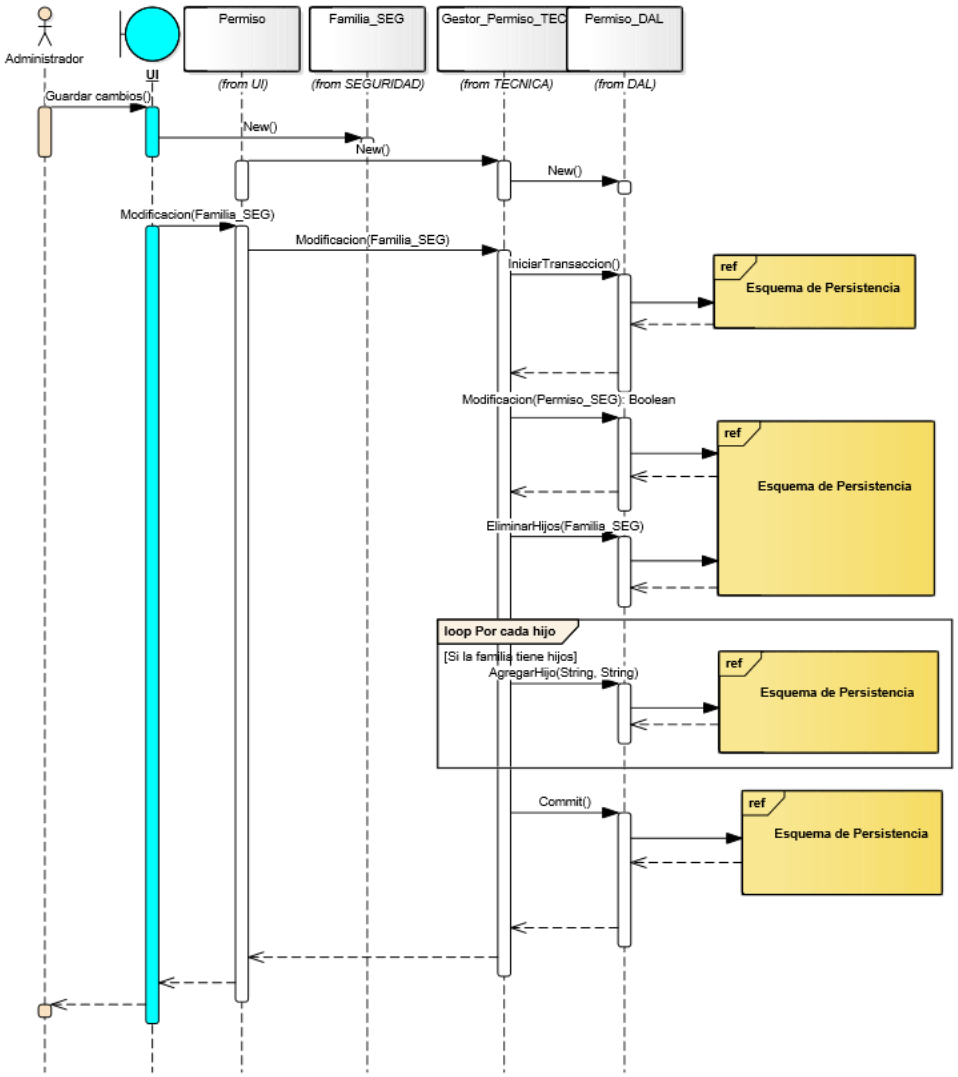
Baja Familia



Lista Familias



Modificación



4.2 Gestión de LogIn / LogOut del Sistema

Objetivo

El sistema deberá garantizar la identificación y autenticación segura de los usuarios, asignando a cada uno un campo único e irrepetible (nombre de usuario) y una contraseña asociada. Una vez validado el acceso, el sistema cargará en memoria los permisos habilitados de acuerdo con el perfil del usuario y ajustará el entorno operativo conforme a sus preferencias personales (por ejemplo, idioma o configuración visual).

Descripción detallada de cómo funciona

El usuario solicita acceder al sistema. El sistema muestra la pantalla de inicio de sesión. El usuario debe ingresar el nombre de usuario que lo identifica y la contraseña. Si el usuario no completa alguno de los campos el sistema debe solicitar que complete el campo faltante. En caso de que complete ambos campos el sistema resta el contador de intentos pendientes en 1. La cantidad inicial es 3.
En el caso de producirse un error el sistema lo registra en bitácora.
En el caso de no existir el usuario en la base de datos el sistema informa usuario o contraseña incorrecta.
En el caso de que la cantidad de intentos pendientes sea igual a cero el sistema informa al usuario que ha superado el número máximo de intentos y procede a cerrar la aplicación.
Una vez que el sistema encuentra exitosamente el usuario a ingresar verifica que no se encuentre bloqueado. Si se encuentra bloqueado informa al usuario y registra en bitácora.
Lugo verifica que el usuario y la contraseña sea la correcta. En el caso de que la contraseña no sea correcta verifica la cantidad de intentos pendientes. Si esta cantidad ha llegado a cero bloquea el usuario.
El sistema debe registrar en bitácora el ingreso del usuario. Luego debe verificar la integridad de los datos. Finalmente debe adaptar la pantalla principal a las preferencias del usuario.

Diagrama de clases

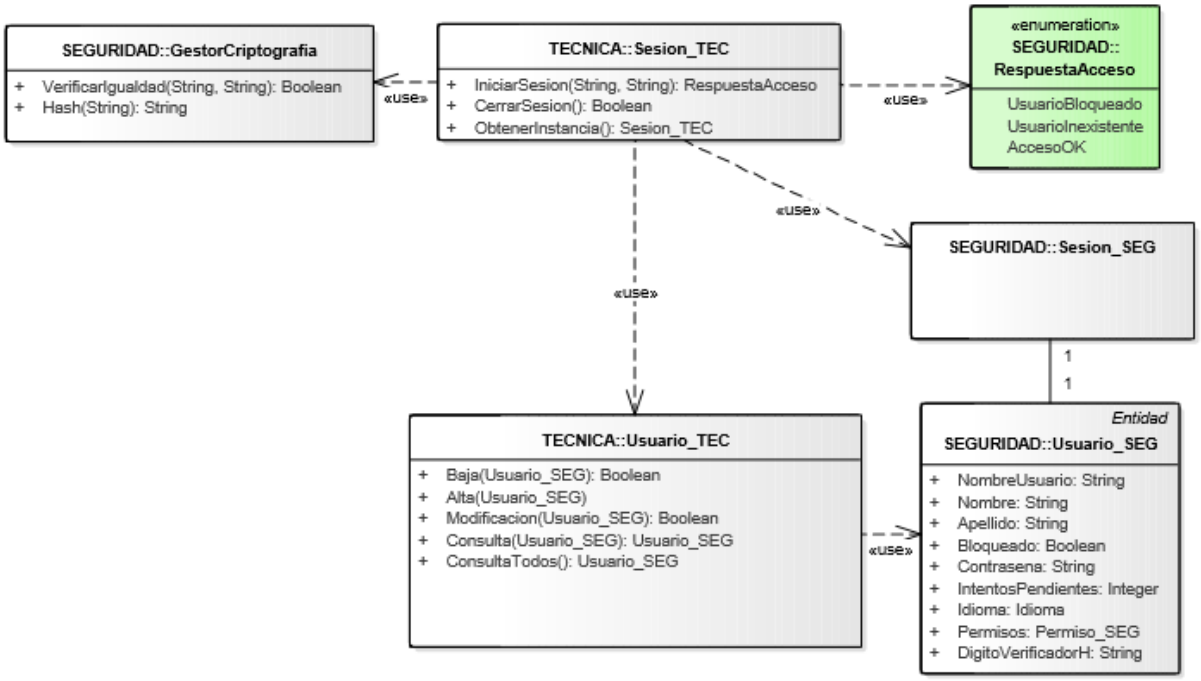
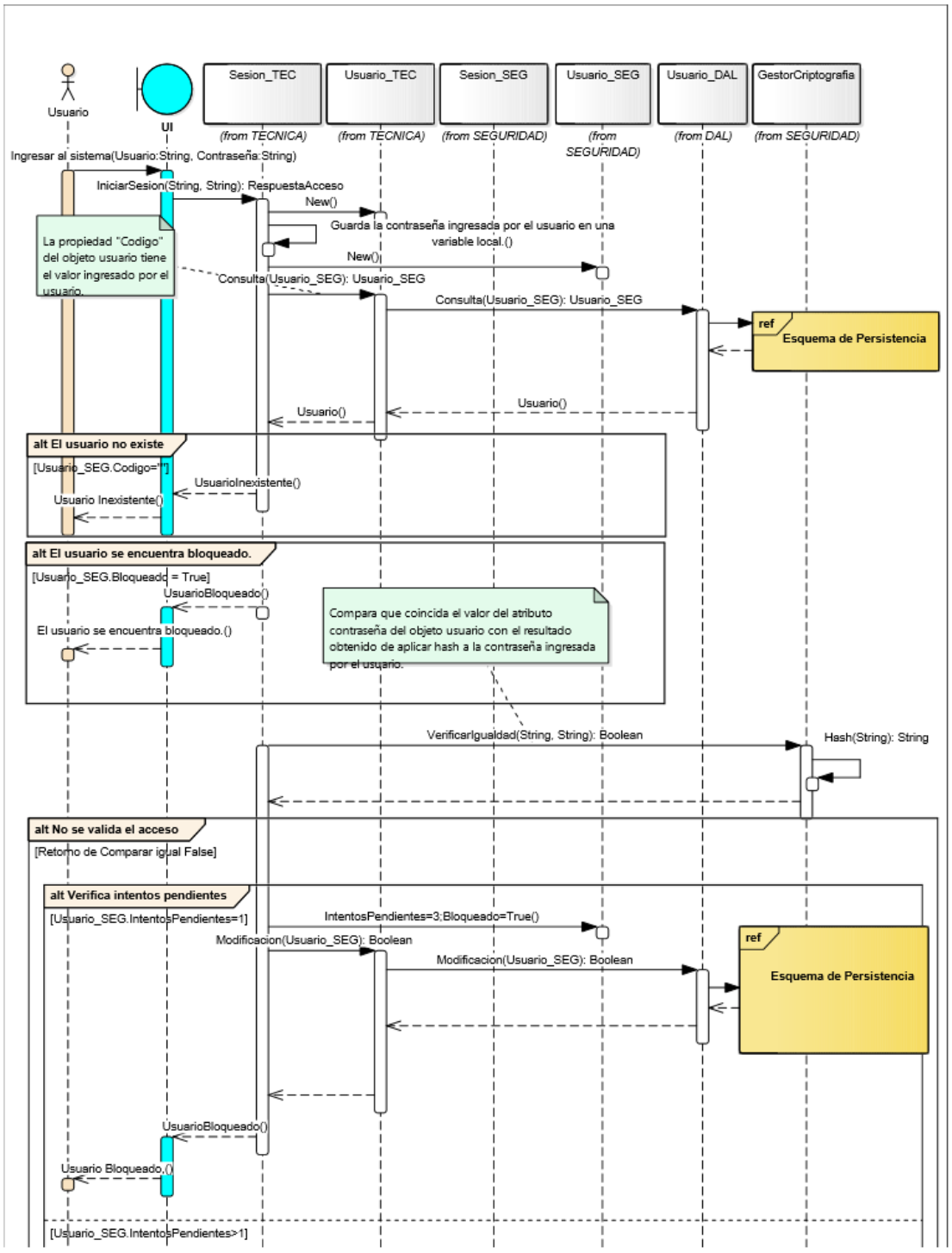
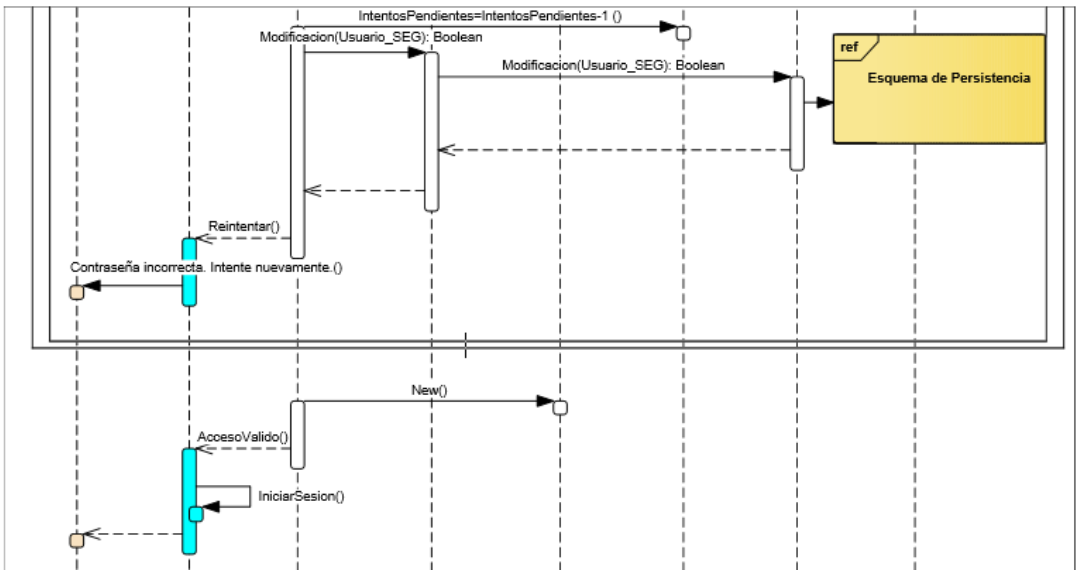


Diagrama de Secuencia

Login





4.3 Gestión de Múltiples Idiomas

Objetivo

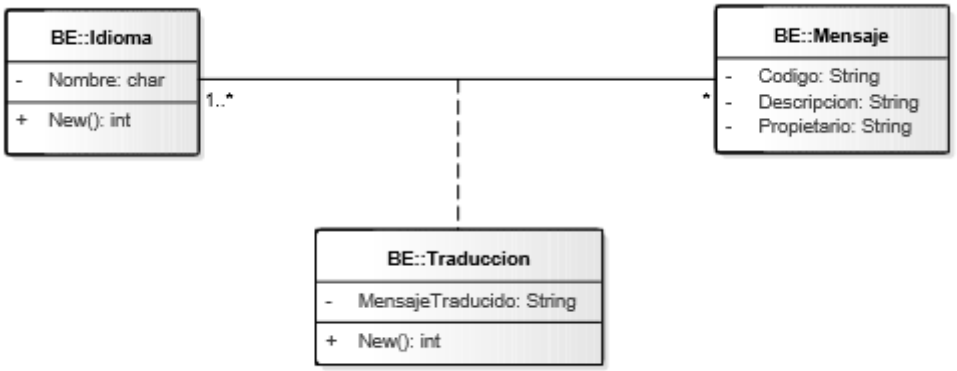
Debe permitir el cambio de idioma de todas las leyendas y títulos que se lean en las interfaces de usuario. El cambio debe ser dinámico. Este concepto implica que desde el sistema se puedan incorporar nuevos idiomas y las leyendas que estén afectadas al mismo.

Descripción detallada de cómo funciona

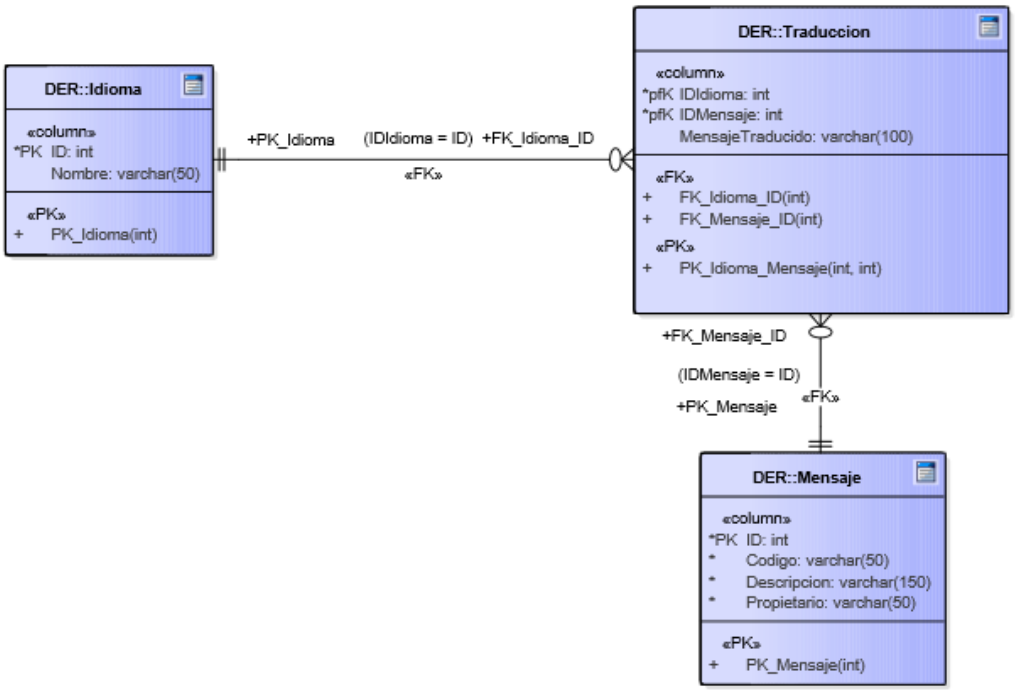
El usuario inicia la carga de un nuevo idioma. El sistema muestra la pantalla de carga. El usuario ingresa el nombre del nuevo idioma y confirma la carga. El sistema guarda el nuevo idioma y carga cada una de las leyendas con la traducción vacía.

Si ocurre un error en la conexión de la base de datos el sistema registra el error en bitácora.
El sistema registra en bitácora el alta del idioma. El sistema actualiza el combo de Idiomas. El usuario elige el nuevo idioma. El sistema Actualiza la grilla de leyendas. El usuario carga la traducción para cada una de las leyendas. El usuario habilito y guarda el nuevo idioma. El sistema verifica que no existan leyendas con su traducción en vacía.
Si existe alguna leyenda sin traducción el sistema informa que no es posible habilitar el idioma hasta que estén todas las traducciones.
El sistema guarda las traducciones cargadas dejando el idioma deshabilitado. El sistema guarda cada una de las traducciones para ese idioma. El sistema registra en bitácora el alta.
El sistema informa que se ha guardado el nuevo idioma con éxito.

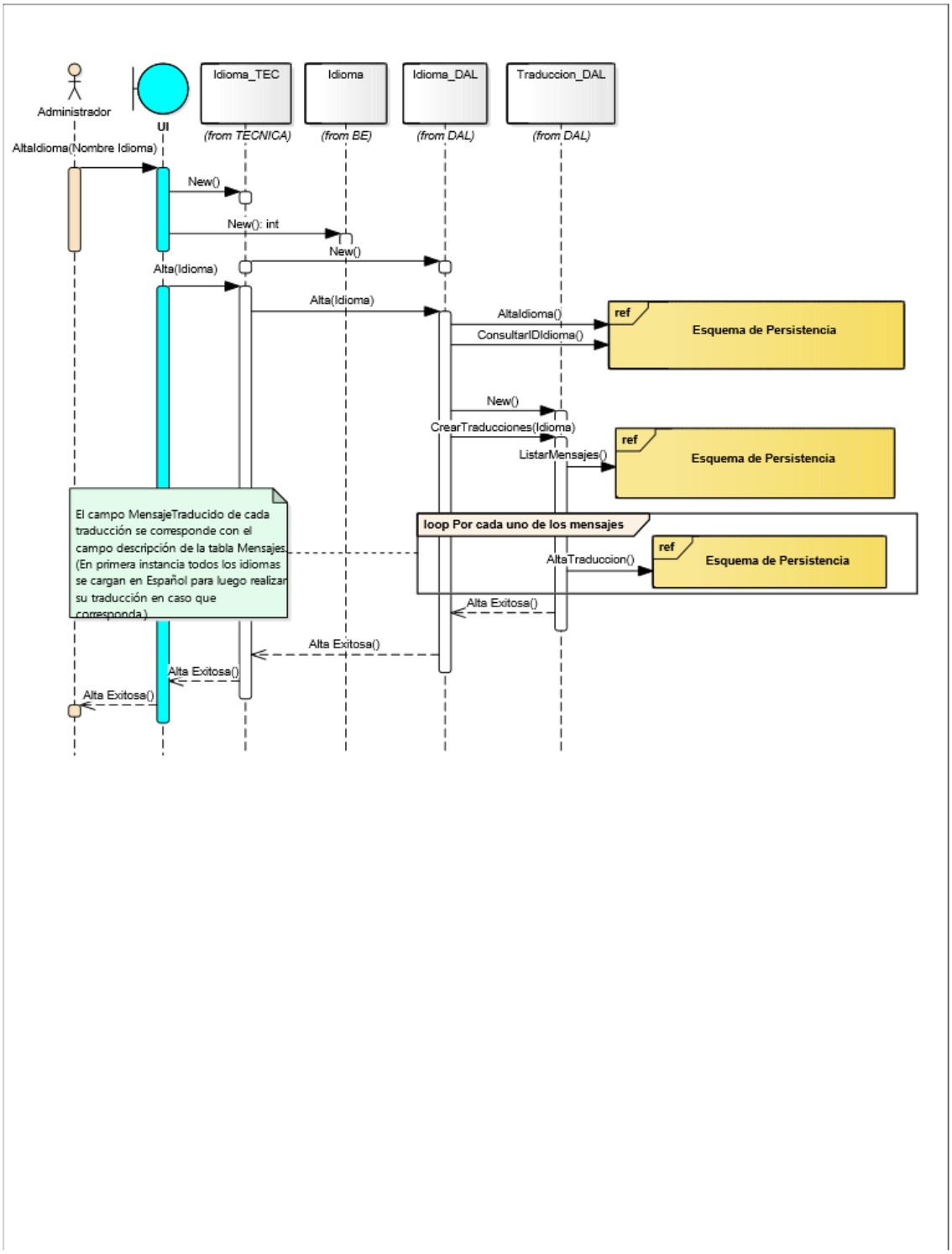
Diagrama de clases



DER



Secuencia


Administrador
UI
Idioma_TEC
(from TECNICA)
Idioma
(from BE)
Idioma_DAL
(from DAL)
Traduccion_DAL
(from DAL)
AltaIdioma(Nombre Idioma)
New()
New(): int
New()
Alta(Idioma)
Alta(Idioma)
AltaIdioma()
ConsultarIDIdioma()
ref
Esquema de Persistencia
New()
CrearTraducciones(Idioma)
ListarMensajes()
ref
Esquema de Persistencia
El campo MensajeTraducido de cada traducción se corresponde con el campo descripción de la tabla Mensajes. (En primera instancia todos los idiomas se cargan en Español para luego realizar su traducción en caso que corresponda.)
loop Por cada uno de los mensajes
AltaTraduccion()
ref
Esquema de Persistencia
Alta Exitosa()
Alta Exitosa()
Alta Exitosa()
Alta Exitosa()

4.4 Gestión de Bitácora

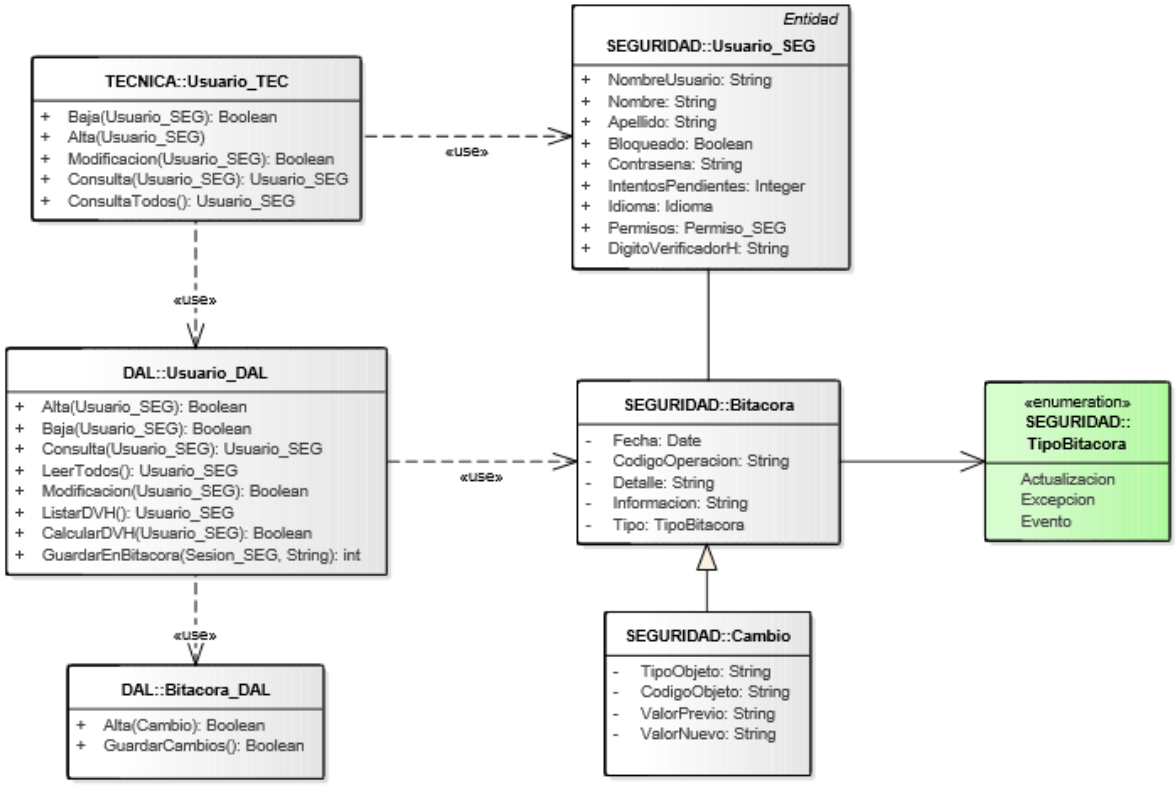
Objetivo

El sistema debe permitir que un usuario pueda consultar en la bitácora de auditoria todos los movimientos relevantes y errores. La consulta debe permitir filtrar por: usuario, fecha y tipo de evento.

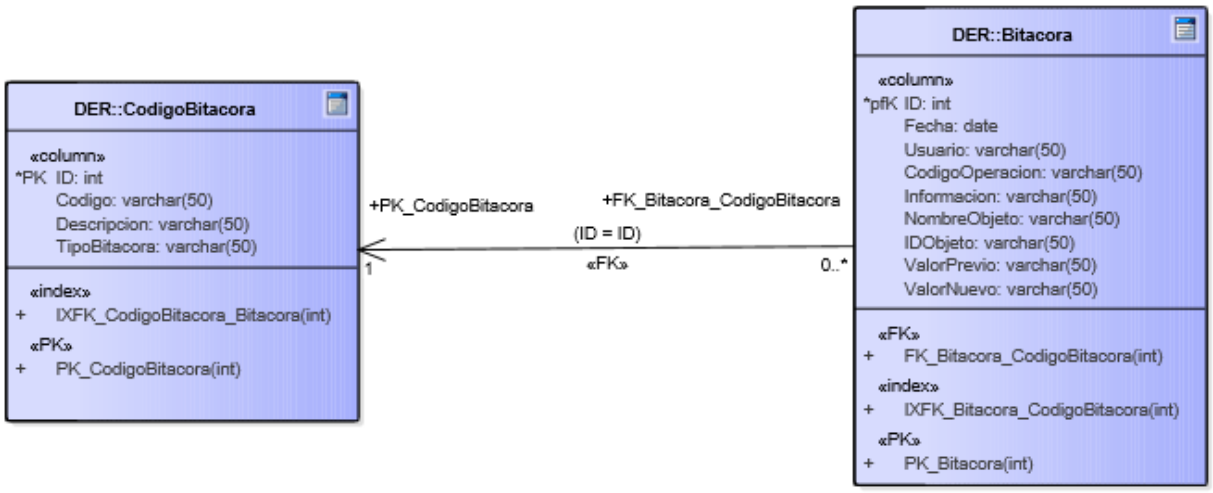
Descripción detallada de cómo funciona

El usuario solicita realizar la búsqueda en la bitácora. El sistema le muestra la ventana de búsqueda para que pueda realizar los filtros. El usuario solicita la búsqueda con los filtros correspondientes. El sistema registra el error en bitácora. El sistema devuelve el listado de registros según el filtro de búsqueda.

Diagrama de clases



DER


DER::CodigoBitacora
«column»
*PK ID: int
Codigo: varchar(50)
Descripcion: varchar(50)
TipoBitacora: varchar(50)
«index»
+ IXFK_CodigoBitacora_Bitacora(int)
«PK»
+ PK_CodigoBitacora(int)
+PK_CodigoBitacora
(ID = ID)
«FK»
1
+FK_Bitacora_CodigoBitacora
0..*
DER::Bitacora
«column»
*pfK ID: int
Fecha: date
Usuario: varchar(50)
CodigoOperacion: varchar(50)
Informacion: varchar(50)
NombreObjeto: varchar(50)
IDObjeto: varchar(50)
ValorPrevio: varchar(50)
ValorNuevo: varchar(50)
«FK»
+ FK_Bitacora_CodigoBitacora(int)
«index»
+ IXFK_Bitacora_CodigoBitacora(int)
«PK»
+ PK_Bitacora(int)

Evento-Excepción

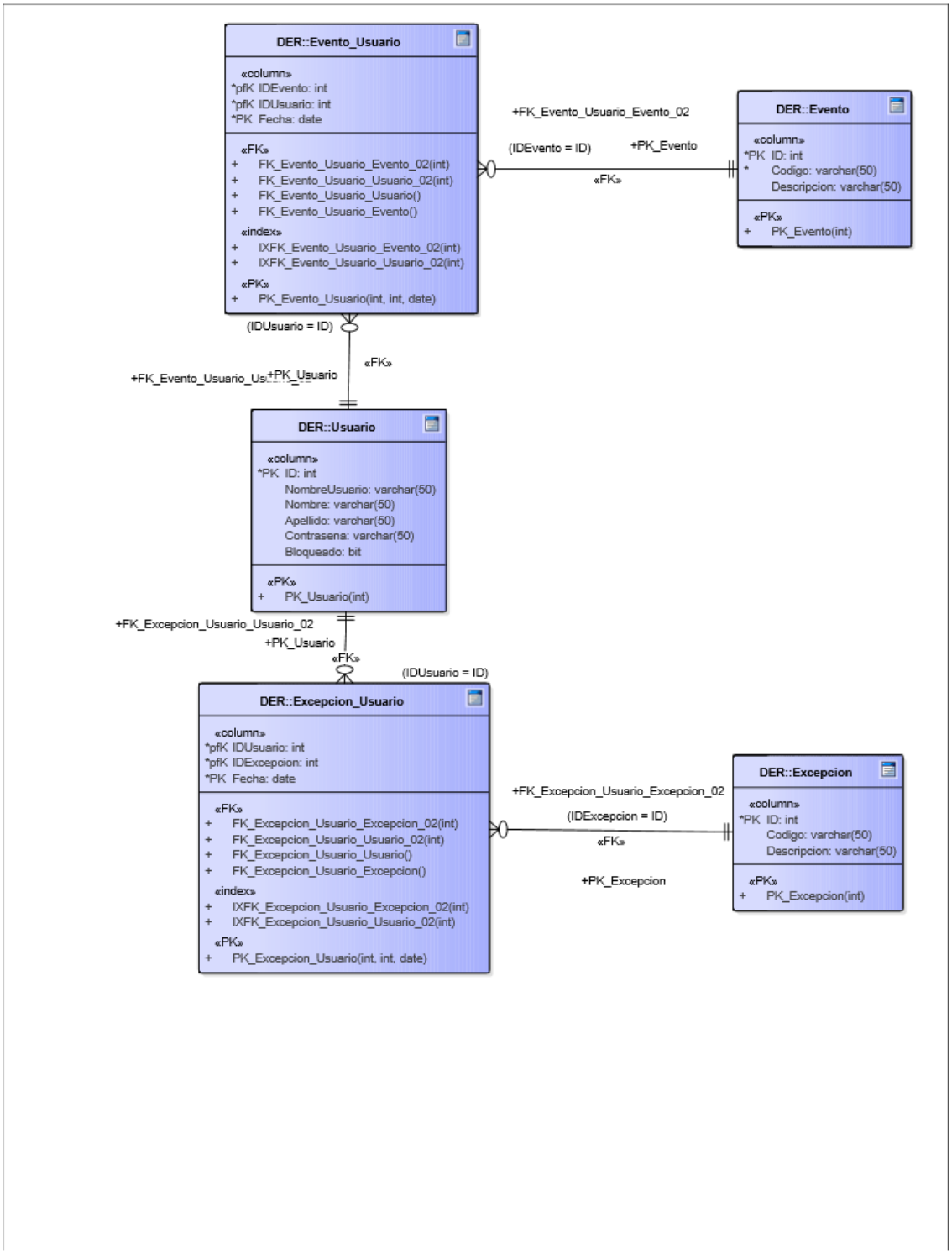
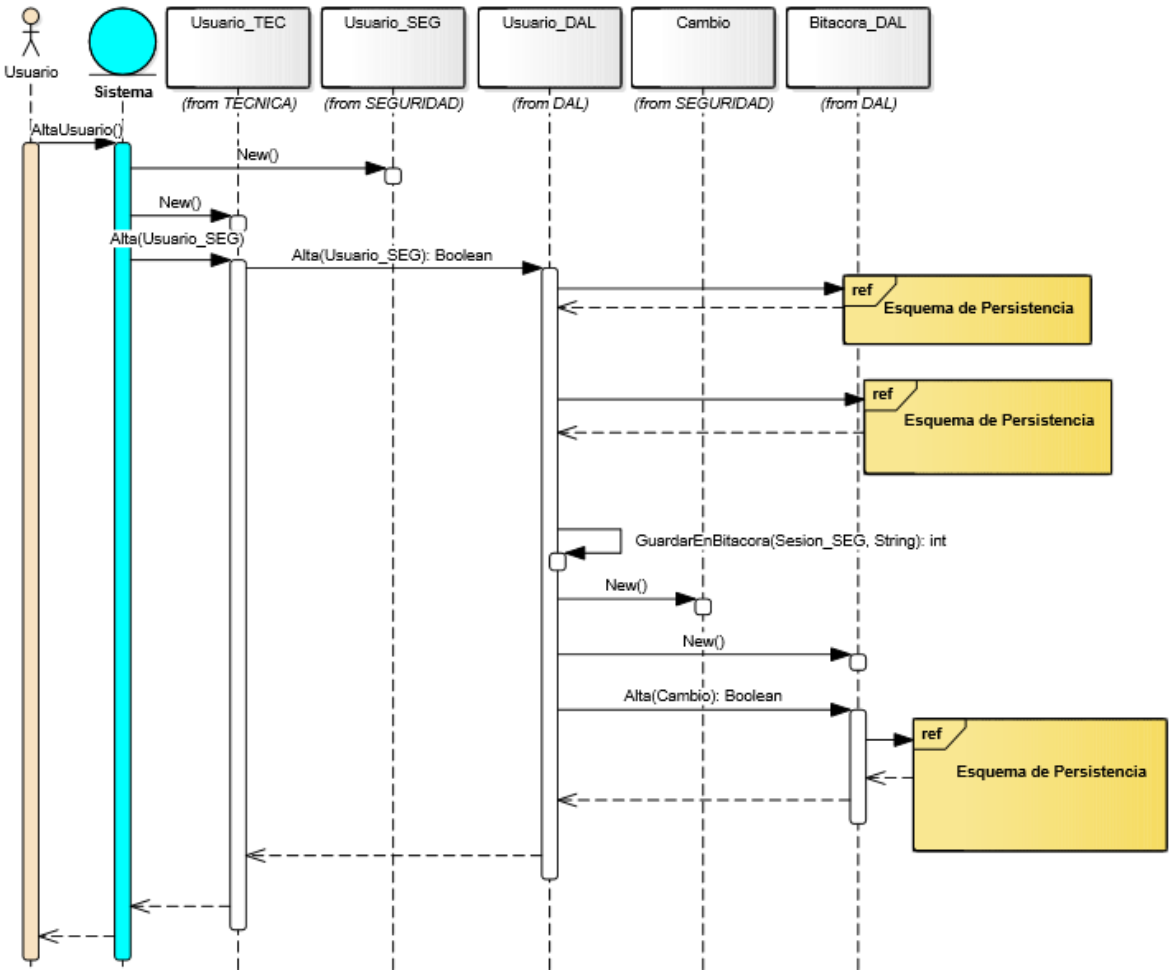


Diagrama de Secuencia



4.5 Gestión de Backup

Objetivo

Esta gestión se utiliza para administrar las copias de seguridad. Esto implica gestionar el catálogo de backups así como los archivos físicos que contienen la información resguardada.
Realizar la restauración de la base de datos del sistema.

Descripción detallada de cómo funciona

El usuario selecciona iniciar la operación de realizar un BackUp desde la pantalla principal. El sistema muestra la pantalla de BackUp. El usuario ingresa los datos necesarios para realizar el BackUp (directorio y nombre del archivo a generar) y confirma el inicio de la operación. El directorio o el nombre del archivo no están definidos. El sistema notifica al usuario que debe completar ambos campos. El sistema ejecuta el script de generación del BackUp de la base de datos y guarda la

copia de seguridad en el directorio establecido. Error en la conexión de base de datos. El sistema registra el error en bitácora. El sistema registra en bitácora el resguardo.
El usuario selecciona iniciar la operación de realizar un Restore desde la pantalla principal. El sistema muestra la pantalla de Restore. El usuario ingresa los datos necesarios para realizar el Restore (directorio y nombre del archivo) y confirma el inicio de la operación. El directorio para restaurar la base de datos no está definido. El sistema notifica al usuario que el directorio no está definido.
El sistema ejecuta el script de Restore de la base de datos. Si se produce un Error en la conexión de la base de datos el sistema registra el error en bitácora. El sistema registra en bitácora el resguardo.

Diagrama de clases

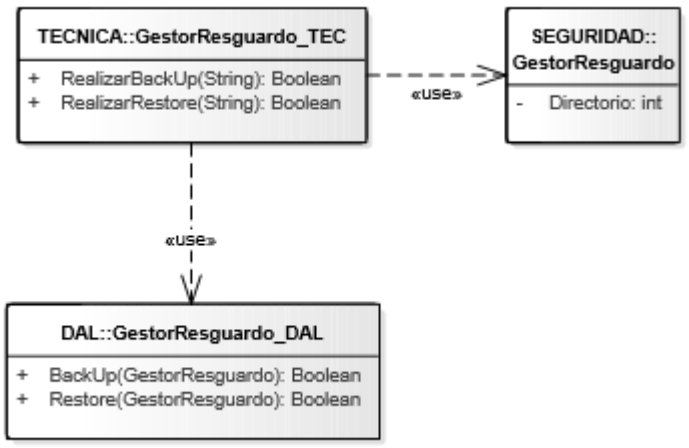
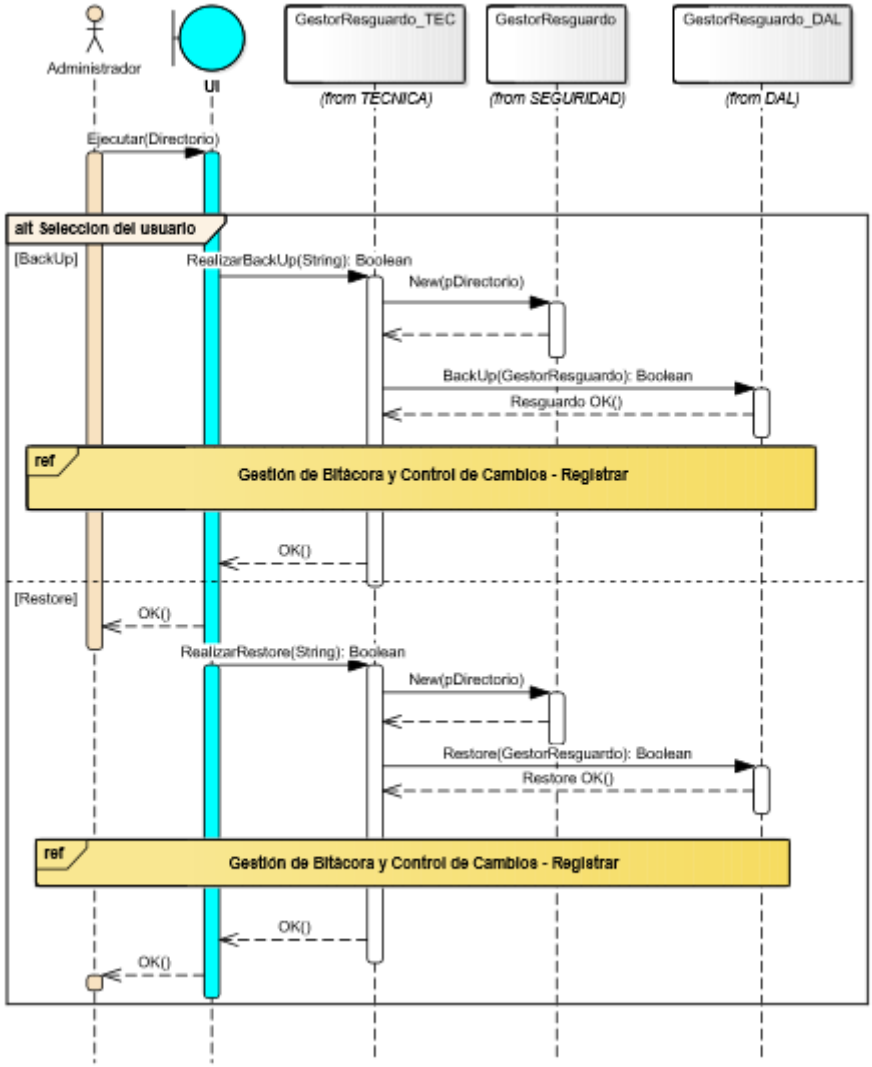


Diagrama de Secuencia



4.6 Gestión de Dígitos Verificadores

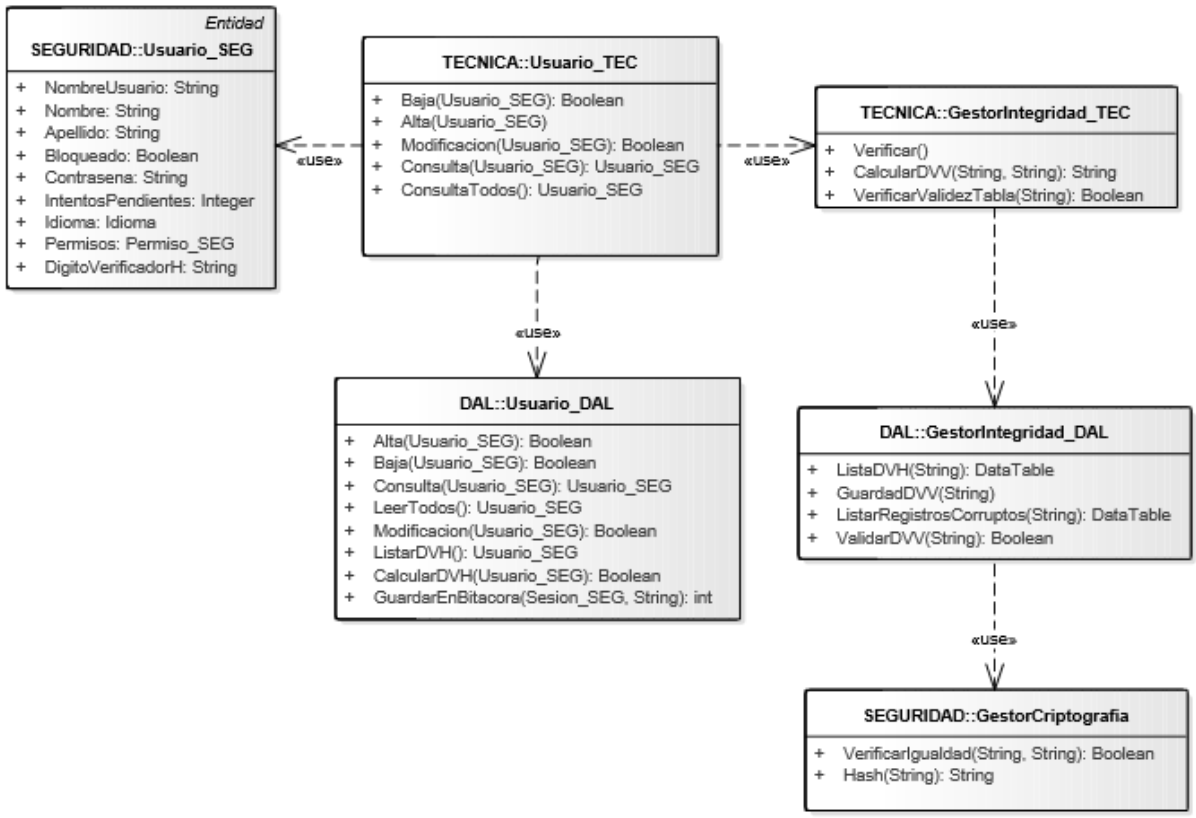
Objetivo

La función de los dígitos verificadores es la de permitir comprobar la integridad de los datos almacenados en la base de datos. Se desea poder detectar dos cosas. La primera es si se han agregado o quitado datos de la base de datos por fuera del sistema y la segunda es si se han intercambiado datos de posición o modificado.

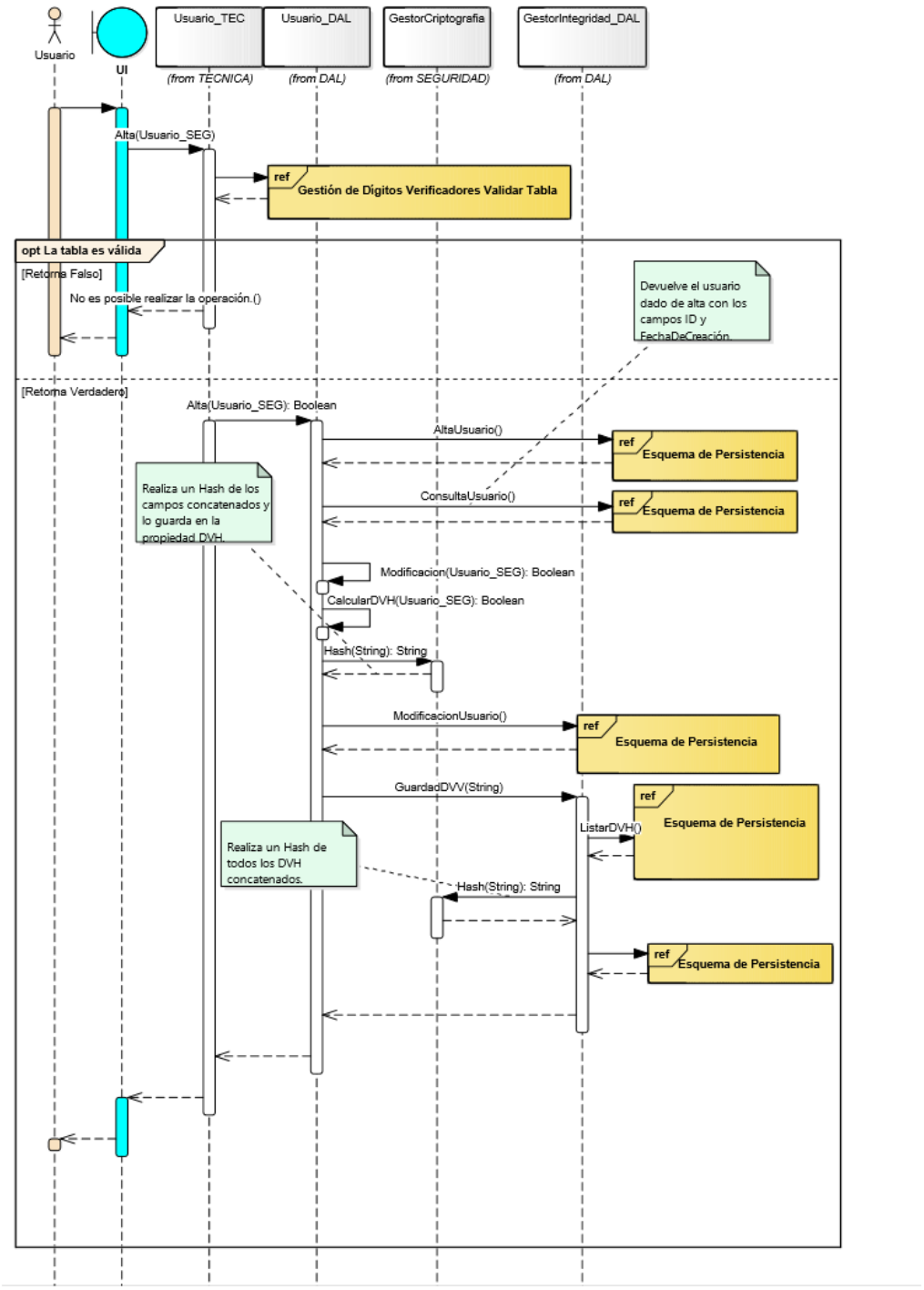
Descripción detallada de cómo funciona

Para cada una de las tablas críticas. El sistema busca en la base de datos todos los registros de la tabla. El sistema registra el error en bitácora.
Para cada uno de los registros de la tabla. El sistema calcula el DVV. El sistema compara el DVV de la tabla con el cálculo anterior. No coinciden los datos.
El sistema registra en bitácora el nombre de la tabla y los datos del registro con errores.
El sistema guarda los datos del registro en una tabla aparte en memoria. El sistema calcula el DVH de la tabla. El sistema compara el DVH de la tabla con el cálculo anterior. No coinciden los datos. El sistema registra en bitácora el nombre de la tabla y los datos del registro con errores.
El sistema informa al usuario que no coinciden los datos de la tabla. El sistema informa la integridad de la base de datos. La base de datos ha sido corrompida. Se le informa al usuario cuales fueron los registros con errores.

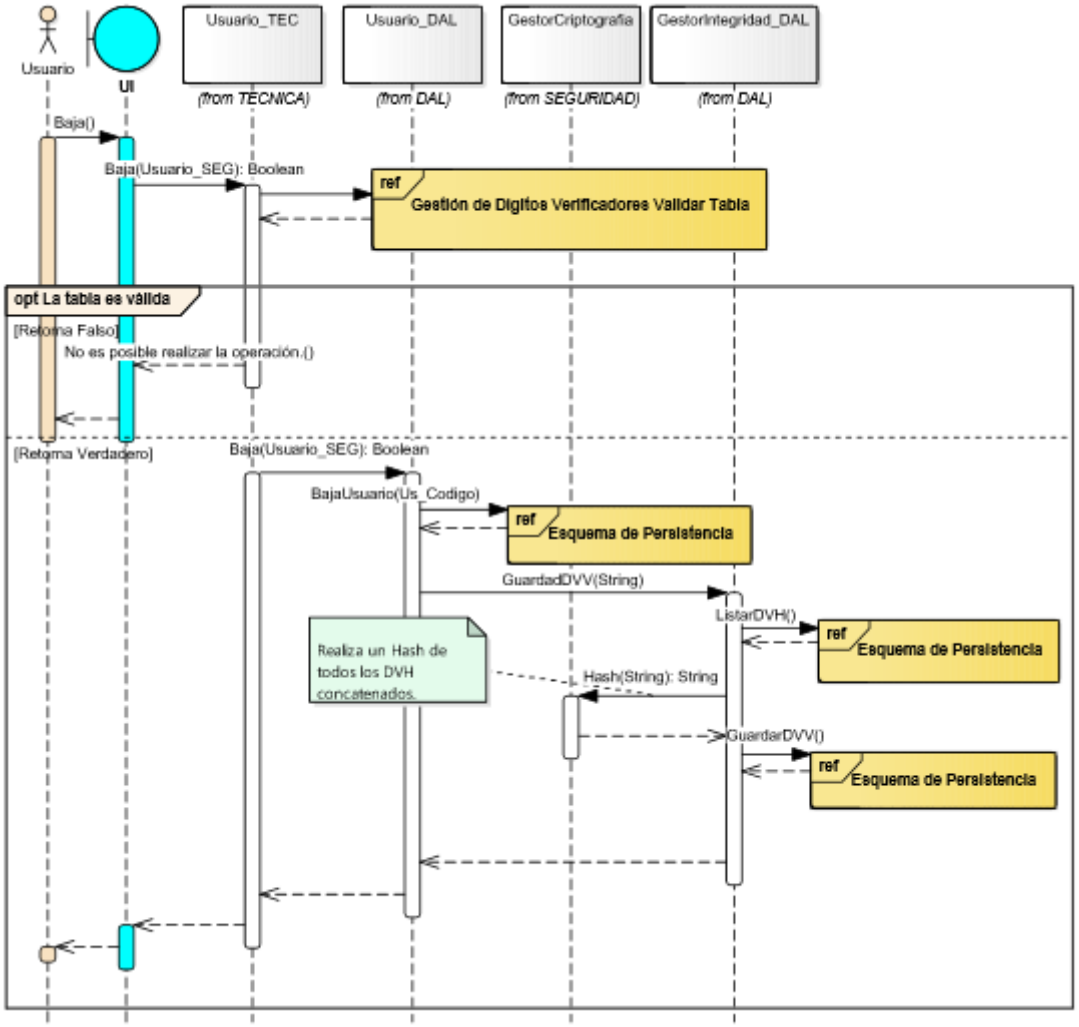
Diagrama de clases



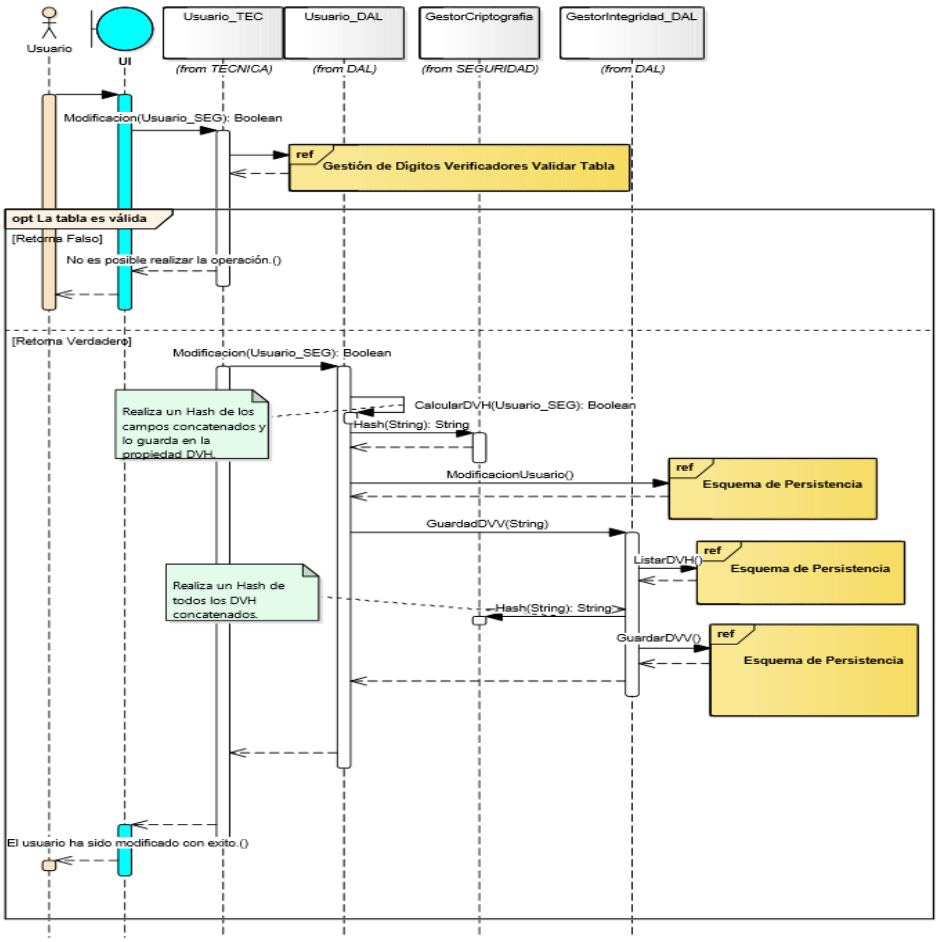
Alta Usuario



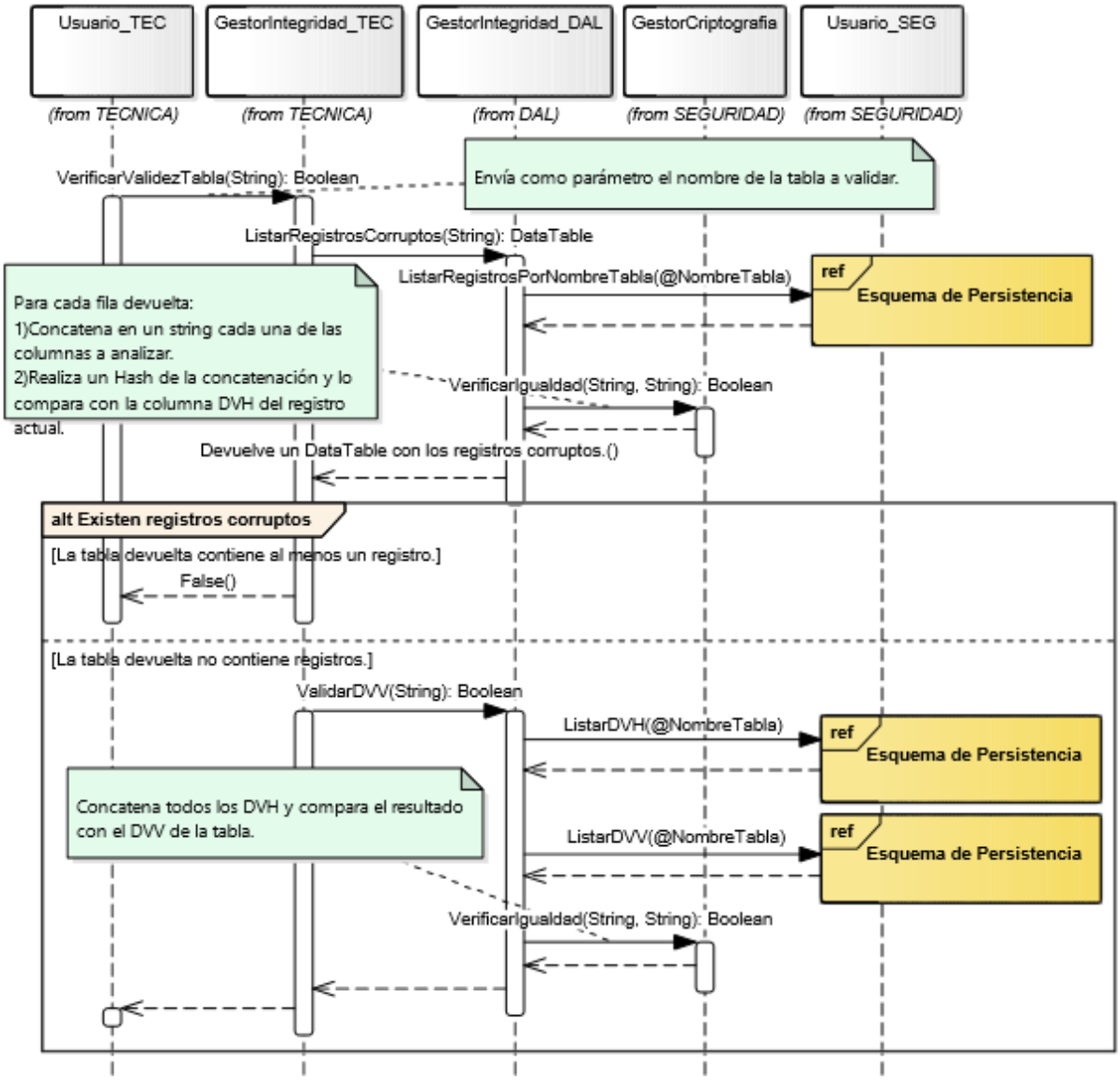
Baja Usuario



Modificacion Usuario



Verificar Validez



4.7 Gestión de Encriptado

Objetivo

La gestión de encriptado es la responsable implementar los algoritmos de encriptación para proteger los datos sensibles del sistema.

Descripción detallada de cómo funciona

Existirá una clase en la capa de seguridad a la cual se le podrán solicitar dos servicios.

- **Comparar.** Se le enviarán dos cadenas de caracteres. Si al aplicar el algoritmo de encriptación a la primer cadena el resultante es una cadena igual a la segunda cadena enviada como parámetro, devolverá True (caso contrario devolverá False).
- **Hash.** Será posible enviarle una cadena de caracteres para que el objeto pueda aplicarle el algoritmo de encriptación SHA1 y devolver la cadena resultante de la encriptación.

4.8 Esquema de persistencia

Objetivo

Insertar, Actualizar, Borrar y Seleccionar registros de la Base de Datos.

Descripción detallada de cómo funciona

El esquema de persistencia correrá en un Microsoft SQL Server 2012 como motor de Base de Datos.
Implementaremos una interfaz denominada IABM donde las operaciones comunes a todas las clases podrán ser implementadas de esta.

Diagrama de clases

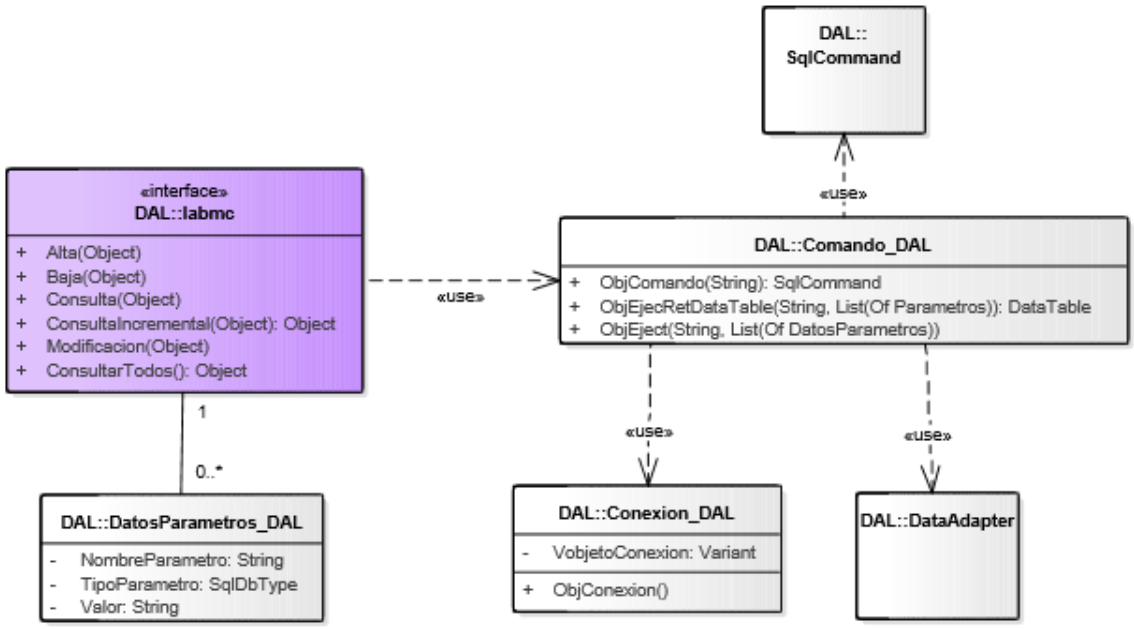
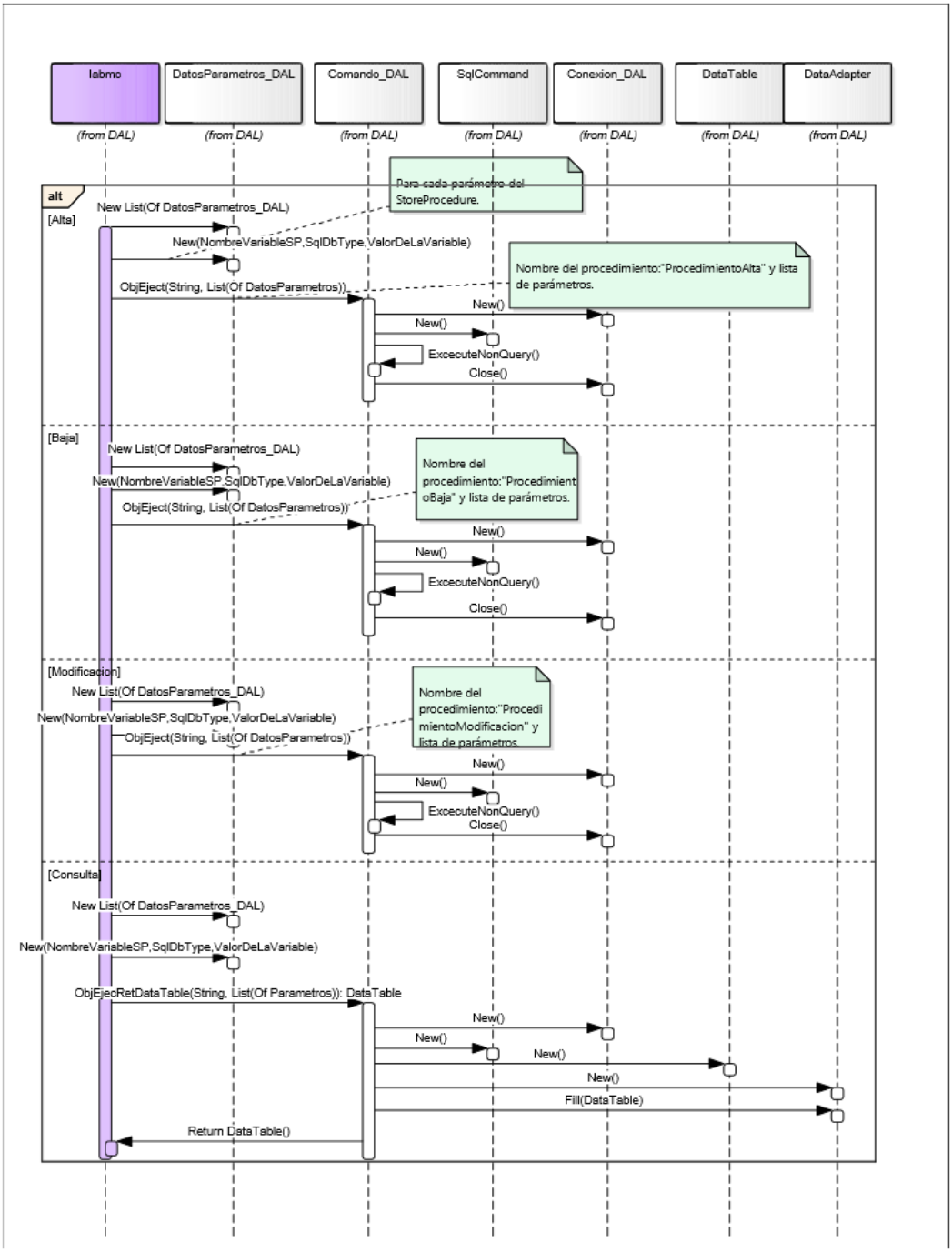
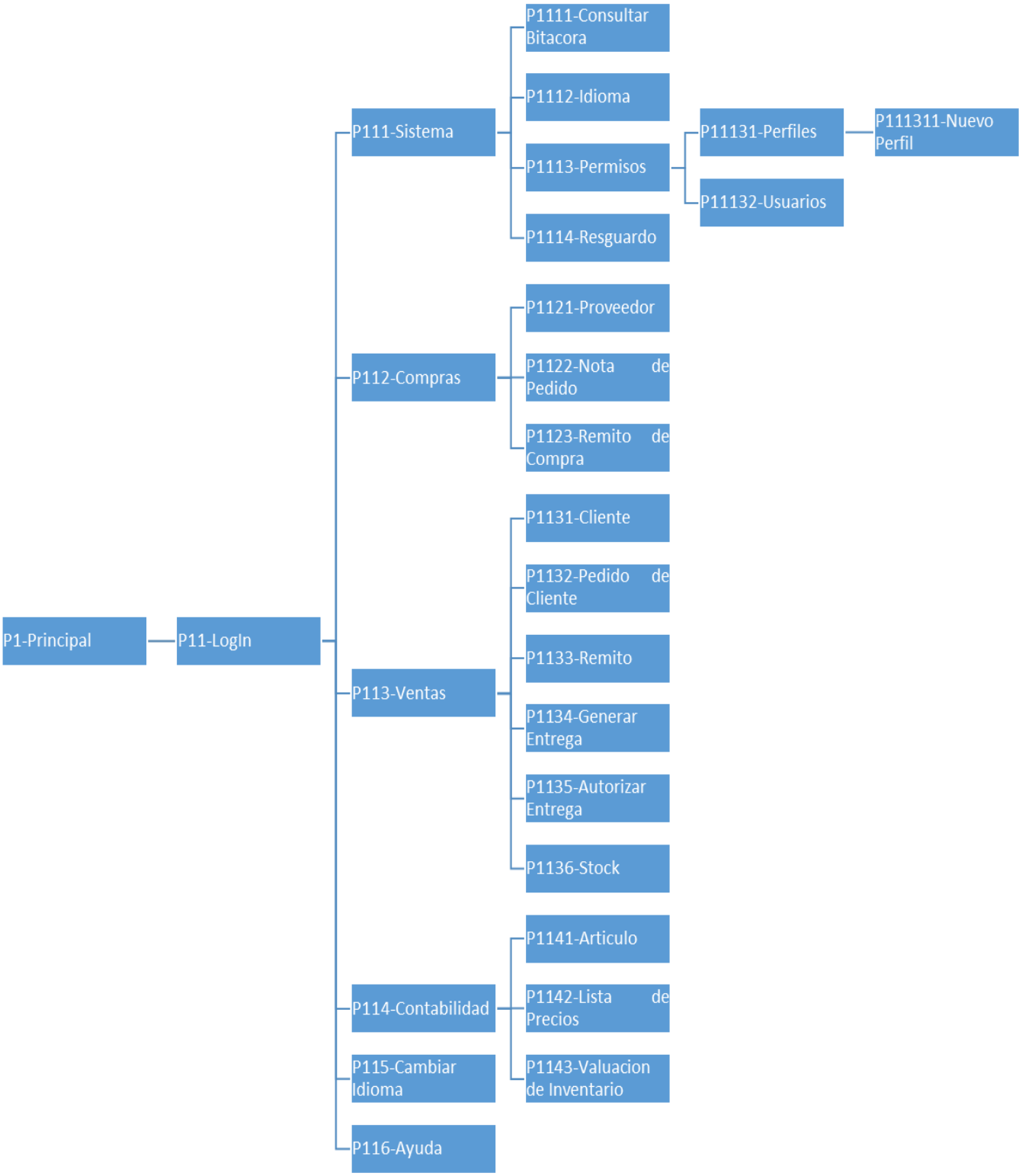


Diagrama de secuencia

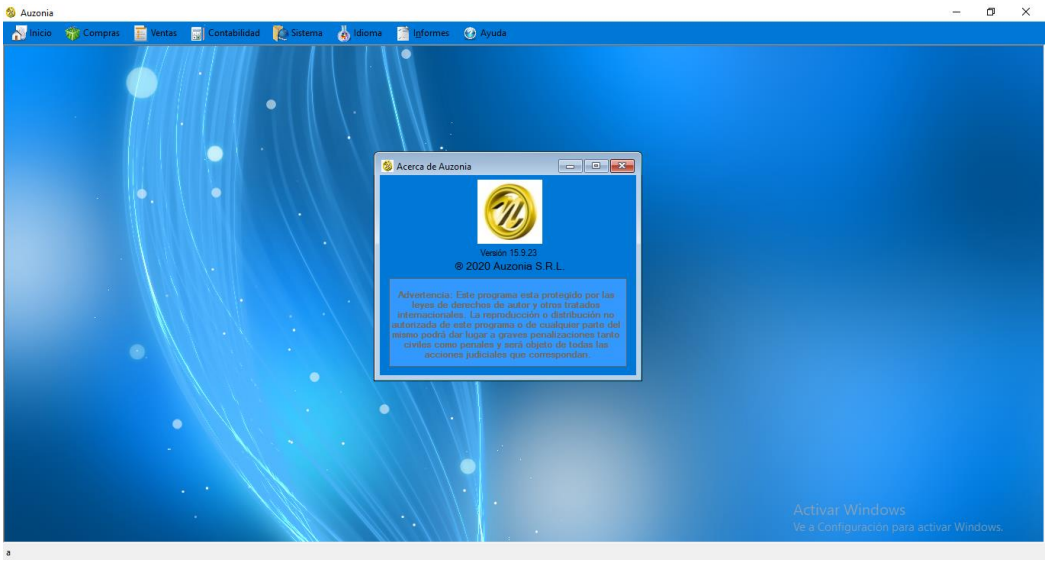


5. Mapa de Navegación

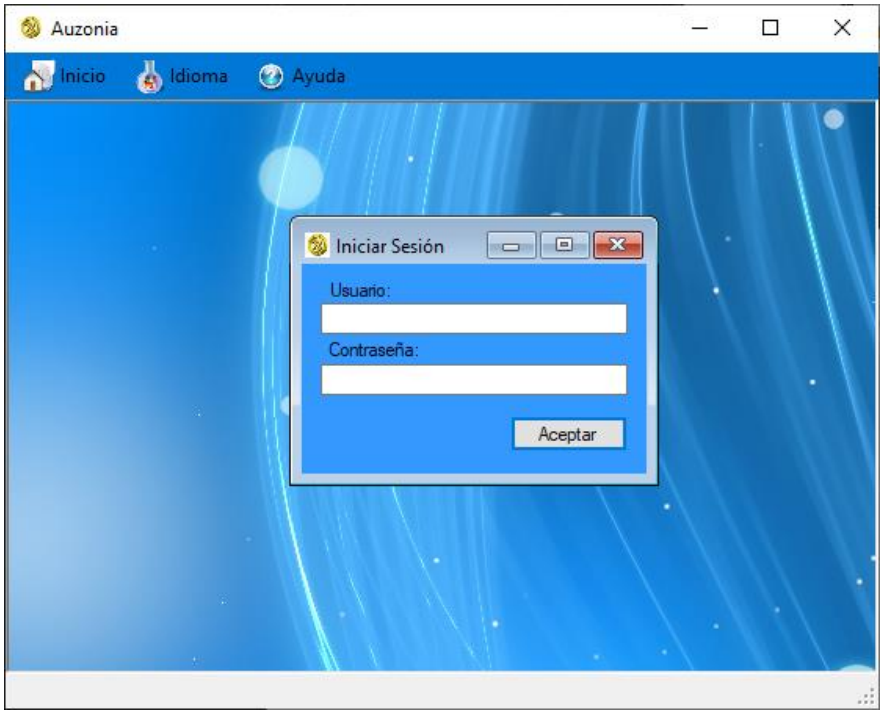


6. Interfaces de Usuario

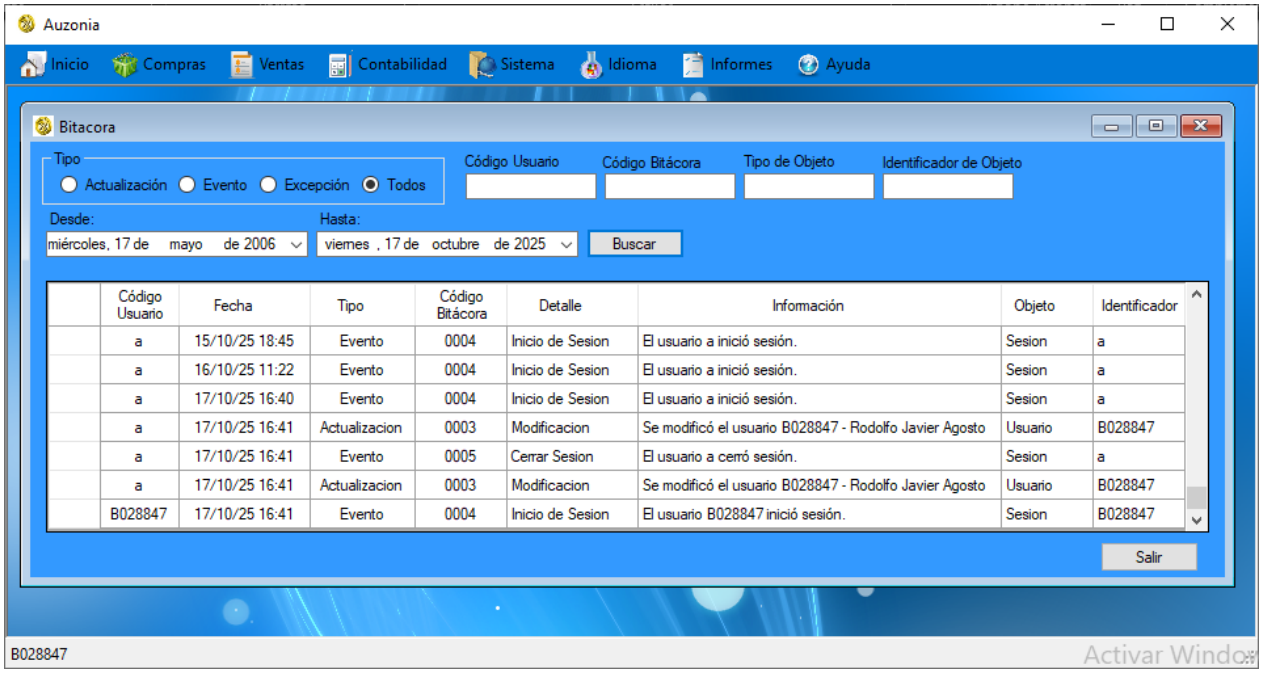
6.1 P1-Principal



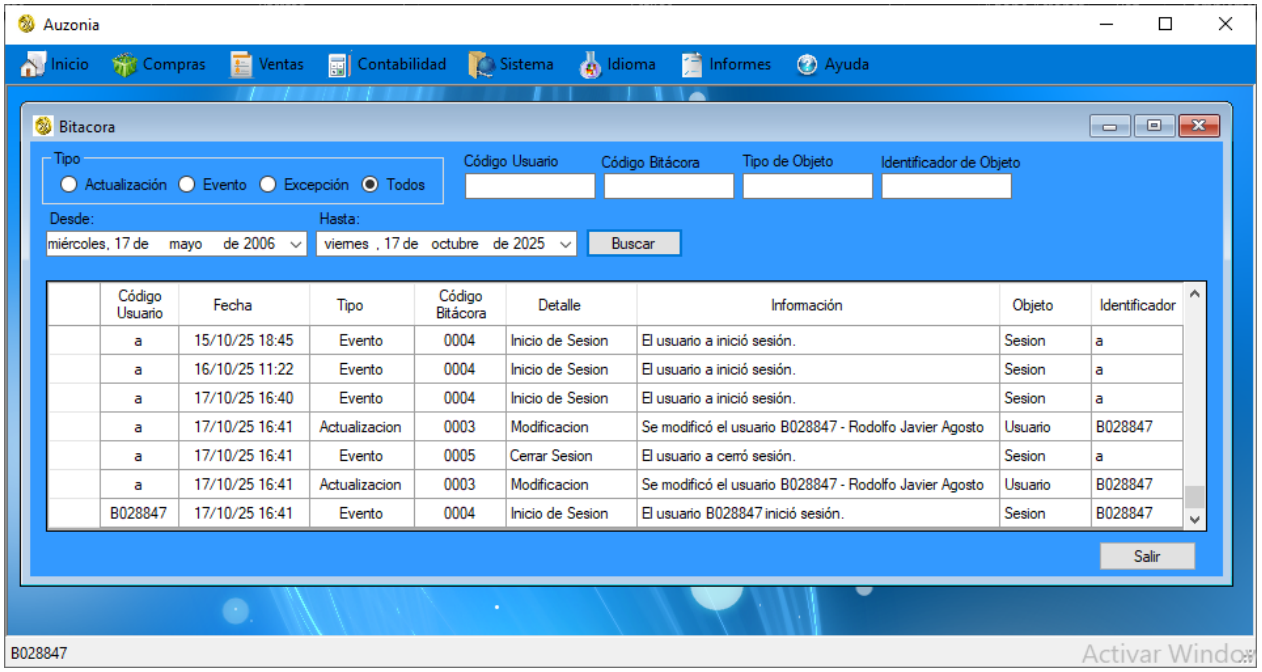
6.2 P11-LogIn



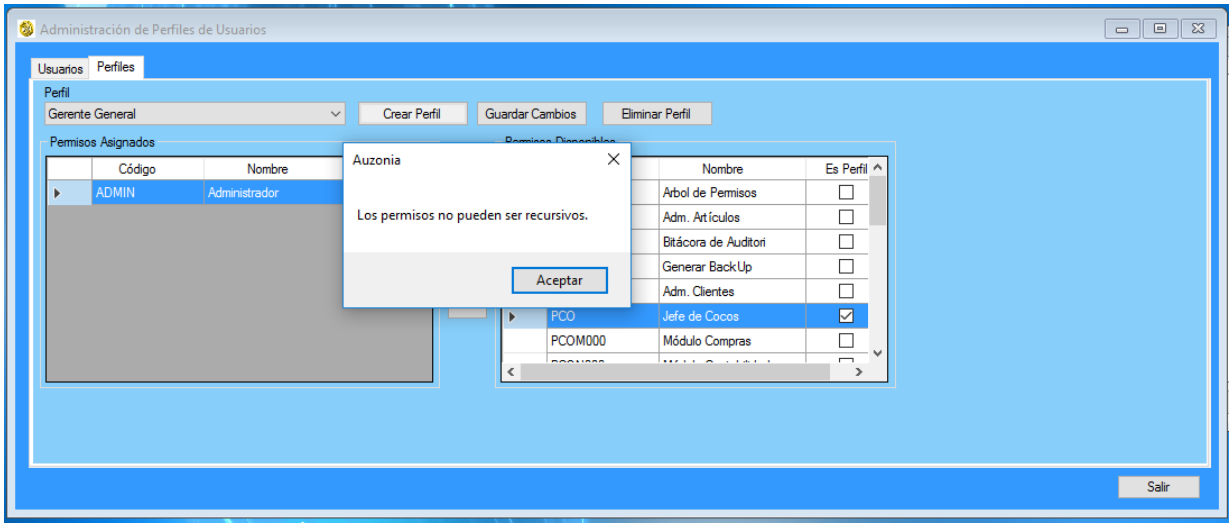
6.3 P1111-Consultar Bitácora



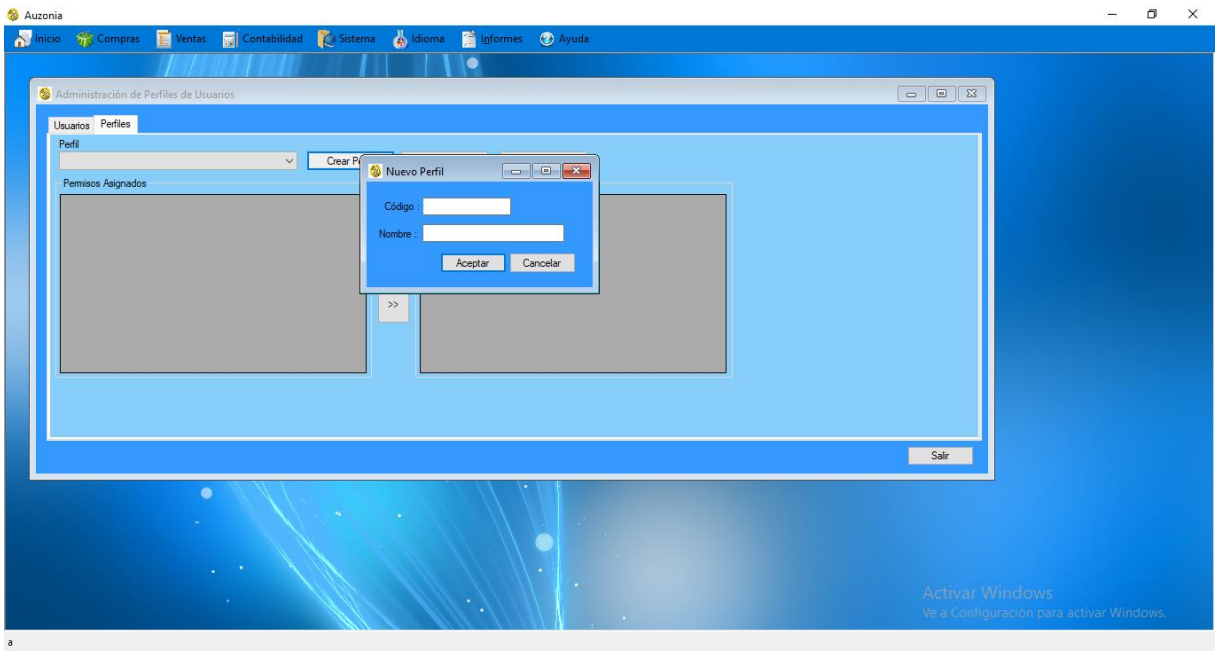
6.4 P1112-Idioma



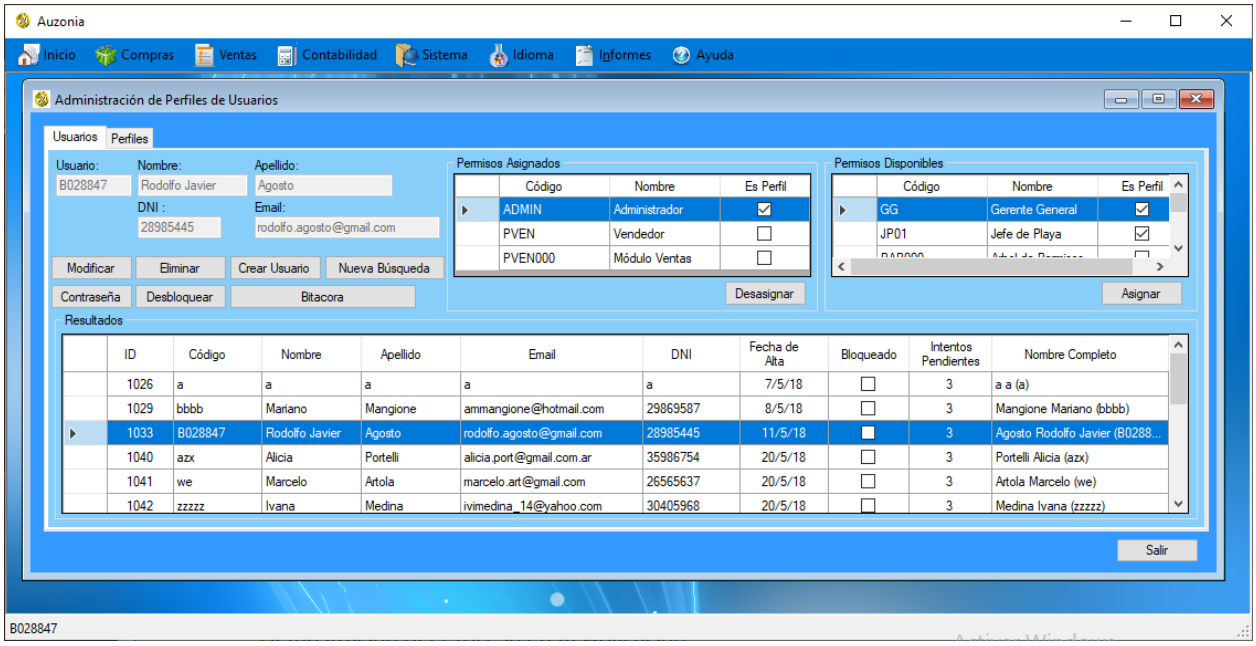
6.5 P11131-Perfiles



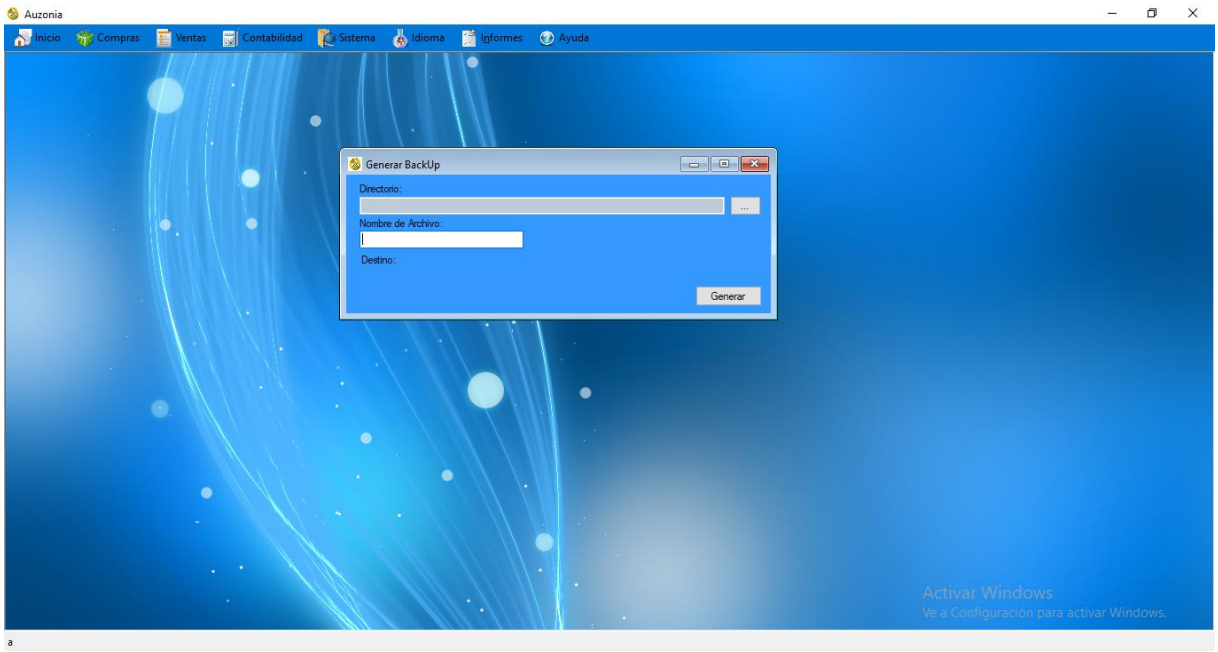
6.6 P111311-Nuevo Perfil



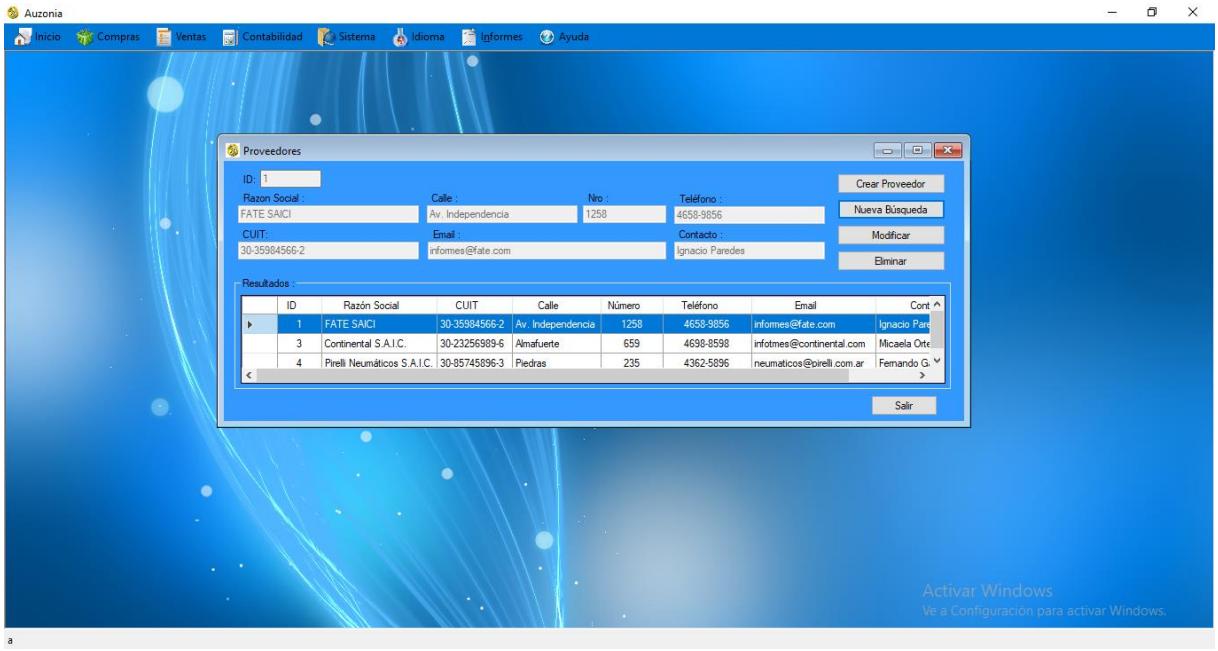
6.7 P11132-Usuarios



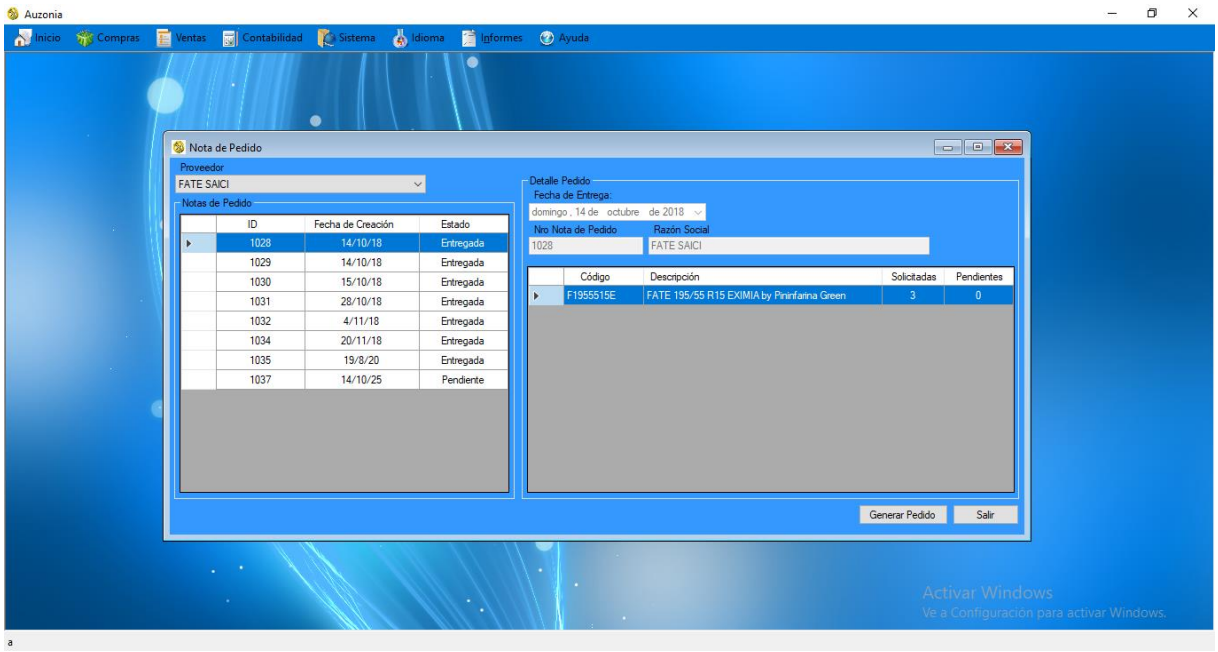
6.8 P1114-Generar BackUp



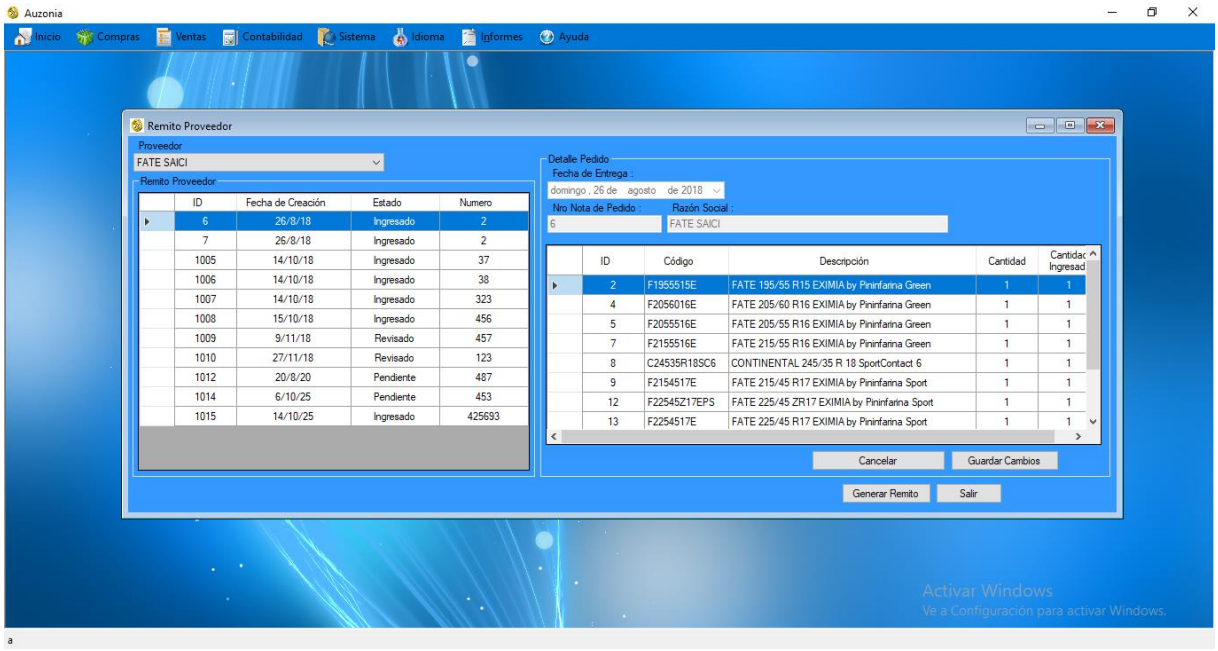
6.9 P1121-Proveedores



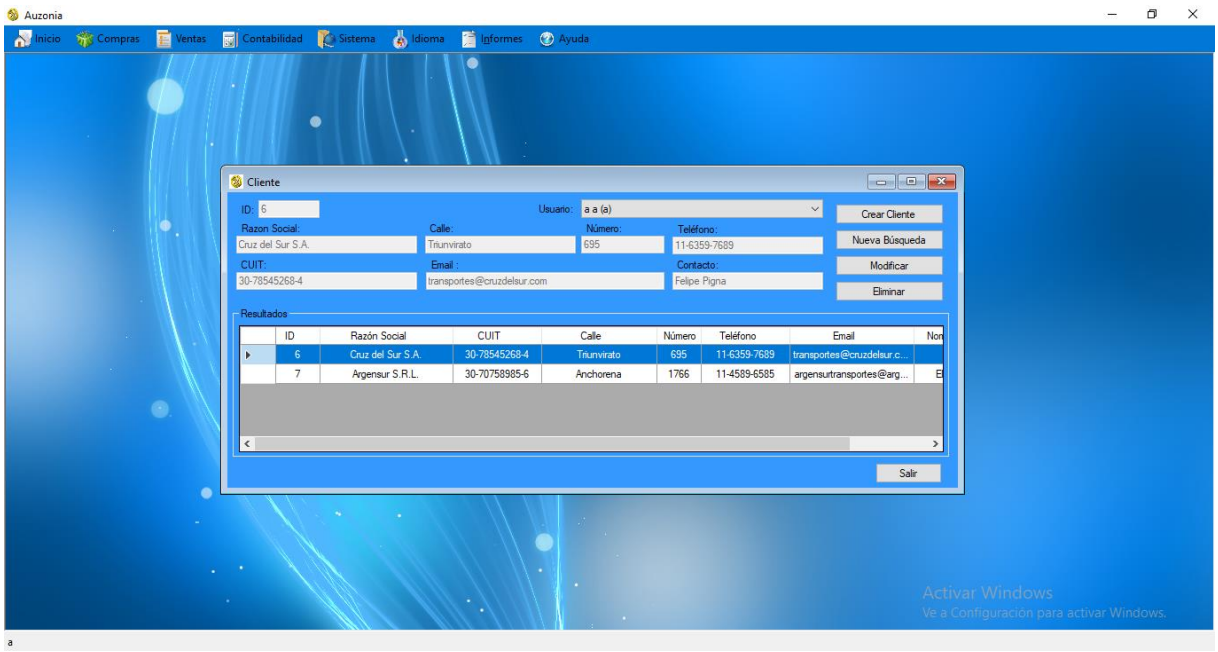
6.10 P1122-Nota de Pedido



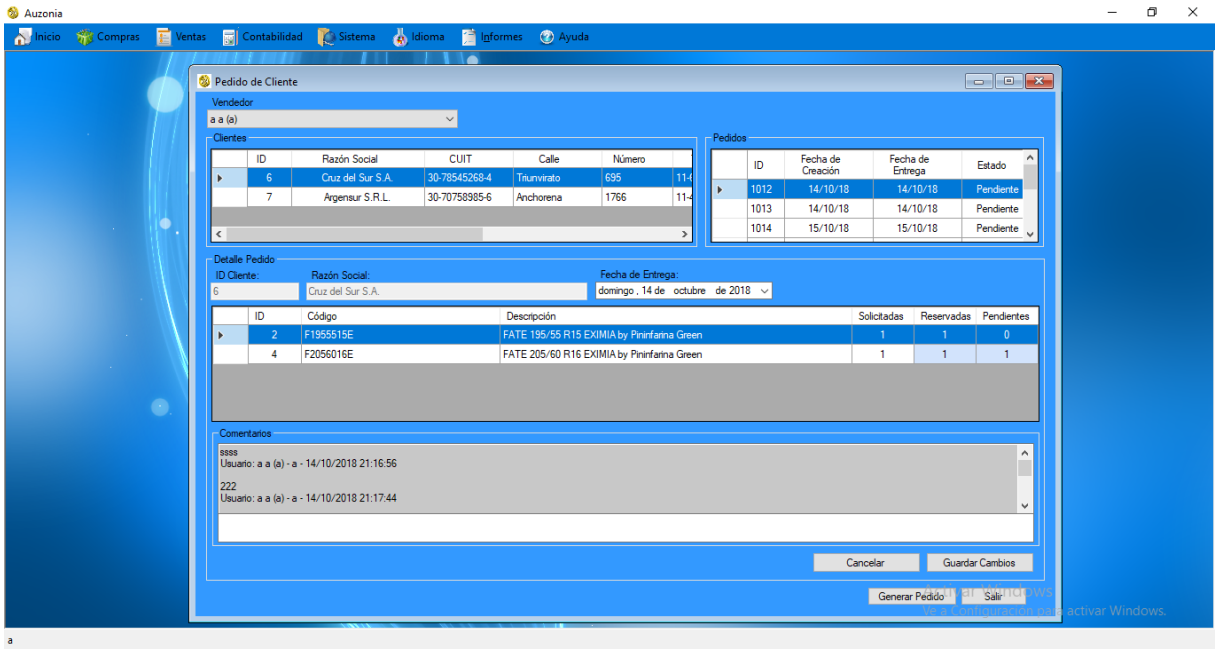
6.11 **P1123-Remito de Compra**



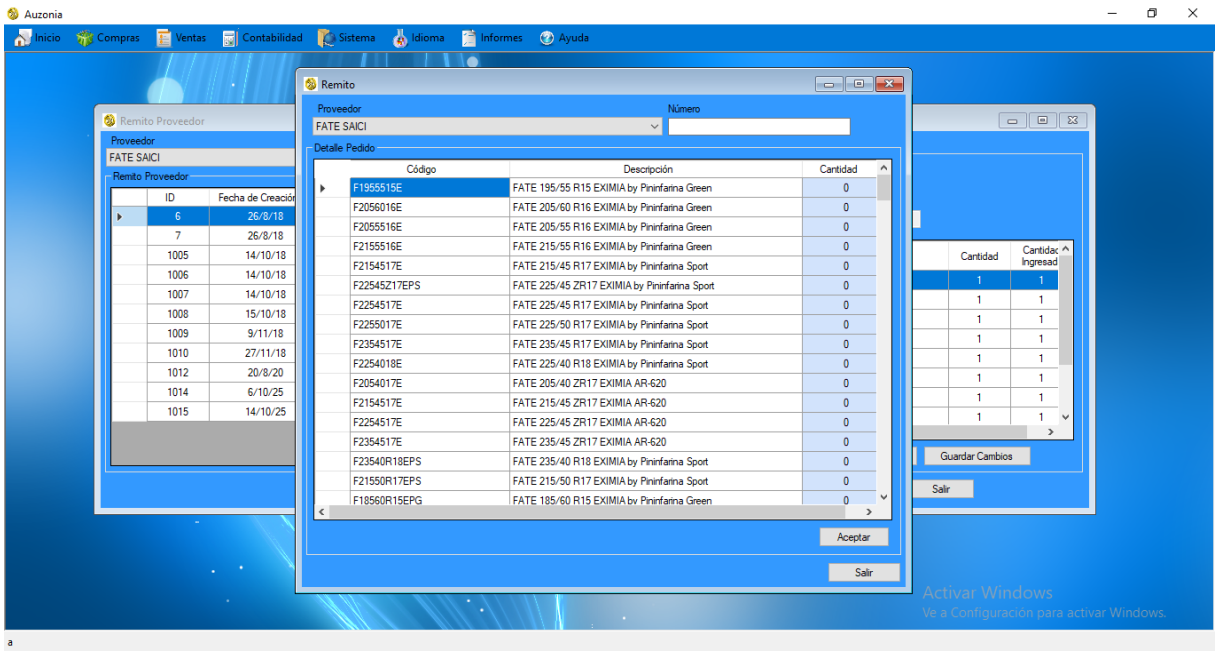
6.12 **P1131-Cliente**



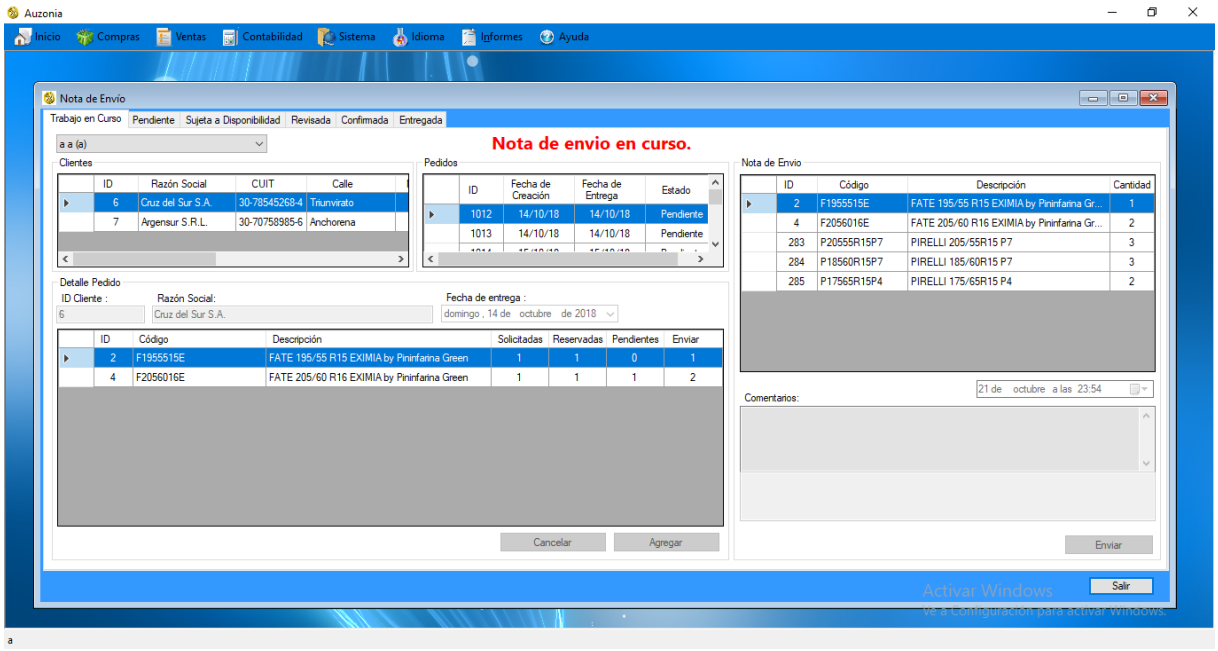
6.13 P1132-Pedido de Cliente



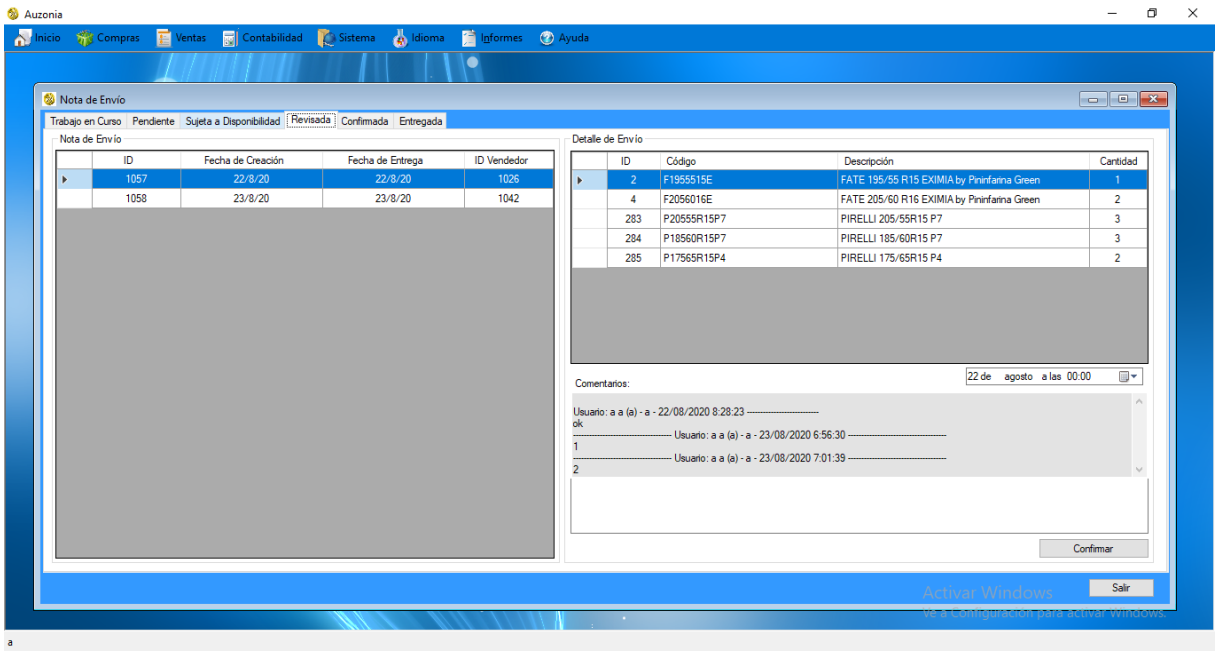
6.14 P1133-Remito



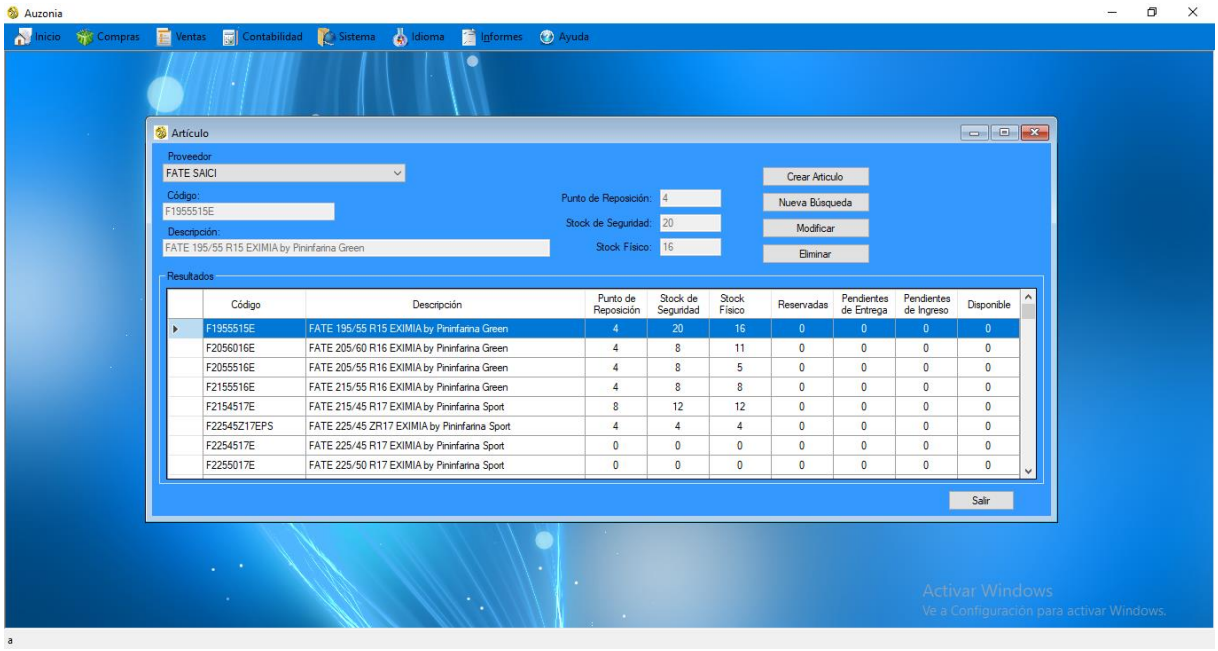
6.15 P1134-Generar Entrega



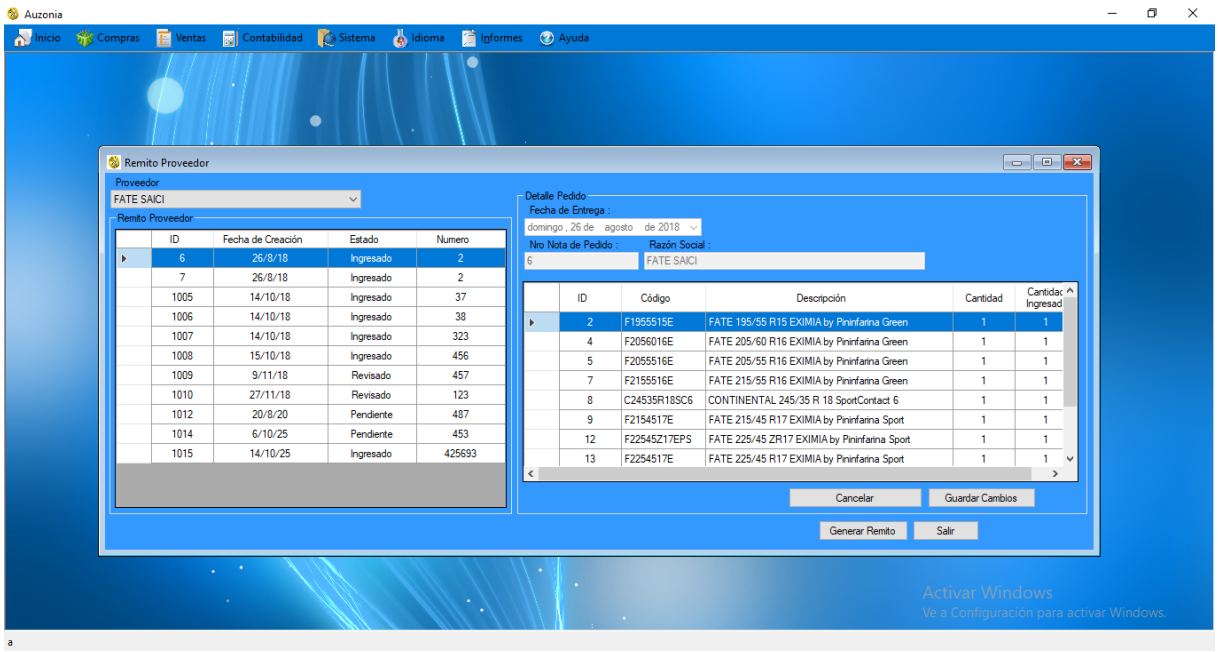
6.16 P1135-Autorizar Entrega



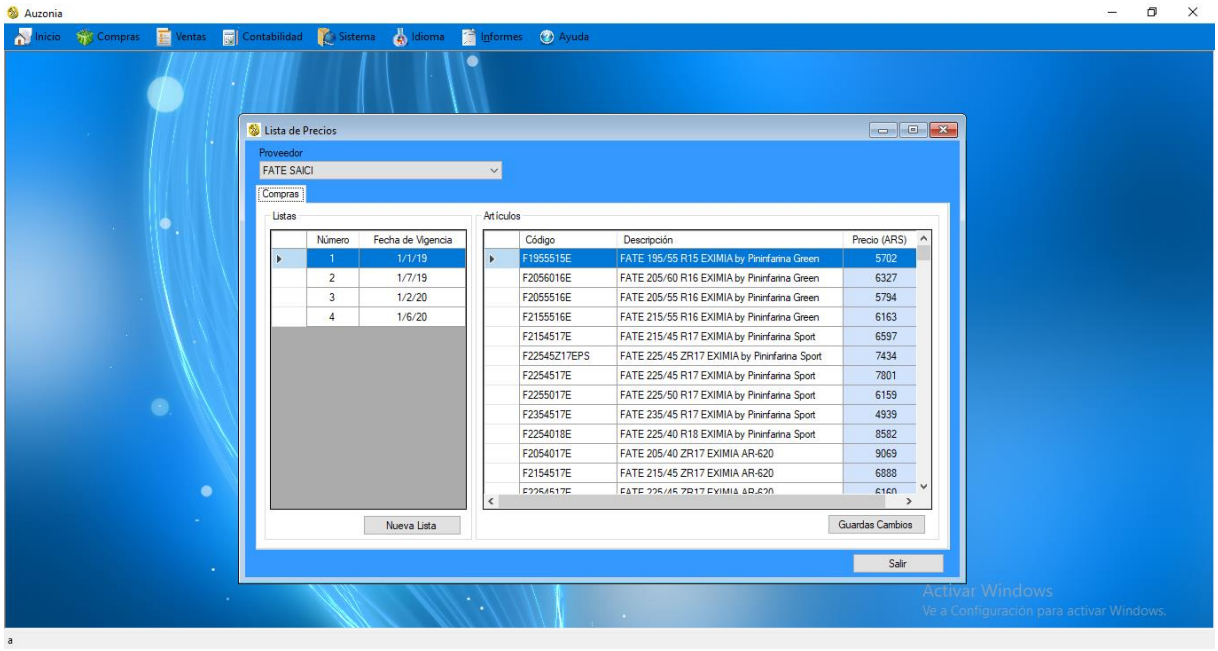
6.17 **P1136-Stock**



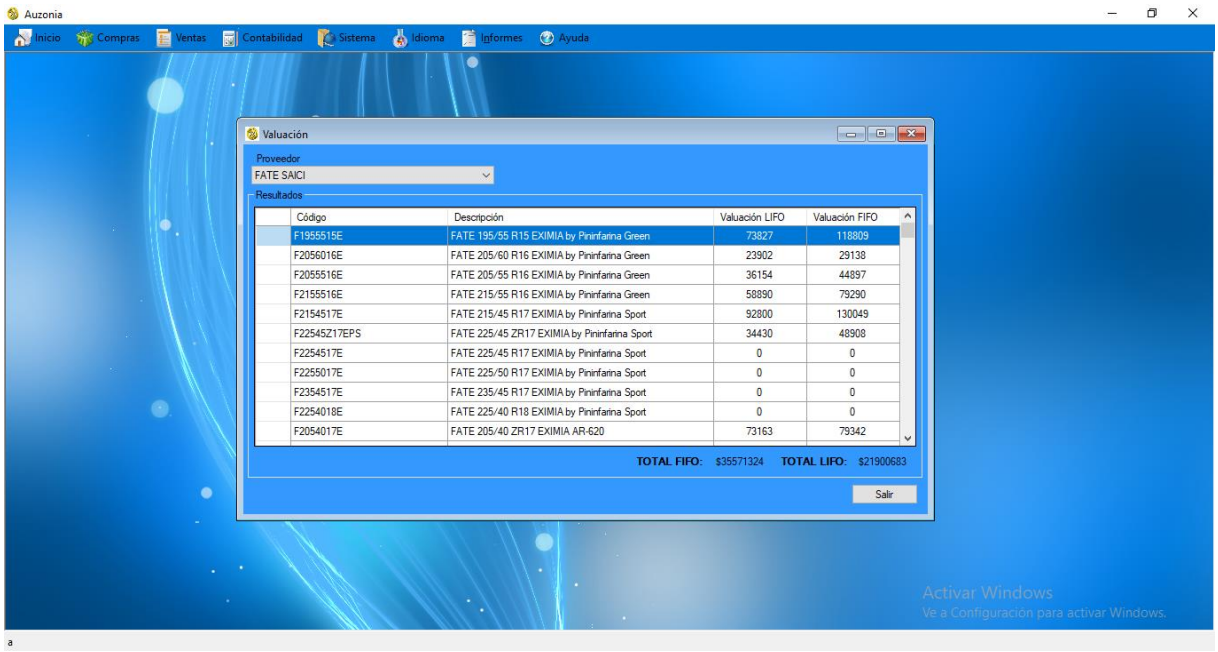
6.18 **P1141-Articulo**



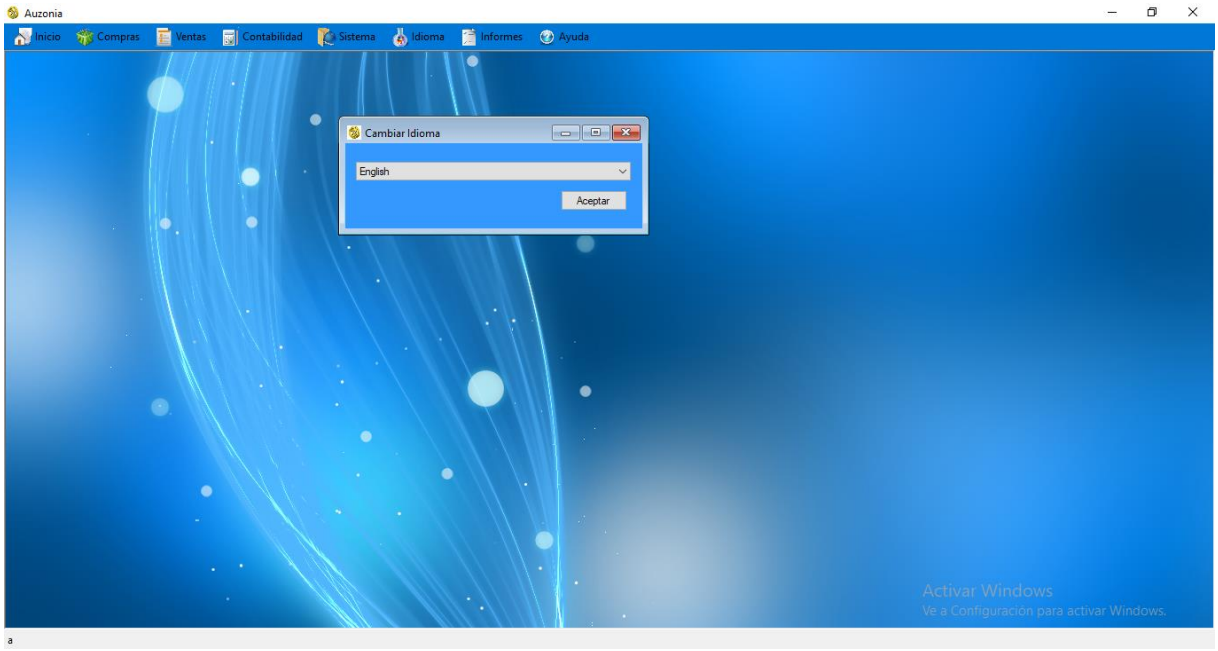
6.19 **P1142-Lista de Precios**



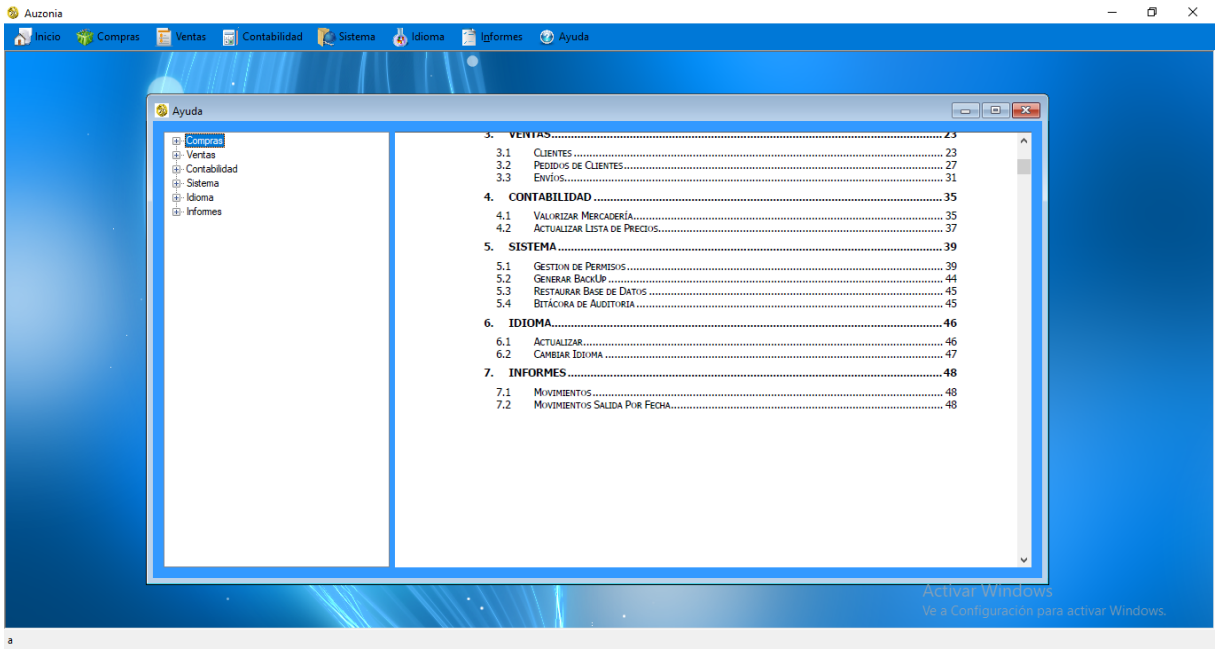
6.20 **P1143-Valuacion de Inventario**



6.21 **P115-Cambiar Idioma**

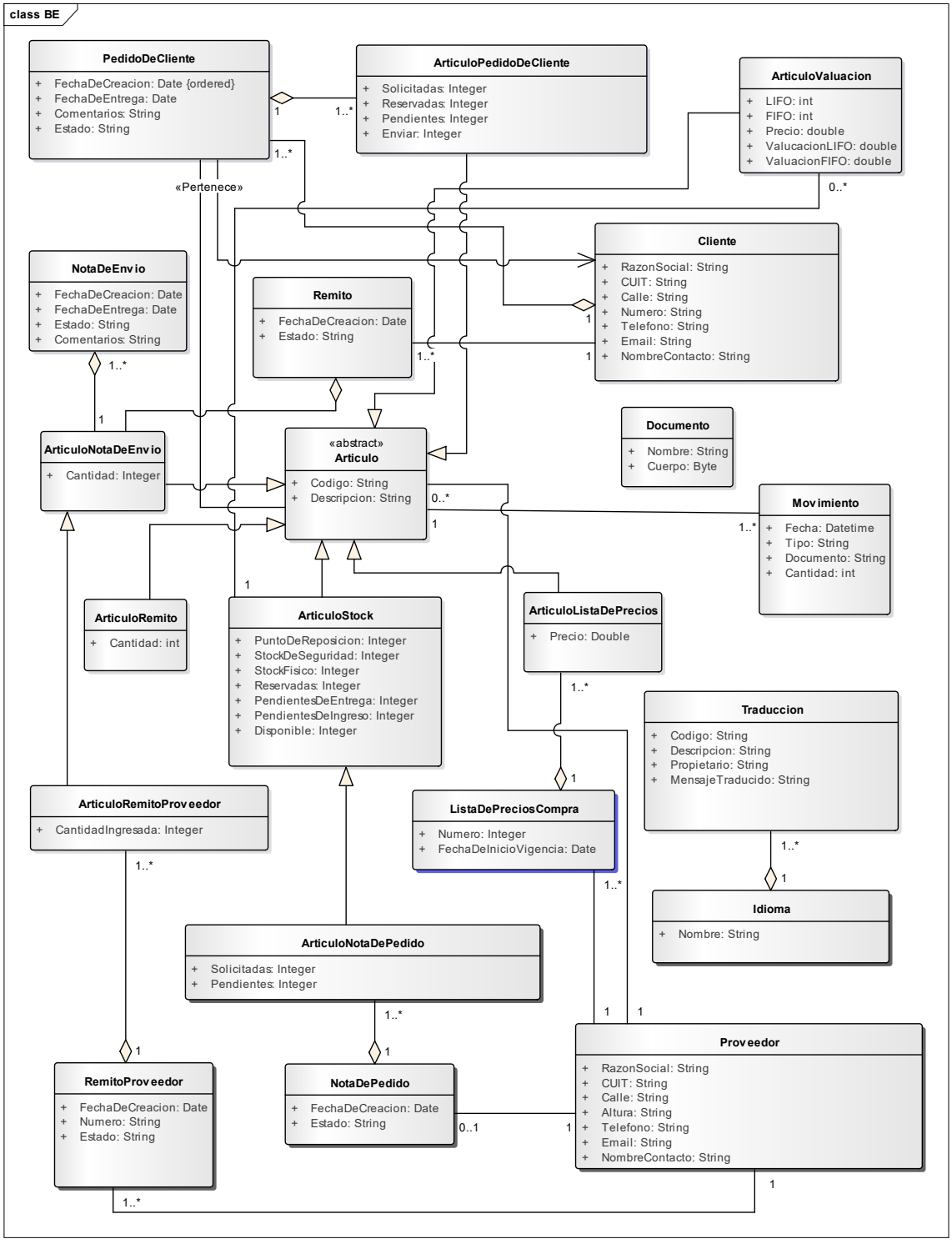


6.22 **P116-Ayuda**

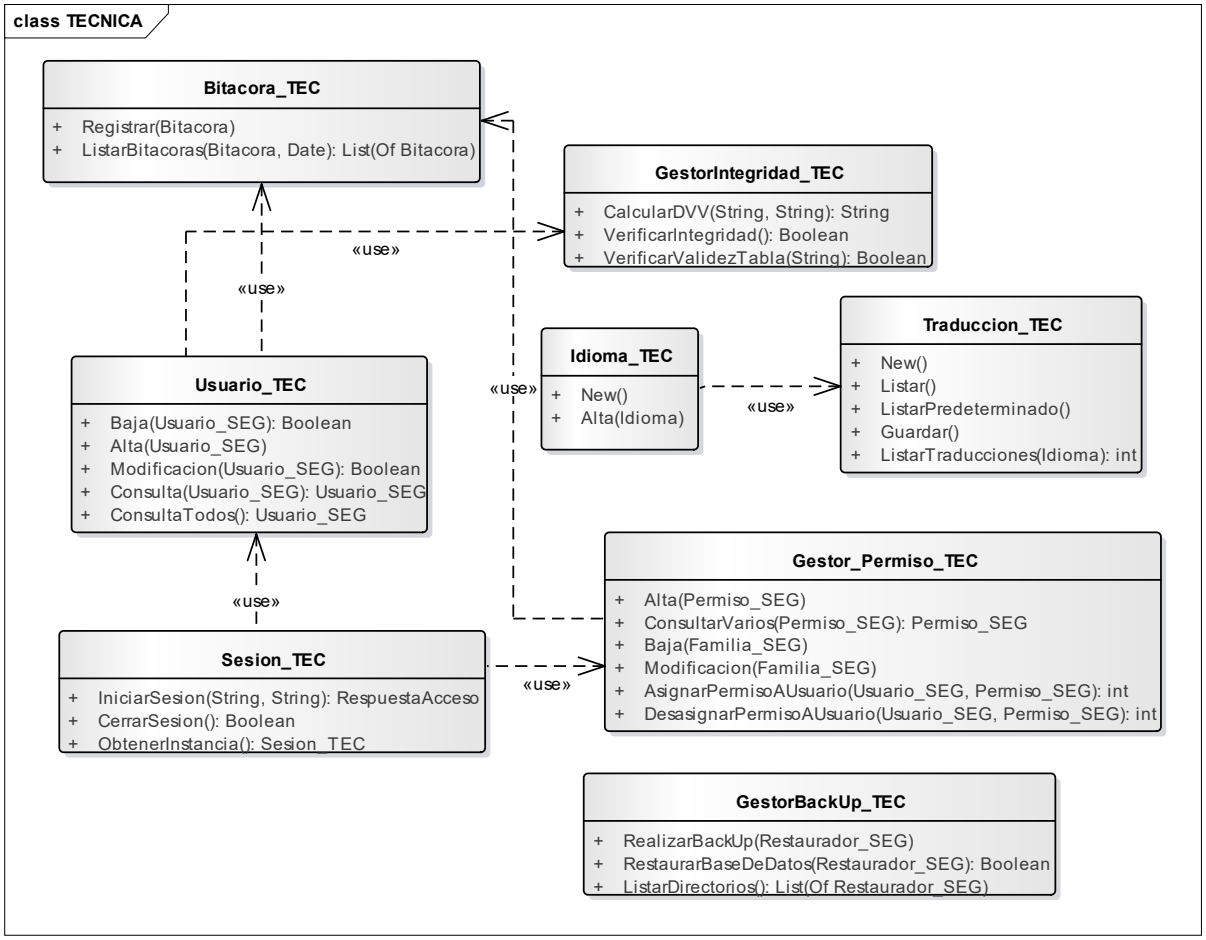


7. Diagrama Global de Clases del Sistema

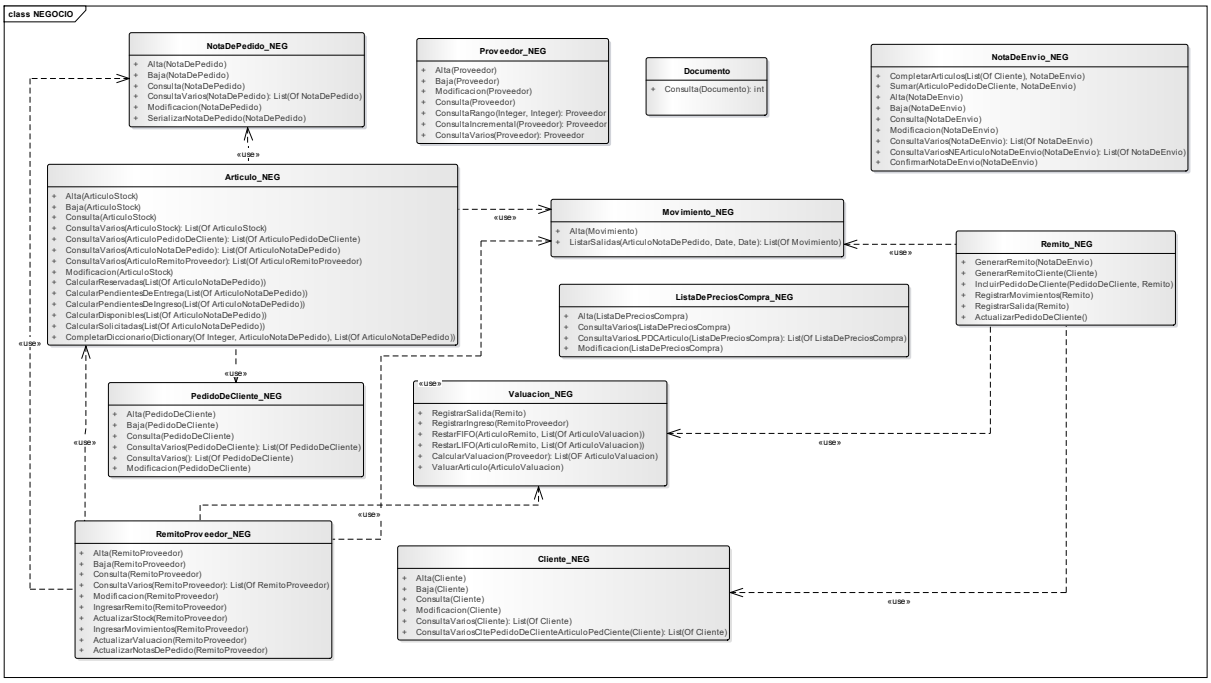
7.1 Capa: BE



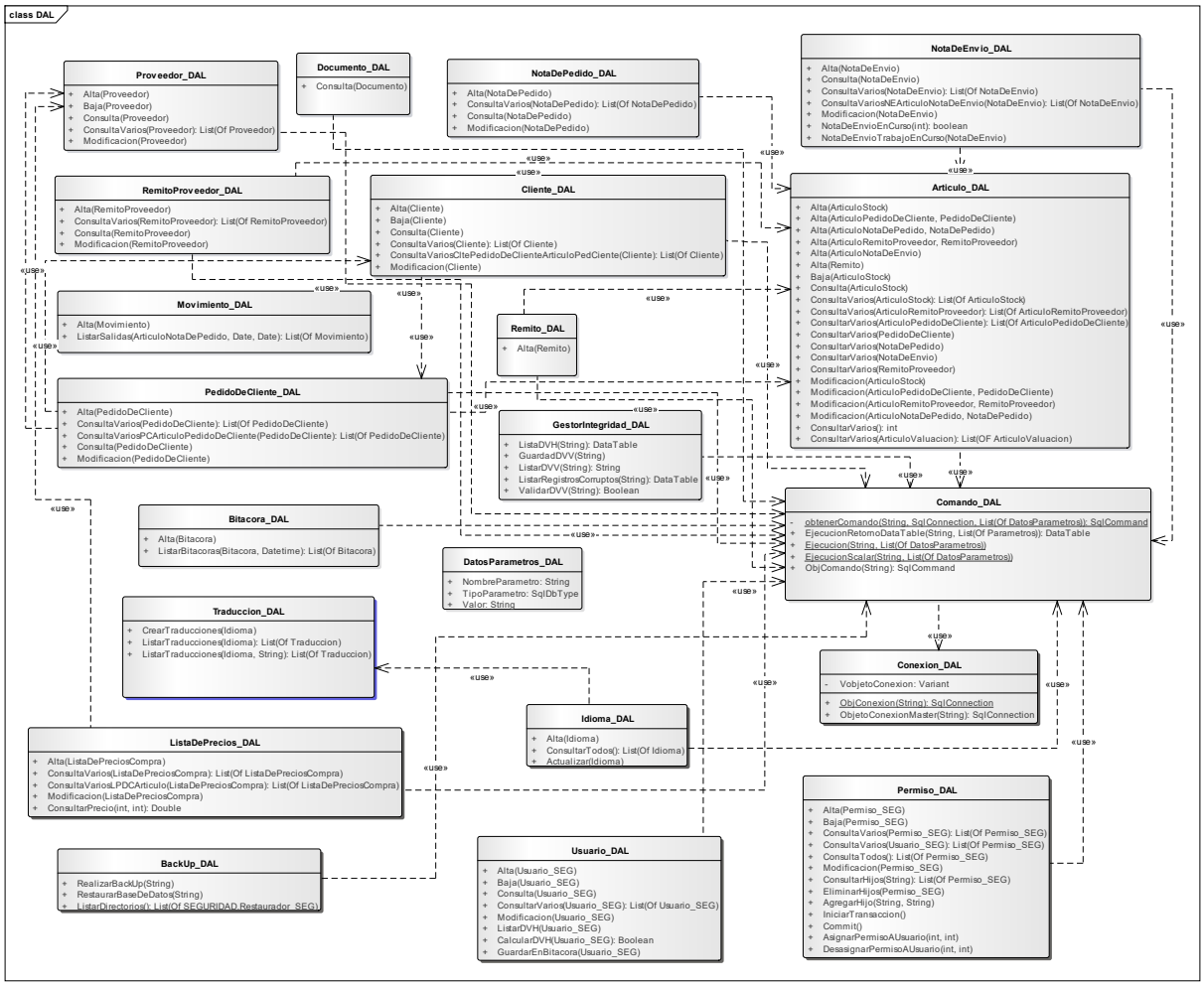
7.2 Capa: BLL Técnica



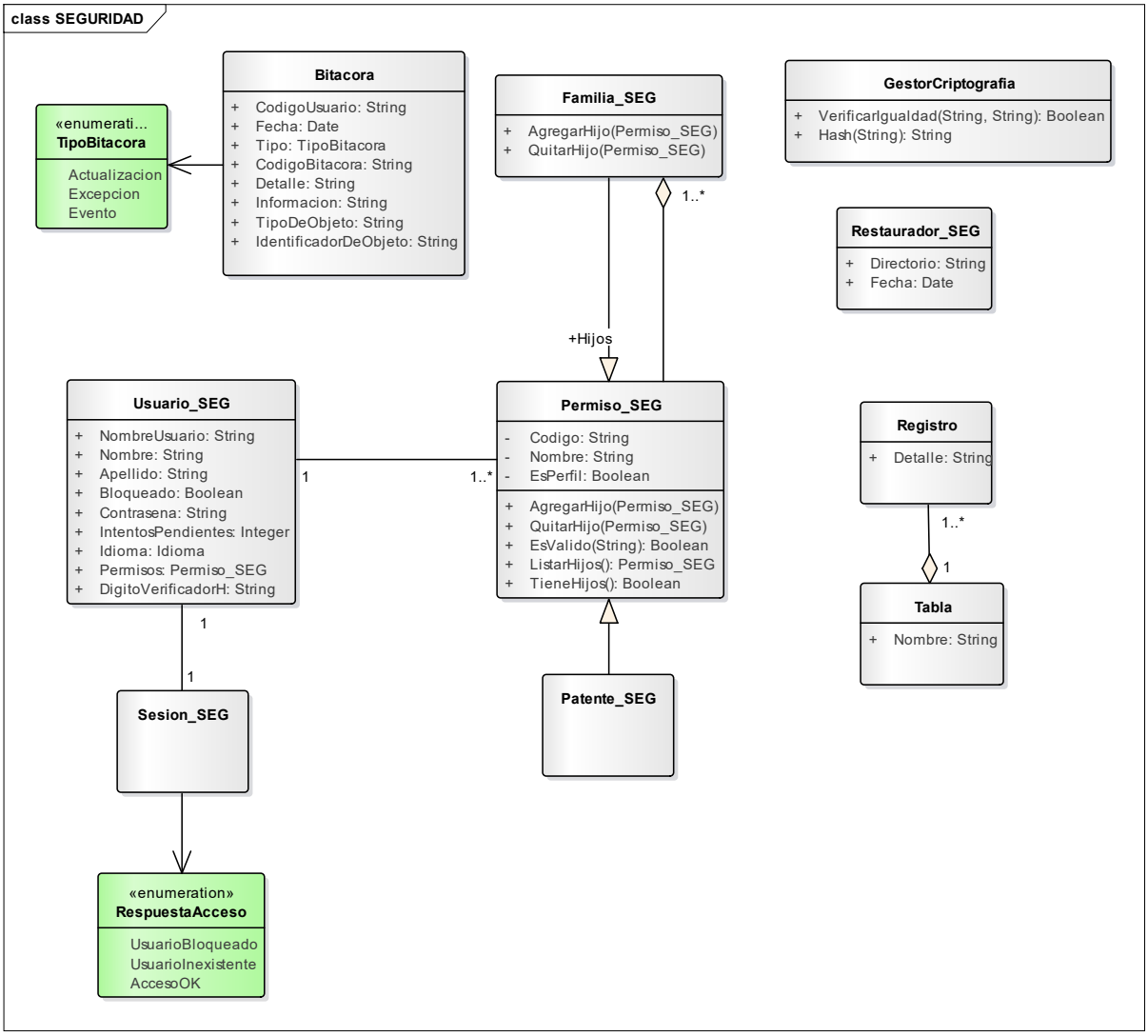
7.3 Capa: BLL Negocio



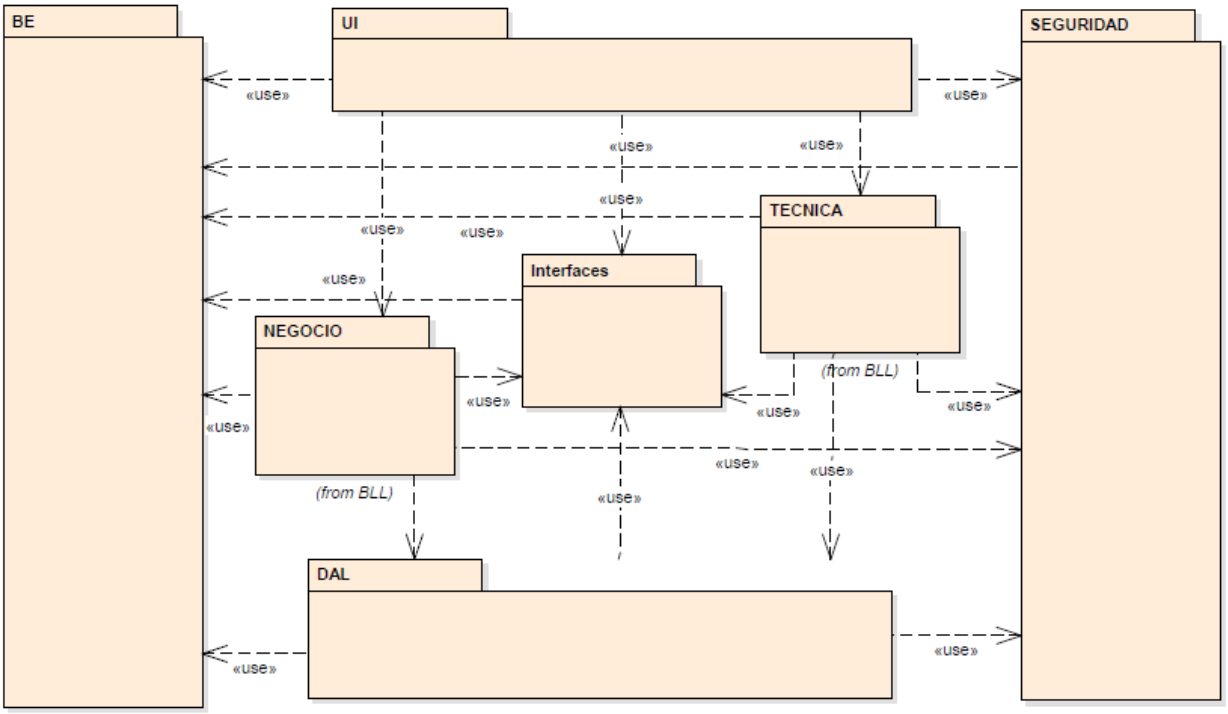
7.4 Capa: DAL


class DAL
Proveedor_DAL
+ Alta(Proveedor)
+ Baja(Proveedor)
+ Consulta(Proveedor)
+ ConsultaVarios(Proveedor): List(Of Proveedor)
+ Modificacion(Proveedor)
Documento_DAL
+ Consulta(Documento)
NotaDePedido_DAL
+ Alta(NotaDePedido)
+ ConsultaVarios(NotaDePedido): List(Of NotaDePedido)
+ Consulta(NotaDePedido)
+ Modificacion(NotaDePedido)
NotaDeEnvio_DAL
+ Alta(NotaDeEnvio)
+ Consulta(NotaDeEnvio)
+ ConsultaVarios(NotaDeEnvio): List(Of NotaDeEnvio)
+ ConsultaVariosNEArticuloNotaDeEnvio(NotaDeEnvio): List(Of NotaDeEnvio)
+ Modificacion(NotaDeEnvio)
+ NotaDeEnvioEnCurso(int): boolean
+ NotaDeEnvioTrabajoEnCurso(NotaDeEnvio)
RemitoProveedor_DAL
+ Alta(RemitoProveedor)
+ ConsultaVarios(RemitoProveedor): List(Of RemitoProveedor)
+ Consulta(RemitoProveedor)
+ Modificacion(RemitoProveedor)
Cliente_DAL
+ Alta(Cliente)
+ Baja(Cliente)
+ Consulta(Cliente)
+ ConsultaVarios(Cliente): List(Of Cliente)
+ ConsultaVariosCltePedidoDeClienteArticuloPedCliente(Cliente): List(Of Cliente)
+ Modificacion(Cliente)
Articulo_DAL
+ Alta(ArticuloStock)
+ Alta(ArticuloPedidoDeCliente, PedidoDeCliente)
+ Alta(ArticuloNotaDePedido, NotaDePedido)
+ Alta(ArticuloRemitoProveedor, RemitoProveedor)
+ Alta(ArticuloNotaDeEnvio)
+ Alta(Remito)
+ Baja(ArticuloStock)
+ Consulta(ArticuloStock)
+ ConsultaVarios(ArticuloStock): List(Of ArticuloStock)
+ ConsultaVarios(ArticuloRemitoProveedor): List(Of ArticuloRemitoProveedor)
+ ConsultarVarios(ArticuloPedidoDeCliente): List(Of ArticuloPedidoDeCliente)
+ ConsultarVarios(PedidoDeCliente)
+ ConsultarVarios(NotaDePedido)
+ ConsultarVarios(NotaDeEnvio)
+ ConsultarVarios(RemitoProveedor)
+ Modificacion(ArticuloStock)
+ Modificacion(ArticuloPedidoDeCliente, PedidoDeCliente)
+ Modificacion(ArticuloRemitoProveedor, RemitoProveedor)
+ Modificacion(ArticuloNotaDePedido, NotaDePedido)
+ ConsultarVarios(): int
+ ConsultarVarios(ArticuloValuacion): List(OF ArticuloValuacion)
Movimiento_DAL
+ Alta(Movimiento)
+ ListarSalidas(ArticuloNotaDePedido, Date, Date): List(Of Movimiento)
Remito_DAL
+ Alta(Remito)
PedidoDeCliente_DAL
+ Alta(PedidoDeCliente)
+ ConsultaVarios(PedidoDeCliente): List(Of PedidoDeCliente)
+ ConsultaVariosPCArticuloPedidoDeCliente(PedidoDeCliente): List(Of PedidoDeCliente)
+ Consulta(PedidoDeCliente)
+ Modificacion(PedidoDeCliente)
GestorIntegridad_DAL
+ ListaDVH(String): DataTable
+ GuardadDVV(String)
+ ListarDVV(String): String
+ ListarRegistrosCorruptos(String): DataTable
+ ValidarDVV(String): Boolean
Comando_DAL
+ obtenerComando(String, SqlConnection, List(Of DatosParametros)): SqlCommand
+ EjecucionRetornoDataTable(String, List(Of Parametros)): DataTable
+ Ejecucion(String, List(Of DatosParametros))
+ EjecucionScalar(String, List(Of DatosParametros))
+ ObjComando(String): SqlCommand
Bitacora_DAL
+ Alta(Bitacora)
+ ListarBitacoras(Bitacora, Datetime): List(Of Bitacora)
DatosParametros_DAL
+ NombreParametro: String
+ TipoParametro: SqlDbType
+ Valor: String
Traduccion_DAL
+ CrearTraducciones(Idioma)
+ ListarTraducciones(Idioma): List(Of Traduccion)
+ ListarTraducciones(Idioma, String): List(Of Traduccion)
Conexion_DAL
- VobjetoConexion: Variant
+ ObjConexion(String): SqlConnection
+ ObjetoConexionMaster(String): SqlConnection
Idioma_DAL
+ Alta(Idioma)
+ ConsultarTodos(): List(Of Idioma)
+ Actualizar(Idioma)
ListaDePrecios_DAL
+ Alta(ListaDePreciosCompra)
+ ConsultaVarios(ListaDePreciosCompra): List(Of ListaDePreciosCompra)
+ ConsultaVariosLPDCArticulo(ListaDePreciosCompra): List(Of ListaDePreciosCompra)
+ Modificacion(ListaDePreciosCompra)
+ ConsultarPrecio(int, int): Double
Permiso_DAL
+ Alta(Permiso_SEG)
+ Baja(Permiso_SEG)
+ ConsultaVarios(Permiso_SEG): List(Of Permiso_SEG)
+ ConsultaVarios(Usuario_SEG): List(Of Permiso_SEG)
+ ConsultaTodos(): List(Of Permiso_SEG)
+ Modificacion(Permiso_SEG)
+ ConsultarHijos(String): List(Of Permiso_SEG)
+ EliminarHijos(Permiso_SEG)
+ AgregarHijo(String, String)
+ IniciarTransaccion()
+ Commit()
+ AsignarPermisoAUsuario(int, int)
+ DesasignarPermisoAUsuario(int, int)
BackUp_DAL
+ RealizarBackUp(String)
+ RestaurarBaseDeDatos(String)
+ ListarDirectorios(): List(Of SEGURIDAD.Restaurador_SEG)
Usuario_DAL
+ Alta(Usuario_SEG)
+ Baja(Usuario_SEG)
+ Consulta(Usuario_SEG)
+ ConsultarVarios(Usuario_SEG): List(Of Usuario_SEG)
+ Modificacion(Usuario_SEG)
+ ListarDVH(Usuario_SEG)
+ CalcularDVH(Usuario_SEG): Boolean
+ GuardarEnBitacora(Usuario_SEG)
«use»

7.5 Capa: Seguridad



8. Arquitectura



9. DER Global del Sistema

